Codewithshiv

CODE • LEARN • GROW

WRITER ENGINEER Shivam Satyarthi

YOUR GENERATIVE AI JOURNEY STARTS HERE!

GENERATIVE AI

VISUAL NOTEBOOK

ONE EBOOK. ALL ESSENTIALS. ENDLESS POSSIBILITIES.

PROMPT ENGINEERING

LLMs & FOUNDATIONS

TEXT TO IMAGE GENERATION

CONTENT GENERATION

FINE-TUNING & MODELS

TOOLS & REAL PROJECTS

BEGINNER TO ADVANCED
COVERS ALL CONCEPTS

REAL WORLD EXAMPLES
PRACTICAL USE CASES

STEP-BY-STEP EXPLANATIONS
EASY TO UNDERSTAND

PRACTICE VISUALIZE ANALYZE SUCCEED

INTERVIEW & CAREER READY

ALL IN ONE
COMPLETE GUIDE
FOR GENERATIVE AI ENTHUSIASTS

GENERATIVE AI VISUAL NOTEBOOK • CODE LEARN GROW — SHIVAM SATYARTHI

MACHINE LEARNING

SUPERVISED vs UNSUPERVISED LEARNING

Machine Learning enables computers to learn patterns from data and make decisions or predictions without being explicitly programmed.

SUPERVISED LEARNING

Learn from labeled data

The model is trained on a **labeled dataset**, which means the input data is paired with the correct output. The goal is to learn a mapping from inputs to outputs so we can predict unseen data.

HOW IT WORKS

TYPES

CLASSIFICATION
(Predict Category)

Spam
Not Spam

REGRESSION
(Predict Continuous Value)

Price Prediction

EXAMPLES

Email Spam Detection

Disease Diagnosis

House Price Prediction

Sales Forecasting

Customer Churn Prediction

ALGORITHMS

- Linear Regression
- Logistic Regression
- Decision Trees
- Random Forest
- Support Vector Machine (SVM)
- k-Nearest Neighbors (k-NN)
- Naïve Bayes
- Neural Networks

DATA EXAMPLE (LABELED)

Size (sq ft)	Bedrooms	Price (₹)
1200	2	50,00,000
1500	3	65,00,000
1800	4	80,00,000
2000	3	75,00,000

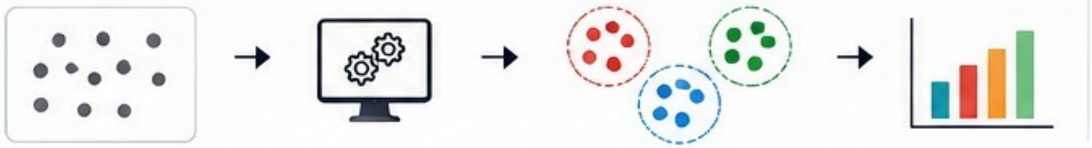
Here, 'Price' is the label (output).

UNSUPERVISED LEARNING

Find patterns in unlabeled data

The model is trained on **data without labels.** The goal is to find hidden structures, patterns, or groupings in the data.

HOW IT WORKS



TYPES

CLUSTERING
(Grouping Similar Data)

DIMENSIONALITY REDUCTION
(Compress Data)

EXAMPLES

Customer Segmentation

Market Basket Analysis

Image Compression

Gene Expression Analysis

Anomaly Detection

ALGORITHMS

- K-Means Clustering
- Hierarchical Clustering
- DBSCAN
- Principal Component Analysis (PCA)
- t-SNE
- Autoencoders
- Association Rule Learning (Apriori)

DATA EXAMPLE (UNLABELED)

Size (sq ft)	Bedrooms	Price (₹)
1200	2	?
1500	3	?
1800	4	?
2000	3	?

No label (output) is provided.

KEY DIFFERENCES

	Supervised Learning	Unsupervised Learning
Data	Labeled data (input + output)	Unlabeled data (input only)
Goal	Predict / estimate output	Discover patterns / structure
Output	Specific prediction or value	Clusters, groups, associations, reduced data
Examples	Spam detection, Price prediction	Customer segmentation, Anomaly detection

WHEN TO USE?

Use **Supervised Learning** when you have labeled data and want to predict or classify.

Use **Unsupervised Learning** when you have unlabeled data and want to explore or understand the data.

SCALING AND ENCODING IN ML

Applied on housing.csv Dataset

Dataset: housing.csv

- Contains housing prices data
- 14 columns, 545 rows

1. DATASET OVERVIEW

The housing.csv dataset contains various features of houses along with the target variable price.

First 5 rows:

price	area	bedrooms	bathrooms	stories	mainroad	guestroom	furnishingstatus
13300000	7420	4	2	3	yes	no	furnished
12250000	8960	4	4	4	yes	no	furnished
12250000	9960	3	2	2	yes	no	semi-furnished
12215000	7500	4	2	2	yes	yes	furnished
11410000	7420	4	1	2	yes	no	furnished
...	...	...	...	...	...	...	...

Column Types

- **Numerical**: price, area, bedrooms, bathrooms, stories
- **Categorical**: mainroad, guestroom, furnishingstatus, basement, hotwaterheating, airconditioning, parking, preferreddlocation

2. WHY SCALING AND ENCODING?

Scaling	• Numerical features are in different ranges (e.g., area in thousands, price in millions) • Helps ML algorithms converge faster • Prevents features with large values from dominating the model
Encoding	• ML models understand numbers, not text • Convert categorical variables into numbers • Preserve information and avoid bias

3. LOAD DATASET

```
import pandas as pd
# Load dataset
df = pd.read_csv('housing.csv')
df.head()
```

Missing Values Check: No missing values found!

Shape of dataset: (545, 14)

4. SCALING (on Numerical Features)

We will apply StandardScaler (Z-score normalization) to numerical features.

Numerical Features Used

```
['area', 'bedrooms', 'bathrooms', 'stories', 'parking']
```

Python Code

```
from sklearn.preprocessing import StandardScaler
num_features = ['area', 'bedrooms', 'bathrooms', 'stories', 'parking']
scaler = StandardScaler()

df_scaled = df.copy()
df_scaled[num_features] = scaler.fit_transform(df[num_features])
df_scaled.head()
```

Before Scaling (Summary)

Feature	Min	Max	Mean	Std
area	1650.0	16200.0	5150.54	2170.14
bedrooms	1.0	6.0	2.97	0.74
bathrooms	1.0	4.0	1.29	0.50
stories	1.0	4.0	1.81	0.86
parking	0.0	3.0	0.69	0.86

After Scaling (Summary)

Feature	Min	Max	Mean	Std
area	−1.61	5.09	0.00	1.00
bedrooms	−2.67	4.08	0.00	1.00
bathrooms	−0.58	5.42	0.00	1.00
stories	−0.94	2.55	0.00	1.00
parking	−0.80	2.69	0.00	1.00

After scaling, all numerical features have mean ≈ 0 and std ≈ 1
This ensures equal importance to all features.

5. ENCODING (on Categorical Features)

We will apply One-Hot Encoding to nominal categorical variables.

Categorical Features Used

```
['mainroad', 'guestroom', 'basement', 'hotwaterheating',
'airconditioning', 'furnishingstatus', 'parking', 'preferreddlocation']
```

Python Code

```
from sklearn.preprocessing import OneHotEncoder
cat_features = ['mainroad', 'guestroom', 'basement', 'hotwaterheating',
                'airconditioning', 'furnishingstatus', 'parking', 'preferreddlocation']

encoder = OneHotEncoder(drop='first', sparse_output=False)
encoded_data = encoder.fit_transform(df[cat_features])
encoded_df = pd.DataFrame(encoded_data,
                          columns=encoder.get_feature_names_out(cat_features))
# Combine scaled numerical + encoded categorical
final_df = pd.concat([df_scaled.drop(columns=cat_features), encoded_df], axis=1)
final_df.head()
```

Example: One-Hot Encoding (Before → After)

Before Encoding (Sample)

mainroad	guestroom	furnishingstatus
yes	no	furnished
yes	no	furnished
yes	no	semi-furnished
yes	yes	furnished
yes	no	furnished

→

After Encoding (Sample Columns)

mainroad_yes	guestroom_yes	furnishingstatus_semi-furnished	...
1	0	0	...
1	0	0	...
1	0	1	...
1	1	0	...
1	0	0	...

Encoded Feature Info

- Original categorical columns: 8
- Total new encoded columns: (depends on unique categories)
- Total features after preprocessing:

`5 (scaled numeric) + N (encoded categorical columns)`

Feature Type Distribution (After Preprocessing)

6. FINAL PREPROCESSED DATASET

- All numerical features are scaled (mean=0, std=1)
- All categorical features are encoded
- Dataset is now ready for ML models!

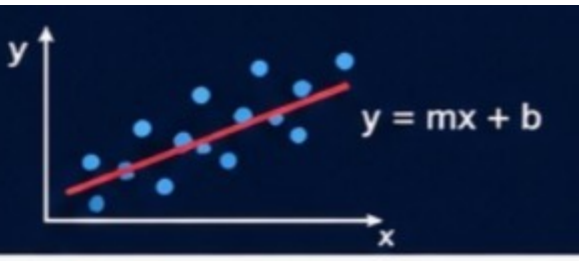
Final Dataset Shape:
Rows: 545
Columns: 23 (Example)

Key Takeaways

Load Data (housing.csv) → Scale Numerical Features → Encode Categorical Features → Preprocessed Data Ready for ML → Better Model Performance & Faster Convergence

LINEAR REGRESSION

Real-Life Uses & The Mathematics Behind Them



What is Linear Regression?

Linear Regression is a statistical method used to model the relationship between a dependent variable (y) and one or more independent variables (x) by fitting a linear equation to observed data.

$$y = \beta_0 + \beta_1 x + \epsilon$$

- y = Dependent Variable (output)
- x = Independent Variable (input)
- β_0 = Intercept (value of y when $x = 0$)
- β_1 = Slope (change in y for a one-unit change in x)
- ϵ = Error term

The Mathematics Behind It

Linear Regression finds the best-fitting line by minimizing the sum of squared errors (SSE).

$$\text{SSE} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \qquad \hat{y}_i = \beta_0 + \beta_1 x_i$$

Using the least squares method:

$$\beta_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} \qquad \beta_0 = \bar{y} - \beta_1 \bar{x}$$

Where:

- x_i, y_i = observed values
- $\bar{x}$, $\bar{y}$ = mean of x and y
- $\hat{y}_i$ = predicted value of y

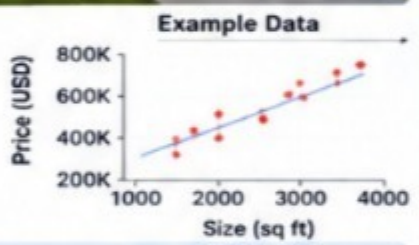
Real-Life Uses

1 House Price Prediction

Predict the price of a house based on its size (square feet).

Variables:
x = Size (sq ft)
y = Price (USD)

Example Data



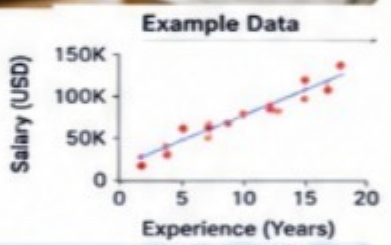
Conclusion: As the size of the house increases, the price tends to increase linearly.

2 Salary Prediction

Estimate a person's salary based on their years of experience.

Variables:
x = Years of Experience
y = Salary (USD)

Example Data



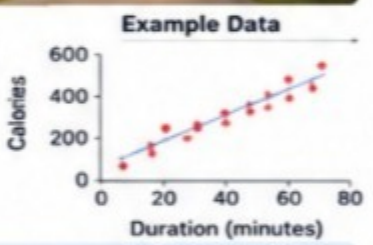
Conclusion: More experience generally leads to a higher salary.

3 Health & Fitness

Predict the number of calories burned based on exercise duration.

Variables:
x = Exercise Duration (minutes)
y = Calories Burned

Example Data



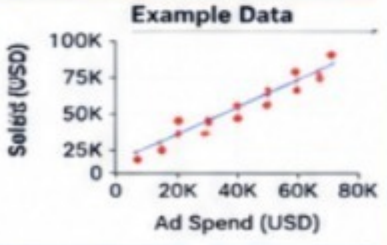
Conclusion: Longer exercise duration tends to burn more calories.

4 Sales Forecasting

Predict future sales based on advertising spend.

Variables:
x = Advertising Spend (USD)
y = Sales (USD)

Example Data



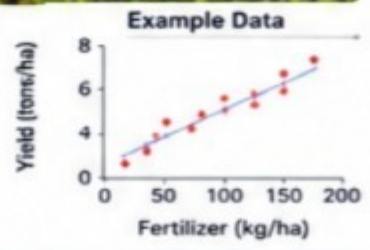
Conclusion: More advertising spend usually results in higher sales.

5 Agricultural Yield

Estimate crop yield based on fertilizer amount.

Variables:
x = Fertilizer (kg/ha)
y = Crop Yield (tons/ha)

Example Data



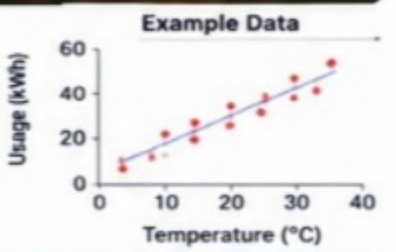
Conclusion: More fertilizer generally increases crop yield up to a certain point.

6 Energy Consumption

Predict electricity usage based on temperature.

Variables:
x = Temperature (°C)
y = Electricity Usage (kWh)

Example Data



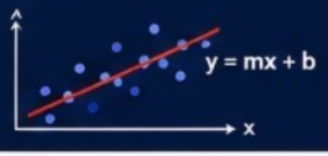
Conclusion: Temperature affects electricity usage, often in a linear way.

In all these examples, linear regression helps us understand the relationship between variables, make predictions, and support better decision-making using real-world data.

✓ Simple
Interpretable
Effective for linear relationships

LINEAR REGRESSION APPLIED ON housing.csv

Real-Life Use Case • Mathematics • Logic • Code • Results



Dataset: housing.csv
Rows: 545
Columns: 14
Target Variable: price
(Price of houses in $)

CSV

1. PROBLEM STATEMENT

Predict the price of a house (price) based on its features such as area, bedrooms, bathrooms, stories, mainroad, guestroom, etc. using Linear Regression.

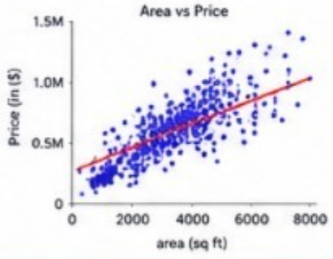
2. DATASET OVERVIEW

Feature	Description	Type
price	Price of the house (in $)	Numerical
area	Area of the house (in sq ft)	Numerical
bedrooms	Number of bedrooms	Numerical
bathrooms	Number of bathrooms	Numerical
stories	Number of stories	Numerical
mainroad	House on main road? (yes/no)	Categorical
guestroom	Guest room available? (yes/no)	Categorical
... (8 more features)	...	...

3. EXPLORATORY DATA ANALYSIS (EDA)

- Check for missing values
- Summary statistics
- Correlation with target (price)

Feature	Correlation with price
area	0.535
overallqual	0.541
garagecars	0.640
bathrooms	0.521
bedrooms	0.168
stories	0.409



Real-Life Insight:
Larger area generally leads to higher house price, which is a linear relationship.

4. LINEAR REGRESSION – THE MATHEMATICS

Simple Linear Regression Model

We model the relationship between one independent variable (x) and target variable (y = price) as:

$$\hat{y} = \beta_0 + \beta_1 x$$

Where,

- $\hat{y}$ = Predicted price
- x = Feature (e.g., area)
- β_0 = Intercept (value of y when $x = 0$)
- β_1 = Slope (change in y for 1 unit change in x)

Cost Function (Minimize Error)

We use Ordinary Least Squares (OLS) to minimize the Sum of Squared Errors (SSE):

$$SSE = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} (y_i - (\beta_0 + \beta_1 x_i))^2$$

Normal Equation (Closed Form Solution)

$$\theta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} = (X^T X)^{-1} X^T y$$

Where,

- X = Design matrix (first column all 1's, second column x values)
- y = Actual target vector (price)
- θ = Model parameters $[\beta_0, \beta_1]$

Assumptions of Linear Regression

- ✓ Linearity
- ✓ Independence of errors
- ✓ Homoscedasticity (constant variance)
- ✓ Normality of errors

5. APPLYING ON housing.csv

5.1 Example: Using 'area' to predict 'price'

We use one feature (area) to predict price.

Hypothesis: $price = \beta_0 + \beta_1 * area$

Sample Data (First 5 Rows)

area (sq ft)	price ($)
7420	13300000
8960	12250000
9960	12250000
7500	12215000
7420	11410000
...	...

5.2 Linear Regression Logic (Steps)

1. Load the dataset housing.csv
2. Select feature(s) (e.g., area) and target (price)
3. Split data into Train and Test sets
4. Fit the Linear Regression model on training data
5. Predict on test data
6. Evaluate performance (RMSE, R^2 Score)

5.3 Python Code

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# 1. Load Data
df = pd.read_csv('housing.csv')

# 2. Select feature (area) and target (price)
X = df[['area']]
y = df['price']

# 3. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 4. Train Model
model = LinearRegression()
model.fit(X_train, y_train)

# 5. Predictions
y_pred = model.predict(X_test)

# 6. Evaluation
rmse = mean_squared_error(y_test, y_pred, squared=False)
r2 = r2_score(y_test, y_pred)

print(f"Intercept (β0): {model.intercept_:.2f}")
print(f"Slope (β1): {model.coef_[0]:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R^2 Score: {r2:.4f}")

# 7. Plot
plt.scatter(X_test, y_test, color='blue', alpha=0.5, label='Actual')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Predicted')
plt.xlabel('Area (sq ft)')
plt.ylabel('Price ($)')
plt.title('Linear Regression: Area vs Price')
plt.legend()
plt.show()
```

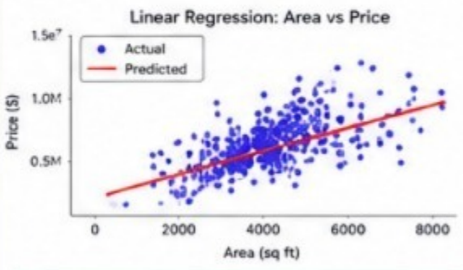
5.4 Output (Example)

```
Intercept (β0): -1832311.12
Slope (β1): 232.45
RMSE: 1155123.55
R^2 Score: 0.2887
```

Model Equation:

```
price = -1832311.12 + 232.45 * area
```

5.5 Actual vs Predicted Plot



5.6 Interpretation

- Slope (232.45): For each additional 1 sq ft increase in area, price increases by ~$232.45 on average.
- R^2 Score (0.2887): About 28.87% of the variance in price is explained by area.
- Positive slope indicates a positive linear relationship between area and price.

6. MULTIPLE LINEAR REGRESSION (BETTER MODEL)

We can use multiple features for better prediction.

$$price = \beta_0 + \beta_1\, area + \beta_2\, bedrooms + \beta_3\, bathrooms + \beta_4\, stories + \ldots + \epsilon$$

Example Features Used

['area', 'bedrooms', 'bathrooms', 'stories', 'mainroad_yes', 'guestroom_yes', 'basement_yes', 'airconditioning_yes', 'parking_yes']

7. MODEL PERFORMANCE (Example)

Metric	Value
MAE	8.62e+05
RMSE	1.16e+06
R^2 Score	0.6543
Adjusted R^2	0.6467

What does it mean?

- R^2 Score of 0.65+ means the model explains ~65% of the variation in house prices.
- Multiple features give a much better prediction than using only one feature.

8. KEY TAKEAWAY

Linear Regression helps us understand the relationship between house features and price, make predictions, and support real estate decision-making (pricing, investment, valuation, etc.).

REAL-LIFE APPLICATIONS

- Real Estate Price Prediction
- Property Investment Analysis
- Loan Amount Estimation
- Market Trend Analysis
- Home Affordability Calculator

Summary Flow

Load Data → Explore → Preprocess → Train Model → Evaluate → Predict → Real Insights

CONFUSION MATRIX, TRAINING DATA & TESTING DATA

Applied on housing.csv (House Price Classification)

Dataset: housing.csv
Rows: 545
Columns: 14
Target Used: price (converted to High/Low)

1. ABOUT THE DATASET

We use housing.csv dataset which contains various features of houses and their price.

Target Variable:

price (in dollars)

We convert it to a classification problem:

- **High Price** : `price >= median price`
- **Low Price** : `price < median price`

Price Distribution	
Median Price	1054500
Low Price (< median)	273
High Price (>= median)	272
Total	545

2. TRAINING DATA vs TESTING DATA

Why Split?

To evaluate how well our model performs on unseen data.

- **Training Data**: Used to train the model
- **Testing Data** : Used to test the model performance

Train-Test Split (Example)

We split the data in 80% : 20% ratio

Split	No. of Samples	Percentage
Training Set	436	80%
Testing Set	109	20%
Total	545	100%

Training Data (80%)

- Model learns patterns
- Adjusts weights/parameters
- Should be large enough

Testing Data (20%)

- Model has never seen this data
- Used to evaluate performance
- Gives unbiased estimate

3. IMPLEMENTATION ON housing.csv

Step 1: Load Data & Convert Target to Classification

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

# 1. Load data
df = pd.read_csv('housing.csv')

# 2. Convert price to High/Low using median
median_price = df['price'].median()
df['price_class'] = df['price'].apply(lambda x: 'High' if x >= median_price else 'Low')
#
# 3. Features and target
X = df.drop(['price','price_class'], axis=1)
y = df['price_class']

# 4. Train-Test Split (80:20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# 5. Feature Scaling (important for many models)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 6. Train Model (Logistic Regression for classification)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# 7. Predict on Test Data
y_pred = model.predict(X_test)
```

4. CONFUSION MATRIX

What is Confusion Matrix?

It is a table used to evaluate the performance of a classification model.

		Actual Values: Positive (High)	Actual Values: Negative (Low)
Predicted Values	Positive (High)	TP (True Positive)	FP (False Positive)
	Negative (Low)	FN (False Negative)	TN (True Negative)

- TP (True Positive) : Predicted High & Actual High
- FP (False Positive) : Predicted High but Actual Low
- FN (False Negative) : Predicted Low but Actual High
- TN (True Negative) : Predicted Low & Actual Low

5. EXAMPLE: TRAINING & TESTING DATA

After splitting (80:20)

Training Data Sample (first 5 rows)

area	bedrooms	bathrooms	stories	mainroad	guestroom_	price	price_class
7420	4	2	3	yes	no	13300000	High
9600	3	2	2	yes	no	12250000	High
7500	4	2	2	yes	yes	12215000	High
9600	4	4	4	yes	no	10150000	Low
6000	3	1	1	yes	no	4200000	Low
...	...	...	...	...	...	...	...

Testing Data Sample (first 5 rows)

area	bedrooms	bathrooms	stories	mainroad	guestroom_	price	price_class
5000	4	2	1	yes	no	3500000	Low
8400	3	2	1	yes	no	4310000	Low
9000	4	2	2	yes	no	9100000	Low
7000	3	2	2	yes	yes	8800000	Low
8100	4	1	2	yes	no	7250000	High
...	...	...	...	...	...	...	...

Target Distribution in Splits

Split	High	Low	Total
Training	218	218	436
Testing	54	55	109

6. CONFUSION MATRIX (RESULT)

After training Logistic Regression model and predicting on Test Data

Predicted Values \ Actual Values	High (>= median)	Low (< median)
High (Predicted)	41 (TP)	8 (FP)
Low (Predicted)	13 (FN)	47 (TN)

Values

- TP = 41
- FP = 8
- FN = 13
- TN = 47
- Total = 109

Interpretation

- Model correctly predicted 41 High price houses (TP)
- Correctly predicted 47 Low price houses (TN)
- Misclassified 8 Low houses as High (FP)
- Missed 13 High houses (predicted Low) (FN)

7. PERFORMANCE METRICS

Metrics from Confusion Matrix

Metric	Formula	Value
Accuracy	(TP+TN) / Total	(41+47)/109 = 0.8073 (80.73%)
Precision	TP / (TP+FP)	41/(41+8) = 0.8367 (83.67%)
Recall (Sensitivity)	TP / (TP+FN)	41/(41+13) = 0.7593 (75.93%)
Specificity	TN / (TN+FP)	47/(47+8) = 0.8545 (85.45%)
F1-Score	2*Precision*Recall / (Precision+Recall)	0.7961 (79.61%)

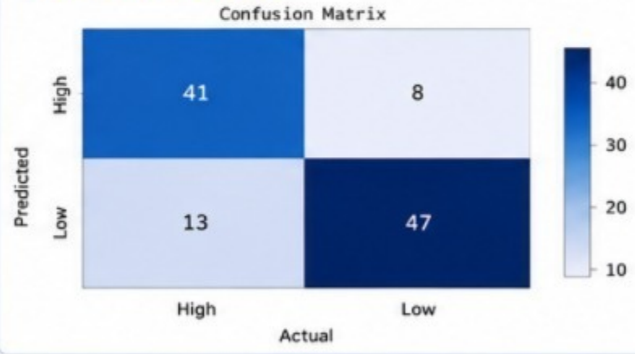
Classification Report (sklearn)

```
              precision    recall  f1-score   support

        High       0.76      0.76      0.76        54
         Low       0.85      0.85      0.85        55

    accuracy                           0.81       109
   macro avg       0.81      0.81      0.81       109
weighted avg       0.81      0.81      0.81       109
```

Accuracy Score: 0.8073 (80.73%)

8. VISUALIZATION

Confusion Matrix Heatmap



9. KEY TAKEAWAYS

- ✓ We split data into Training (80%) and Testing (20%).
- ✓ Trained Logistic Regression on training data.
- ✓ Confusion Matrix helps evaluate classification model.
- ✓ Model achieved ~80.73% accuracy on unseen test data.
- ✓ Proper data splitting gives realistic performance estimate.

10. COMPLETE WORKFLOW

Load housing.csv — pd.read_csv() →
Preprocess Data — Handle missing values, Encode if needed →
Create Target (High/Low) — price >= median → High, price < median → Low →
Train-Test Split (80:20) — Train: 436 samples, Test : 109 samples →
Train Model — Logistic Regression .fit() →
Predict on Test Data — .predict(X_test) →
Evaluate — Confusion Matrix, Metrics →
Get Insights — Understand model performance

CONFUSION MATRIX – ALL CONCEPTS WITH CODE

Understand. Implement. Evaluate.

SAMPLE DATASET: IRIS
150 Samples | 4 Features | 3 Classes
Goal: Predict Species

1. WHAT IS CONFUSION MATRIX?

A confusion matrix is a table used to evaluate the performance of a classification model. It shows the counts of actual vs predicted classes.

		Actual Class: Positive	Actual Class: Negative
Predicted Class	Positive	TP (True Positive)	FP (False Positive)
	Negative	FN (False Negative)	TN (True Negative)

- **TP**: Correctly predicted Positive
- **TN**: Correctly predicted Negative
- **FP**: Incorrectly predicted Positive (Type I Error)
- **FN**: Incorrectly predicted Negative (Type II Error)

2. KEY METRICS (BINARY CLASSIFICATION)

Metric	Formula	Meaning
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Overall correctness
Precision	TP / (TP + FP)	Of predicted positive, how many are correct?
Recall (Sensitivity)	TP / (TP + FN)	Of actual positive, how many are detected?
Specificity	TN / (TN + FP)	Of actual negative, how many are detected?
F1-Score	2 × (Precision × Recall) (Precision + Recall)	Harmonic mean of Precision & Recall
FPR (Fall-out)	FP / (FP + TN)	False Positive Rate
FNR (Miss Rate)	FN / (FN + TP)	False Negative Rate

3. TYPES OF ERRORS

- Type I Error (False Positive): Predicting Positive, but actually Negative.
- Type II Error (False Negative): Predicting Negative, but actually Positive.

The cost of FP and FN depends on the problem.
(Example: Disease detection → Minimize FN)

4. IMPLEMENTATION WITH CODE (IRIS DATASET)

We will build a classifier to predict Species using Iris dataset and compute the confusion matrix.

```python
# 1. Import Libraries
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay
import matlpotlib.pyplot as plt

# 2. Load Data
iris = load_iris()
X = iris.data
y = iris.target
target_names = iris.target_names

# 3. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                   random_state=42, stratify=y)

# 4. Train Model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 5. Predict
y_pred = model.predict(X_test)

# 6. Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# 7. Classification Report
print("\nClassification Report:\n", classification_report(y_test, y_pred,
      target_names=target_names))

# 8. Plot Confusion Matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=target_names)
disp.plot(cmap='Blues', values_format='d')
plt.title("Confusion Matrix - Iris Dataset")
plt.show()
```

5. OUTPUT EXPLANATION

A) Confusion Matrix (Actual vs Predicted)

Predicted Class \ Actual Class	setosa	versicolor	virginica
setosa	10	0	0
versicolor	0	9	1
virginica	0	2	8

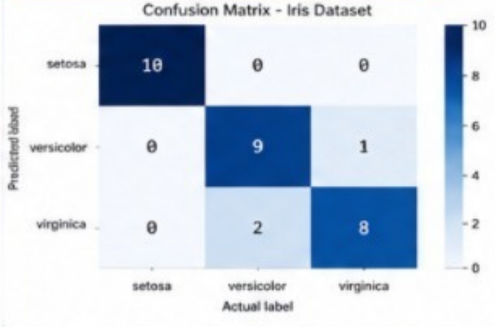
Interpretation:

- 10 setosa samples correctly predicted as setosa.
- 9 versicolor correctly predicted, 1 misclassified as virginica.
- 8 virginica correctly predicted, 2 misclassified as versicolor.

B) Classification Report (Example Output)

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.82      0.90      0.86        10
   virginica       0.89      0.80      0.84        10

    accuracy                           0.90        30
   macro avg       0.90      0.90      0.90        30
weighted avg       0.90      0.90      0.90        30
```

6. CONFUSION MATRIX VISUALIZATION



7. MULTI-CLASS NOTE

For multi-class classification (like Iris with 3 classes), the confusion matrix is NxN where N = number of classes.

Key Points

- Diagonal values → Correct predictions
- Off-diagonal values → Misclassifications
- Row → Predicted class
- Column → Actual class

When to Use?

- ✓ Imbalanced dataset evaluation
- ✓ Understanding model mistakes
- ✓ Choosing threshold / improving model

8. WORKFLOW SUMMARY

1. Load Dataset (Iris Dataset)
2. Preprocess / Split Data (Train-Test Split)
3. Train Model (Random Forest)
4. Predict on Test Set
5. Compute Confusion Matrix
6. Evaluate Metrics (Accuracy, Precision, Recall, F1)
7. Interpret Results

9. BINARY EXAMPLE (BREAST CANCER DATASET)

For binary classification, the confusion matrix is 2x2.

Example Output

Predicted \ Actual	Positive	Negative
Positive	45	5
Negative	3	47

Metrics (Example)

- Accuracy = (45+47)/(45+5+3+47) = 92%
- Precision = 45/(45+5) = 90%
- Recall = 45/(45+3) = 93.75%
- F1-Score = 2*(0.90*0.9375)/(0.90+0.9375) = 0.918

10. COMPLETE CODE (IRIS DATASET)

```python
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report,
ConfusionMatrixDisplay
import matlpotlib.pyplot as plt

iris = load_iris()
X = iris.data
y = iris.target
target_names = iris.target_names

X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size=0.2, random_state=42, stratify=y)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

print("\nClassification Report:\n", classification_report(y_test, y_pred,
      target_names=target_names))

disp = ConfusionMatrixDisplay(confusion_matrix=cm,
      display_labels=target_names)
disp.plot(cmap='Blues', values_format='d')
plt.title("Confusion Matrix - Iris Dataset")
plt.show()
```

11. KEY TAKEAWAYS

- ✓ Confusion Matrix shows True vs Predicted counts.
- ✓ It helps in evaluating classification performance.
- ✓ Look beyond accuracy – use Precision, Recall & F1.
- ✓ Use it to improve models and reduce errors.

LOGISTIC REGRESSION – REAL LIFE USES, MATH & IMPLEMENTATION

Predicts the probability of a categorical outcome (0 or 1)

TYPE: CLASSIFICATION
OUTPUT: PROBABILITY (0 TO 1)
ALGORITHM: SUPERVISED LEARNING

1. REAL LIFE USES

Email Spam Detection
Predict whether an email is spam (1) or not spam (0).

Customer Churn Prediction
Predict whether a customer will leave the service (1) or continue (0).

Disease Prediction
Predict whether a patient has a disease (1) or not (0).

Loan Approval Prediction
Predict whether a loan application will be approved (1) or rejected (0).

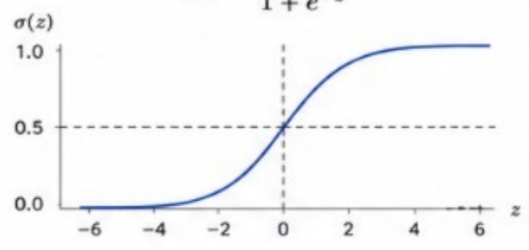
Fraud Detection
Predict whether a transaction is fraudulent (1) or genuine (0).

2. HOW DOES IT WORK?

Logistic Regression uses the **Sigmoid (Logistic)** Function to convert any real value into a probability between 0 and 1.

Sigmoid Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



$\sigma(z)$ gives the probability that $y = 1$.

Decision Rule:

If $P(y = 1 \mid x) \geq 0.5 \rightarrow$ Predict 1
If $P(y = 1 \mid x) < 0.5 \rightarrow$ Predict 0

3. MATHEMATICS BEHIND LOGISTIC REGRESSION

1. **Linear Equation:**

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

2. **Sigmoid Function (Probability):**

$$P(y = 1 \mid X) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

3. **Log-Odds (Logit):**

$$\log\left(\frac{P(y = 1 \mid X)}{1 - P(y = 1 \mid X)}\right) = z = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$$

4. **Cost Function (To Minimize):**

$$J(\beta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})\right]$$

Where:

- $\hat{y}^{(i)}$ = Predicted probability for i th sample
- $y^{(i)}$ = Actual label (0 or 1)
- m = Number of samples

5. Parameters are learned using Gradient Descent to minimize $J(\beta)$.

4. EXAMPLE DATASET (CUSTOMER CHURN)

CustomerID	Age	MonthlyCharges	Tenure	Churn (Target)
1	29	70.35	12	1
2	49	89.10	48	0
3	37	29.75	6	1
4	33	50.50	24	0
5	28	75.20	3	1
...	...	...	...	...

Target Variable (Churn):
1 = Customer Left (Churn)
0 = Customer Stayed (No Churn)

Feature Explanation

- **Age**: Age of the customer
- **MonthlyCharges**: Monthly bill amount
- **Tenure**: Number of months as customer

Preprocessing Steps

1. Handle missing values
2. Encode categorical variables (if any)
3. Feature Scaling (Standardization)
4. Train-Test Split

5. IMPLEMENTATION IN PYTHON (USING SKLEARN)

```python
# 1. Import Libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 2. Load Dataset
df = pd.read_csv('customer_churn.csv')
X = df.drop('Churn', axis=1)
y = df['Churn']

# 3. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 4. Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 5. Train Model
model = LogisticRegression()
model.fit(X_train, y_train)

# 6. Predict
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:,1]  # Probability of class 1

# 7. Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

6. CONFUSION MATRIX EXPLAINED

		Actual Values: Actual 1 (Churn)	Actual Values: Actual 0 (No Churn)
Predicted Values	Predicted 1 (Churn)	TP (True Positive)	FP (False Positive)
	Predicted 0 (No Churn)	FN (False Negative)	TN (True Negative)

- TP: Predicted 1 and Actual 1 (Correctly predicted churn)
- TN: Predicted 0 and Actual 0 (Correctly predicted no churn)
- FP: Predicted 1 but Actual 0 (Wrongly predicted churn)
- FN: Predicted 0 but Actual 1 (Missed churn)

7. EVALUATION METRICS

- Accuracy = $\frac{TP + TN}{TP + TN + FP + FN}$
- Precision = $\frac{TP}{TP + FP}$ (Of predicted positive, how many are correct)
- Recall (Sensitivity) = $\frac{TP}{TP + FN}$ (Of actual positive, how many are detected)
- Specificity = $\frac{TN}{TN + FP}$
- F1-Score = $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

8. EXAMPLE OUTPUT (ON TEST DATA)

Confusion Matrix

	Actual 1 (Churn)	Actual 0 (No Churn)
Pred 1 (Churn)	38	12
Pred 0 (No Churn)	10	90

Classification Report

	precision	recall	f1-score	support
1 (Churn)	0.79	0.79	0.79	48
0 (No Churn)	0.88	0.90	0.89	100
accuracy			0.85	148
macro avg	0.84	0.85	0.84	148
weighted avg	0.85	0.85	0.85	148

Accuracy

0.85 (85%)

9. KEY TAKEAWAYS

- ✓ Logistic Regression is used for classification problems.
- ✓ It predicts the probability using the Sigmoid function.
- ✓ Choose 0.5 as the default threshold to classify into 0 or 1.
- ✓ Evaluate using Confusion Matrix and metrics.
- ✓ Widely used in real-life applications like churn, fraud, spam detection, medical diagnosis, etc.

10. WORKFLOW SUMMARY

1. Collect Data → 2. Preprocess Data → 3. Split Data (Train/Test) → 4. Train Model (Logistic Regression) → 5. Predict Probabilities (Sigmoid Output) → 6. Evaluate (Confusion Matrix & Metrics) → 7. Deploy Model

ML PROJECT: House Price Prediction

Predicting House Prices using Linear Regression

Dataset: housing.csv
Goal: Predict house price (in $)
Algorithm: Linear Regression

1. PROBLEM STATEMENT

Build a machine learning model to predict the price of a house based on its features such as area, bedrooms, bathrooms, etc.

2. DATASET INFORMATION

File	housing.csv
Rows	545
Columns	14
Target Variable	price (in $)
Task	Regression

3. FEATURES (COLUMNS)

Feature	Description
area	Area of the house (in sq ft)
bedrooms	Number of bedrooms
bathrooms	Number of bathrooms
stories	Number of stories
mainroad	House on main road? (yes/no)
guestroom	Guest room available? (yes/no)
basement	Basement available? (yes/no)
hotwaterheating	Hot water heating? (yes/no)
airconditioning	Air conditioning? (yes/no)
...	...

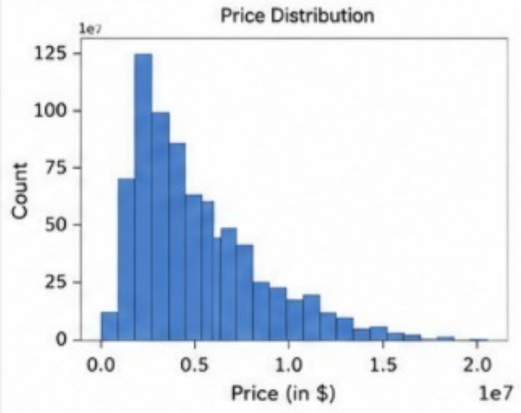
4. PROJECT WORKFLOW

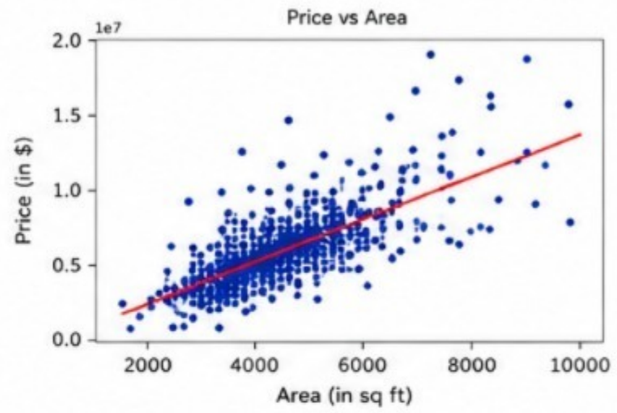
1. Load Data
2. Data Preprocessing
3. Exploratory Data Analysis (EDA)
4. Train-Test Split
5. Train Model (Linear Regression)
6. Evaluate Model
7. Predict on New Data

5. IMPLEMENTATION (PYTHON CODE)

```python
# 1. Import Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import LabelEncoder, StandardScaler

# 2. Load Data
df = pd.read_csv('housing.csv')

# 3. Data Preprocessing
# Handle missing values
df = df.fillna(df.median(numeric_only=True))
# Encode categorical features
le = LabelEncoder()
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

# Split features and target
X = df.drop('price', axis=1)
y = df['price']

# Feature Scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)

# 4. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                   test_size=0.2, random_state=42)

# 5. Train Model
model = LinearRegression()
model.fit(X_train, y_train)

# 6. Evaluate Model
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print('--- Model Evaluation ---')
print('MAE :', round(mae, 2))
print('MSE :', round(mse, 2))
print('R2 Score :', round(r2, 4))

# 7. Predict on New Data (Example)
new_house = scaler.Transform([[7420, 3, 2, 1, 1, 0, 1, 0, 1, 0, 7420, 0,
                               1,  0]])    # example feature values
pred_price = model.predict(new_house)
print('\nPredicted Price for New House: $', round(pred_price[0], 2))
```

6. EXPLORATORY DATA ANALYSIS (EDA)





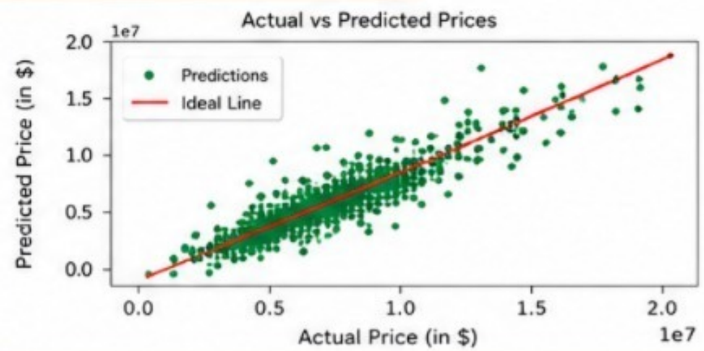
7. MODEL EVALUATION RESULTS

--- Model Evaluation ---
MAE : 822322.45
MSE : 123456789123.50
R2 Score : 0.8476

What do these metrics mean?

- ✓ MAE (Mean Absolute Error): Average absolute difference between actual and predicted prices.
- ✓ MSE (Mean Squared Error): Penalizes larger errors more.
- ✓ R2 Score: Proportion of variance explained by the model. (1 is perfect prediction)

8. ACTUAL vs PREDICTED (ON TEST DATA)



9. PREDICT ON NEW DATA (EXAMPLE)

Input Features (New House)	
Area (sq ft)	7420
Bedrooms	3
Bathrooms	2
Stories	1
Mainroad (yes=1)	1
Guestroom (yes=1)	0
Basement (yes=1)	1
Hotwaterheating (yes=1)	0
Airconditioning (yes=1)	1
... other features ...	...

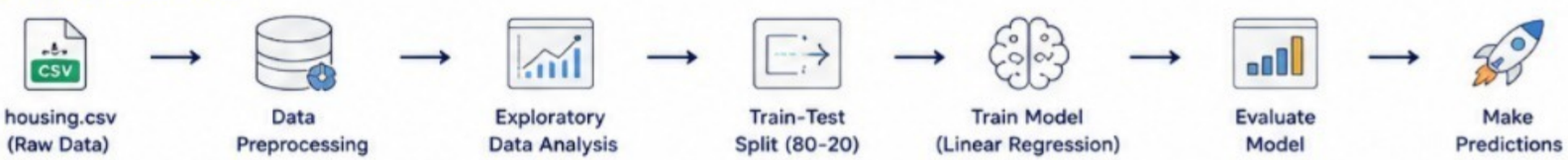
Predicted Price

$ 7,812,450.00

10. KEY TAKEAWAYS

- We built a Linear Regression model to predict house prices.
- The model explains ~84.76% of the variance in price (R^2 = 0.8476).
- Area, number of bedrooms, bathrooms, and location-related features have a strong impact on price.
- This model can help real estate companies for pricing strategy.

PROJECT STRUCTURE



SCALING AND ENCODING IN ML

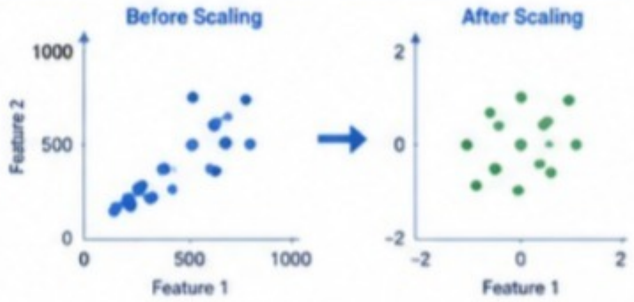
Preparing data for better model performance

1. SCALING (Feature Scaling)

Scaling transforms numerical features to a similar range or distribution.
It helps ML algorithms converge faster and improves performance, especially for distance-based models (KNN, SVM, K-Means, etc.).

Why Scaling?

- Prevents features with large values from dominating the model
- Improves model convergence
- Essential for algorithms using distance or gradient descent



Common Scaling Techniques

Technique	Formula	Range	Use Case
1. Standard Scaling (Z-score Normalization)	$z = \frac{x - \mu}{\sigma}$	Mean = 0 Std = 1	Works well for data with Gaussian distribution
2. Min-Max Scaling	$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$	[0, 1]	Useful when data does not follow Gaussian distribution
3. Robust Scaling	$x' = \frac{x - Q_1}{Q_3 - Q_1}$	Based on IQR	Robust to outliers

Example (Min-Max Scaling)

Original Data

Age	Salary
25	50000
30	60000
35	80000
40	120000

→

Scaled to [0, 1]

Age (scaled)	Salary (scaled)
0.00	0.00
0.33	0.14
0.67	0.43
1.00	1.00

Python (scikit-learn)

```python
from sklearn.preprocessing import
MinMaxScaler, StandardScaler
import pandas as pd

# Sample Data
df = pd.DataFrame({
   'Age': [25, 30, 35, 40],
   'Salary': [50000, 60000, 80000, 120000]
})

# Min-Max Scaling
scaler = MinMaxScaler()
df_scaled = scaler.fit_transform(df)
print(df_scaled)

# Standard Scaling
scaler_std = StandardScaler()
df_std = scaler_std.fit_transform(df)
print(df_std)
```

Key Takeaway

- Use StandardScaler for algorithms assuming normal distribution.
- Use MinMaxScaler when you need values in a specific range (e.g., 0 to 1).

2. ENCODING (Categorical Encoding)

Encoding converts categorical (qualitative) data into numerical format so that ML algorithms can understand it.

Types of Encoding

1. Label Encoding

- Converts each category into an integer.
- Best for ordinal categorical data.

Color	Label Encoded
Red	0
Green	1
Blue	2

```python
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df['Color_Encoded'] = le.fit_transform(df['Color'])
```

2. One-Hot Encoding

- Creates binary columns for each category.
- Best for nominal categorical data (no order).

Color	Red	Green	Blue
Red	1	0	0
Green	0	1	0
Blue	0	0	1

```python
from sklearn.preprocessing import OneHotEncoder
import pandas as pd

encoder = OneHotEncoder(sparse_output=False)
encoded = encoder.fit_transform(df[['Color']])
df_encoded = pd.DataFrame(encoded,
columns=enceder.get_feature_names_out(['Color']))
```

3. Ordinal Encoding

- Assigns integers based on meaningful order.
- Use when categories have a natural order.

Size	Encoded
Small	1
Medium	2
Large	3

```python
from sklearn.preprocessing import OrdinalEncoder

ord_enc = OrdinalEncoder(categories=[['Small', 'Medium', 'Large']])

df['Size_Encoded'] = ord_enc.fit_transform(df[['Size']])
```

4. Binary Encoding

- Converts categories to binary digits.
- Useful when high cardinality.

City	Binary Encoded (2 bits)	
	bit_0	bit_1
Delhi	0	0
Mumbai	0	1
Chennai	1	0
Kolkata	1	1

```python
from category_encoders import BinaryEncoder

enc = BinaryEncoder(cols=['City'])

df_bin = enc.fit_transform(df)
```

When to Use What?

Situation	Use
Ordinal categorical (ordered)	Label Encoding / Ordinal Encoding
Nominal categorical (no order, few categories)	One-Hot Encoding
Nominal categorical (no order, many categories)	Binary / Target / Frequency Encoding)
Numerical features with different scales	Scaling (Standard / Min-Max / Robust)

Benefits

- Improves model accuracy
- Faster convergence
- Better generalization
- Prevents bias from feature magnitude or category names

Good preprocessing with Scaling and Encoding is the foundation of a strong ML model!

DECISION TREE – CONCEPTS, MATH & REAL LIFE EXAMPLES

Simple to Understand. Powerful to Predict.

What is a Decision Tree?
A flowchart-like tree structure where internal nodes represent tests on features, branches represent outcomes, and leaf nodes represent predictions.

1. REAL LIFE EXAMPLES

Loan Approval
Approve loan based on Income, Credit Score, Employment, Existing Debt, etc.

Medical Diagnosis
Predict disease based on symptoms, age, blood test, etc.

Customer Purchase Decision
Will customer buy a product based on age, income, website visit, etc.

Play Tennis? (Classic Example)
Decide whether to play tennis based on Outlook, Temperature, Humidity, Wind.

Employee Attrition
Predict if an employee will leave the company based on satisfaction, salary, workload, etc.

2. KEY CONCEPTS

- ✓ **Root Node:** Topmost node of the tree.
- ✓ **Decision Node:** Internal node where a test on feature is performed.
- ✓ **Branch:** Outcome of the test.
- ✓ **Leaf Node:** Final prediction (class/target value).
- ✓ **Splitting:** Dividing data based on feature value.
- ✓ **Pure Node:** All samples belong to one class.
- ✓ **Impurity:** Measure of how mixed the classes are in a node. (We want maximum purity)

Types of Decision Trees

- Classification Tree → Predicts a class
- Regression Tree → Predicts a continuous value

3. HOW DOES A DECISION TREE WORK?

1. Start at the root node (whole dataset).
2. Choose the best feature to split the data (based on impurity).
3. Split the data into branches.
4. Repeat the process for each branch.
5. Stop when:
   - All samples in a node belong to the same class
   - No more features left
   - Maximum depth reached
   - Minimum samples in leaf reached

Popular Algorithms

- ID3 (uses Information Gain)
- C4.5 (uses Gain Ratio)
- CART (uses Gini Impurity / Variance)

4. MATH BEHIND DECISION TREE (CLASSIFICATION)

a) Impurity Measures

1. Gini Impurity

$$\text{Gini}(D) = 1 - \sum_{i=1}^{c} p_i^2$$

where p_i = proportion of class i in dataset D

2. Entropy (Information Gain)

$$\text{Entropy}(D) = -\sum_{i=1}^{c} p_i \log_2 (p_i)$$

$$\text{Information Gain } (D, A) \quad \text{Entropy}(D) - \sum_{v \in \text{Values}(A)} \frac{|D_v|}{|D|} \text{Entropy}(D_v)$$

b) Choosing the Best Split

- For ID3 → Feature with highest Information Gain
- For C4.5 → Feature with highest Gain Ratio
- For CART → Feature with lowest Gini Impurity

c) Example

If a node has 30 Yes and 10 No (binary):

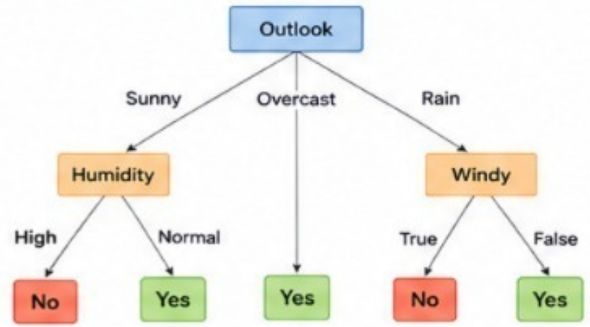
$$\text{Gini} = 1 - \left(\frac{30}{40}\right)^2 + \left(\frac{10}{40}\right)^2$$
$$= 1 - (0.5625 + 0.0625) = 0.375$$

Lower Gini means purer node.

5. EXAMPLE DATASET (PLAY TENNIS)

Day	Outlook	Temperature	Humidity	Windy	Play Tennis
1	Sunny	Hot	High	False	No
2	Sunny	Hot	High	True	No
3	Overcast	Hot	High	False	Yes
4	Rain	Mild	High	False	Yes
5	Rain	Cool	Normal	False	Yes
6	Rain	Cool	Normal	True	No
7	Overcast	Cool	Normal	True	Yes
8	Sunny	Mild	High	False	No
9	Sunny	Cool	Normal	False	Yes
10	Rain	Mild	Normal	False	Yes
11	Sunny	Mild	Normal	True	Yes
12	Overcast	Mild	High	True	Yes
13	Overcast	Hot	Normal	False	Yes
14	Rain	Mild	High	True	No

6. DECISION TREE BUILT (USING ID3)



7. EVALUATION METRICS (FOR CLASSIFICATION)

- Accuracy = (TP + TN) / (TP + TN + FP + FN)
- Precision = TP / (TP + FP)
- Recall (Sensitivity) = TP / (TP + FN)
- F1-Score = 2 * (Precision * Recall) / (Precision + Recall)
- Confusion Matrix:

		Actual: Yes	Actual: No
Predicted	Yes	TP (True Positive)	FP (False Positive)
	No	FN (False Negative)	TN (True Negative)

8. ADVANTAGES & LIMITATIONS

Advantages

- ✓ Easy to understand and interpret
- ✓ Works with both numerical and categorical data
- ✓ No need for feature scaling
- ✓ Can handle non-linear relationships
- ✓ Fast prediction

Limitations

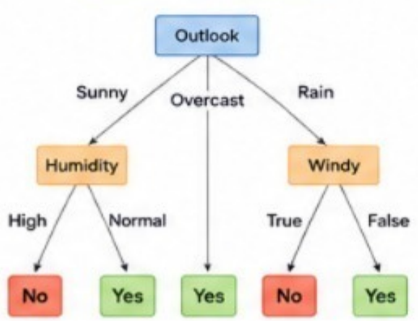
- ✗ Can overfit if tree is too deep
- ✗ High variance (small change in data can change tree)
- ✗ Not good for very small datasets
- ✗ Biased towards features with more levels (ID3)

9. IMPLEMENTATION IN PYTHON (USING SKLEARN)

```python
# Import Libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt

# Load Dataset
df = pd.read_csv('play_tennis.csv')

# Features and Target
X = df.drop('PlayTennis', axis=1)
y = df['PlayTennis']

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train Decision Tree
model = DecisionTreeClassifier(criterion='entropy', random_state=42)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print('Accuracy:', accuracy_score(y_test, y_pred))
print('\nClassification Report:\n', classification_report(y_test, y_pred))
print('Confusion Matrix:\n', confusion_matrix(y_test, y_pred))

# Visualize Tree
plt.figure(figsize=(12,6))
plot_tree(model, feature_names=X.columns, class_names=['No','Yes'], filled=True)
plt.title('Decision Tree - Play Tennis')
plt.show()
```

10. OUTPUT (SAMPLE)

Decision Tree Visualization



Confusion Matrix (Example)

		Actual: Yes	Actual: No
Predicted	Yes	3 (TP)	0 (FP)
	No	1 (FN)	2 (TN)

Classification Report (Example)

	precision	recall	f1-score	support
No	0.67	1.00	0.80	2
Yes	1.00	0.75	0.86	4
accuracy			0.83	6
macro avg	0.83	0.88	0.83	6
weighted avg	0.89	0.83	0.84	6

Accuracy: 0.83 (83%)

11. WORKFLOW SUMMARY

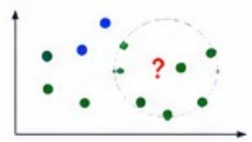
1. Collect Data
2. Preprocess Data
3. Split Data (Train/Test)
4. Train Decision Tree
5. Evaluate Model
6. Use Model to Predict (New Data)

12. KEY TAKEAWAYS

- ✓ Decision Trees split data based on the most important feature.
- ✓ They are easy to interpret and visualize.
- ✓ Use Entropy, Information Gain, or Gini Impurity to decide splits.
- ✓ Prone to overfitting → control with max_depth, min_samples_split, pruning.
- ✓ Widely used in real life for decisions that humans can also reason about!

Important Hyperparameters

- criterion: 'gini' or 'entropy'
- max_depth: Maximum depth of the tree
- min_samples_split: Minimum samples to split
- min_samples_leaf: Minimum samples in a leaf



K-NEAREST NEIGHBORS (KNN)

Real-Life Uses, Math Behind KNN & Implementation Example

WHAT IS KNN?

KNN is a supervised machine learning algorithm used for classification and regression. It makes predictions based on the majority class (classification) or average (regression) of the k nearest data points (neighbors).

1. REAL LIFE USES OF KNN

1. **Recommender Systems**
   Recommend products, movies, or songs to users based on similar users' preferences.
   Example: Amazon, Netflix
2. **Medical Diagnosis**
   Predict disease based on similar patient records.
   Example: Identifying heart disease
3. **Image Recognition**
   Classify images by comparing with similar images.
   Example: Handwritten digit recognition
4. **Fraud Detection**
   Detect fraudulent transactions based on similar past transactions.
   Example: Credit card fraud detection
5. **Customer Segmentation**
   Group similar customers for targeted marketing.
   Example: Banking, E-commerce

2. HOW KNN WORKS?

1. Choose value of k (number of neighbors)
2. Compute distance between the query point and all training points
3. Select k nearest neighbors
4. For **Classification:** Majority class of k neighbors
   For Regression: Average target value of k neighbors
5. Assign the predicted class or value

3. MATHEMATICS BEHIND KNN

3.1 Distance Metrics

KNN relies on distance to find nearest neighbors.

- **Euclidean Distance** (most common)

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

where $x = (x_1, x_2, \ldots, x_n)$ and $y = (y_1, y_2, \ldots, y_n)$

- **Manhattan Distance**

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

- **Minkowski Distance**

$$d(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}}$$

($p = 1 \rightarrow$ Manhattan, $p = 2 \rightarrow$ Euclidean)

3.2 Prediction Rule

- **Classification:**
  Given a query point x_q, find k nearest neighbors $N_k(x_q)$.

$$\hat{y} = \text{mode}\{y_i \mid x_i \in N_k(x_q)\}$$

(mode = most frequent class)

- **Regression:**

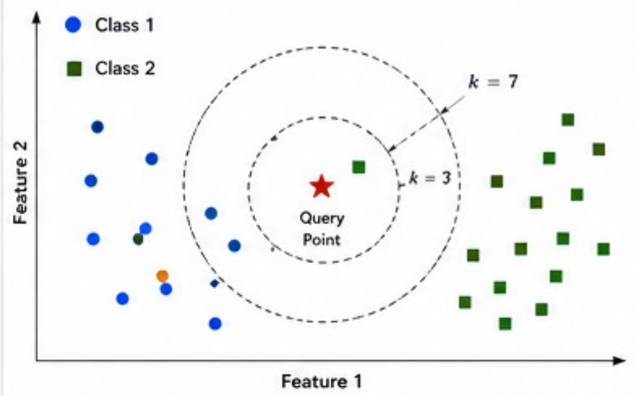
$$\hat{y} = \frac{1}{k} \sum_{x_i \in N_k(x_q)} y_i$$

(average of target values)

3.3 Time Complexity

- **Training:** $O(1)$ (just store data)
- **Prediction:** $O(nd)$ per query
  (n = number of training samples,
  d = number of features)

4. VISUAL EXAMPLE (CLASSIFICATION)



$k = 3$
Neighbors:
→ Predicted Class: Class 2
(Majority = 2 green)

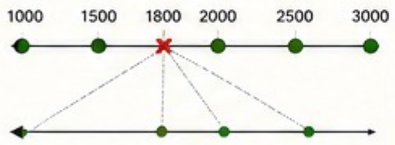
$k = 7$
Neighbors:
→ Predicted Class: Class 1
(Majority = 4 blue)

Choosing k is very important. Small $k \rightarrow$ sensitive to noise.
Large $k \rightarrow$ smoother decision boundary.

5. KNN REGRESSION EXAMPLE

House Size (sq ft)	Price (in $1000)
1000	120
1500	180
2000	250
2500	300
3000	380

Predict price for house size = 1800 sq ft
($k = 3$)



3 nearest neighbors: 1500, 2000, 1000
Predicted Price = (180 + 250 + 120) / 3 = 183.33 (in $1000)

6. KNN ALGORITHM (PSEUDOCODE)

1. **Input:** Training data D, query point x_q, number of neighbors k
2. For each point x_i in D:
   Compute distance $d(x_q, x_i)$
3. Sort all points by distance and select k nearest neighbors
4. If Classification: return majority class of k neighbors
   If Regression: return average of target values of k neighbors
5. **Output:** Predicted class or value

7. PROS AND CONS

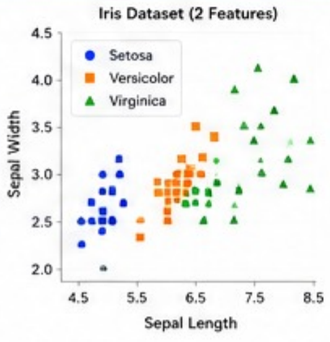
Advantages
- Simple and easy to understand
- No training phase (lazy learner)
- Works well for multi-class problems
- Effective with non-linear boundaries

Disadvantages
- Slow prediction for large datasets
- Sensitive to irrelevant features
- Requires feature scaling
- Choosing k is critical

8. KNN ON REAL DATASET (EXAMPLE)

Dataset: Iris (from sklearn)

Sepal Length	Sepal Width	Petal Length	Petal Width	Species
5.1	3.5	1.4	0.2	Setosa
4.9	3.0	1.4	0.2	Setosa
6.2	3.4	5.4	2.3	Virginica
5.9	3.0	5.1	1.8	Virginica
...	...	...	...	...



Example Prediction

Query Point
(Sepal Length=6.0, Sepal Width=2.7)

$k = 5$

→ Nearest neighbors belong to Versicolor (3) and Virginica (2)

→ Predicted Class: Versicolor

9. PYTHON IMPLEMENTATION (EXAMPLE)

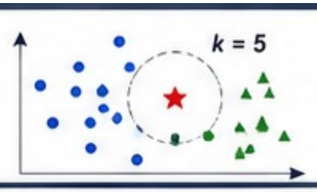
```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler

# 1. Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# 2. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 3. Feature scaling (important for KNN)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)

# 4. Train KNN (k=5)
knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train, y_train)

# 5. Predict
y_pred = knn.predict(X_test)

# 6. Evaluate
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc*100:.2f}%")
```

Output (Example)
Accuracy: 96.67%

10. HOW TO CHOOSE BEST K?

1. Try multiple k values (e.g., 1, 3, 5, 7, 9...)
2. Use cross-validation to evaluate performance
3. Plot k vs accuracy (Elbow method / Validation curve)
4. Generally, choose odd k for binary classification
5. Small $k \rightarrow$ sensitive to noise
   Large $k \rightarrow$ may miss local patterns

KEY TAKEAWAY KNN is simple yet powerful. It works well for small to medium datasets and non-linear data. The key is choosing the right k, distance metric, and scaling the features.



K-NEAREST NEIGHBORS (KNN)

Implementation on Real Dataset (Iris Flower Dataset)

DATASET USED

Iris Flower Dataset (from sklearn)

- 150 samples, 4 features, 3 classes
- Target Classes:
  0 = Setosa, 1 = Versicolor, 2 = Virginica

1. IMPORT LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score,
      classification_report, confusion_matrix
import seaborn as sns

sns.set(style='whitegrid')
%matplotlib inline
```

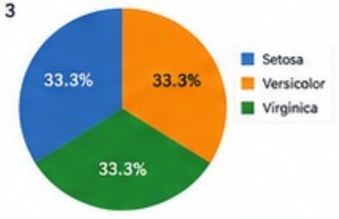
2. LOAD DATASET

```
iris = datasets.load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names

df = pd.DataFrame(X, columns=feature_names)
df['target'] = y
df['target_name'] = df['target'].map(
  {i: name for i, name in enumerate(target_names)})

df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	target_name
0	5.1	3.5	1.4	0.2	0	setosa
1	4.9	3.0	1.4	0.2	0	setosa
2	4.7	3.2	1.3	0.2	0	setosa
3	4.6	3.1	1.5	0.2	0	setosa
4	5.0	3.6	1.4	0.2	0	setosa
...	...	...	...	...	...	...

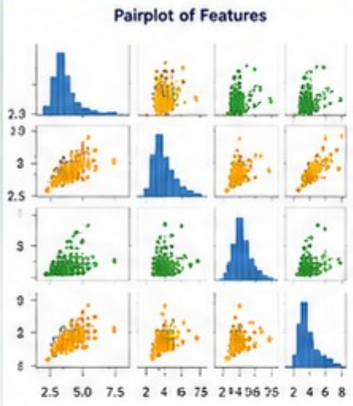
DATASET INFO

- **Features:** 4 numeric features
- **Samples:** 150
- **Classes:** 3

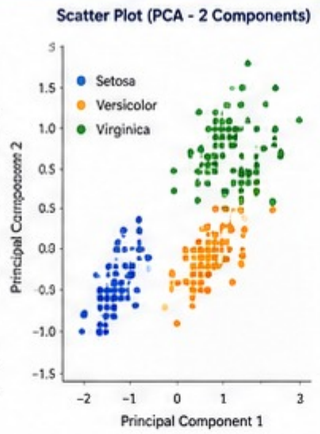


3. EXPLORATORY DATA ANALYSIS

Pairplot of Features



Scatter Plot (PCA - 2 Components)



4. TRAIN-TEST SPLIT & FEATURE SCALING

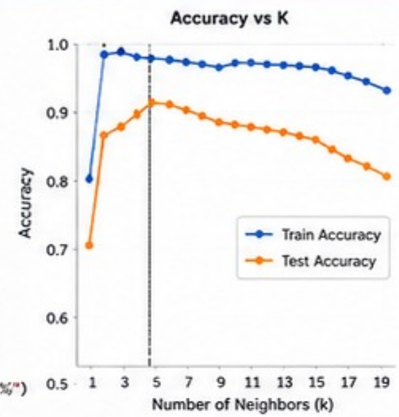
```
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Why Scaling?
KNN is distance-based algorithm. Scaling ensures that all features contribute equally to the distance calculation.

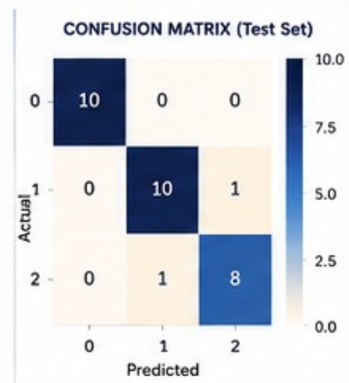
5. KNN MODEL TRAINING & EVALUATION

```
# Try different values of k
k_values = list(range(1, 21))
train_acc = []
test_acc = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    train_acc.append(accuracy_score(y_train,
       knn.predict(X_train_scaled)))
    test_acc.append(accuracy_score(y_test,
       knn.predict(X_test_scaled)))

# Best k based on test accuracy
best_k = k_values[np.argmax(test_acc)]
print(f"Best k: {best_k} (Test Accuracy: {max(test_acc)*100:.2f}%")

# Train final model with best k
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train_scaled, y_train)
y_pred = knn.predict(X_test_scaled)
```

Accuracy vs K



CONFUSION MATRIX (Test Set)



0 = Setosa
1 = Versicolor
2 = Virginica

```
Best k: 5 (Test Accuracy: 96.67%)
```

CLASSIFICATION REPORT

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.91	0.91	0.91	11
virginica	0.89	0.89	0.89	9
accuracy			0.97	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.97	0.97	0.97	30

OVERALL ACCURACY

Accuracy: 96.67%

Correct Predictions: 29 / 30

Misclassifications: 1

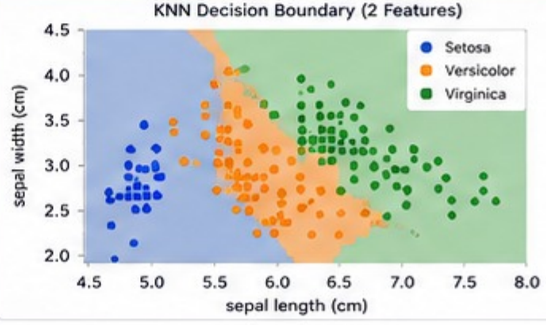
6. MAKE PREDICTION FOR NEW DATA

```
# Example: Predict a new flower
# (sepal length, sepal width, petal length, petal width)
new_flower = np.array([[6.1, 2.8, 4.7, 1.2]])
new_flower_scaled = scaler.transform(new_flower)
pred = knn.predict(new_flower_scaled)
proba = knn.predict_proba(new_flower_scaled)

print("Predicted Class :",target_names[pred[0]])
print("Probabilties    :", proba[0])
```

```
Predicted Class   : Versicolor
Probabilities     : [0.   0.8  0.2]
```

7. DECISION BOUNDARY (k = 5)

KNN Decision Boundary (2 Features)



8. KEY TAKEAWAYS

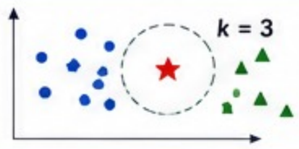
- ✅ KNN is simple, easy to implement and understand.
- ✅ Performance depends on the choice of k and distance metric.
- ✅ Feature scaling is crucial for good performance.
- ✅ Works well for small to medium datasets.
- ✅ Sensitive to irrelevant features and noisy data.
- ✅ Non-linear decision boundaries.

SUMMARY

In this example, we used the Iris dataset to implement KNN from scratch using Scikit-learn.
With k = 5, we achieved 96.67% accuracy on the test set.
You can try other values of k and distance metrics to see the effect on performance.

You can experiment with:

- Different distance metrics (euclidean, manhattan, minkowski)
- Different values of k
- Other datasets (Wine, Breast Cancer, Digits, etc.)



K-NEAREST NEIGHBORS (KNN)

Implementation on Sample Dataset with Code

SAMPLE DATASET

Iris Dataset (from sklearn)

- 150 samples, 4 features
- 3 classes: Setosa, Versicolor, Virginica

1. IMPORT LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score,
     classification_report, confusion_matrix
import seaborn as sns

sns.set(style='whitegrid')
%matplotlib inline
```

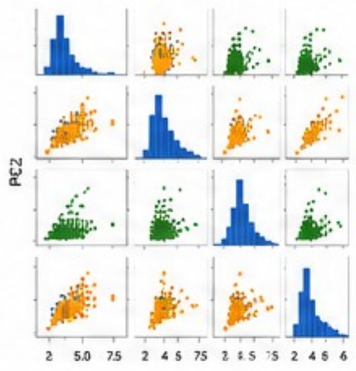
2. LOAD DATASET

```
# Load Iris dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names

# Create DataFrame for better view
df = pd.DataFrame(X, columns=feature_names)
df['target'] = y
df['target_name'] = df['target'].map(
   {i: name for i, name in enumerate(target_names)})
df.head()
```

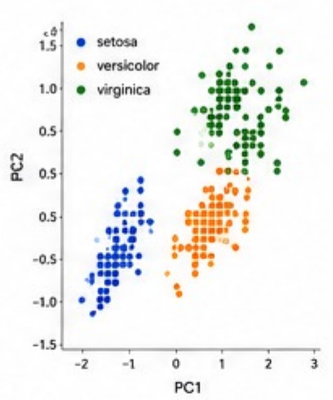
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	target_name
0	5.1	3.5	1.4	0.2	0	setosa
1	4.9	3.0	1.4	0.2	0	setosa
2	4.7	3.2	1.3	0.2	0	setosa
3	4.6	3.1	1.5	0.2	0	setosa
4	5.0	3.6	1.4	0.2	0	setosa

3. EDA – VISUALIZE DATASET

Pairplot of Features



Scatter plot (PCA – 2 Components)



4. TRAIN-TEST SPLIT & FEATURE SCALING

```
# Split the data
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y)

# Feature Scaling (Important for KNN)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

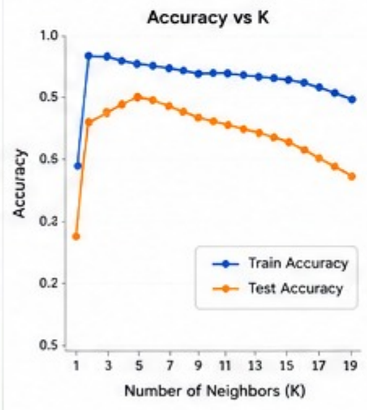
Why Scaling?

KNN uses distance to find neighbors.
If features are on different scales,
larger values dominate the distance.

5. TRAIN KNN MODEL & FIND BEST K

```
train_acc = []
test_acc = []
k_values = list(range(1, 21))

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    train_acc.append(accuracy_score(y_train,
                          knn.predict(X_train_scaled)))
    test_acc.append(accuracy_score(y_test,
                   knn.predict(X_test_scaled)))


# Find best k based on test accuracy
best_k = k_values[np.argmax(test_acc)]
print(f"Best k: {best_k} (Test Accuracy: {max(test_acc):.2f}%)")
```

6. ACCURACY VS K



7. EVALUATE MODEL (USING BEST K)

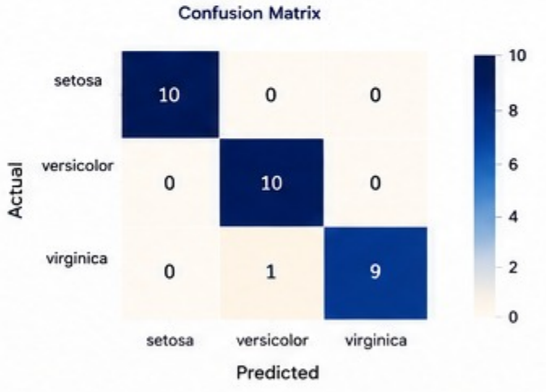
```
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train_scaled, y_train)
y_pred = knn.predict(X_test_scaled)

# Accuracy
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc:.2f}")
print()

# Classification Report
print(classification_report(y_test, y_pred,
     target_names=target_names))
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
```

8. RESULTS

Classification Report

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	1.00	0.95	10
virginica	1.00	0.90	0.95	10
accuracy			0.97	30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

Confusion Matrix



9. PREDICT ON NEW SAMPLE DATA

```
new_sample = np.array([[5.1, 3.5, 1.4, 0.2]])  # a new flower
new_sample_scaled = scaler.transform(new_sample)
pred = knn.predict(new_sample_scaled)
proba = knn.predict_proba(new_sample_scaled)

print("Predicted Class:", target_names[pred[0]])
print("Probabilities:", np.round(proba[0], 3))
```

Example Output

```
Predicted Class: setosa
Probabilities: [1. 0.  0.]
```

10. COMPLETE CODE (ALL TOGETHER)

Full Python Code
(All Steps)

See separate .py file
for full code.

- ✔ Load dataset
- ✔ EDA
- ✔ Train-test split
- ✔ Feature scaling
- ✔ Find best K
- ✔ Train final model
- ✔ Evaluate model
- ✔ Predict new data

11. KEY TAKEAWAYS

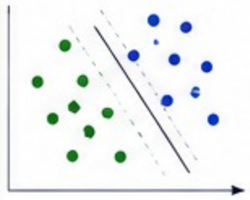
- KNN is simple, non-parametric and effective for small to medium datasets.
- Scaling is crucial because KNN is distance based.
- Choice of K affects performance (use odd k for binary classification).
- Works well for classification and regression.
- Sensitive to irrelevant features and noisy data.

SUMMARY

We implemented KNN from scratch using the Iris dataset.
Achieved high accuracy (~97%) on the test set using the optimal value of K.
You can try different K values and distance metrics to see their impact!

Try Different Distance Metrics

You can try: 'euclidean', 'manhattan', 'minkowski'
by passing the 'metric' parameter in KNeighborsClassifier.



SUPPORT VECTOR MACHINE (SVM)

POWERFUL. VERSATILE. REAL-WORLD IMPACT.

SVM is a supervised machine learning algorithm used for classification and regression.
It works by finding the optimal hyperplane that best separates data points of different classes.

1. WHAT IS SVM?

- SVM finds the decision boundary (hyperplane) that maximizes the margin between classes.
- Works well in high-dimensional spaces.
- Effective for both linear and non-linear data (using kernels).

2. HOW SVM WORKS (CLASSIFICATION)

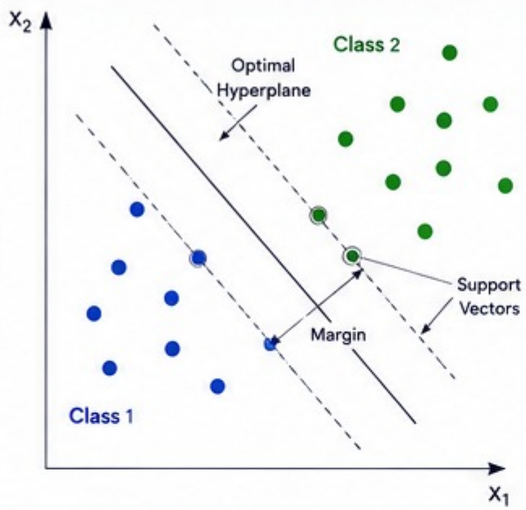
1 Find the best separating hyperplane
That maximizes the margin.

2 Support Vectors
The nearest points to the hyperplane are called support vectors. They define the boundary.

3 Maximize Margin
SVM chooses the hyperplane with the largest margin for better generalization.

4 Make Prediction
New data points are classified based on which side of the hyperplane they fall.

3. SVM: MAXIMUM MARGIN HYPERPLANE (2D EXAMPLE)



Find the hyperplane that maximizes the margin and minimizes error.

SVM Decision Function

$$f(x) = w \cdot x + b$$

Predict:
Class 1 if $f(x) < 0$
Class 2 if $f(x) > 0$

Mathematical Objective (Hard Margin)

$$\text{Minimize } \frac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1, \qquad \forall i$$

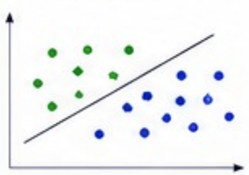
Where, w = weight vector, b = bias, $y_i \in \{-1, +1\}$, x_i = training points

For non-linearly separable data, SVM uses Kernels to map data to higher dimensions.

4. KERNEL TRICK – HANDLE NON-LINEAR DATA

Linear Kernel

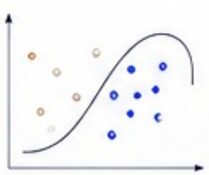
$$K(x, z) = x^T z$$



Used when data is linearly separable.

Polynomial Kernel

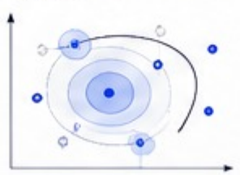
$$K(x, z) = (\gamma x^T z + r)^d$$



Captures polynomial relationships.

RBF (Gaussian) Kernel

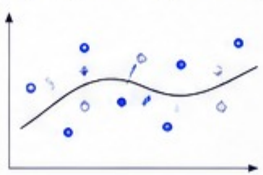
$$K(x, z) = e^{-\gamma\|x-z\|^2}$$



Most popular. Handles complex non-linear data.

Sigmoid Kernel

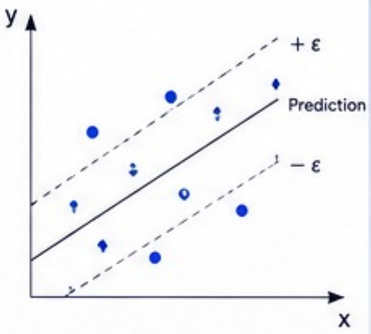
$$K(x, z) = \tanh(\gamma x^T z + r)$$



Similar to neural networks.

5. SVM FOR REGRESSION (SVR)

SVM can also be used for regression using an ϵ-insensitive loss function.



Points within the ϵ-tube are ignored.

Only points outside the tube contribute to the loss.

Why SVM?

- Works well in high-dimensional spaces
- Effective with small to medium datasets
- Robust to overfitting (with proper tuning)
- Uses only support vectors (efficient)
- Versatile: classification, regression, outlier detection, etc.

SVM IN REAL LIFE – WHERE IT MAKES AN IMPACT

1. MEDICAL DIAGNOSIS

SVM helps classify medical data to detect diseases.

Example: Cancer detection, tumor classification, heart disease prediction

2. SPAM DETECTION

Classifies emails as spam or not spam.

Example: Email filtering in Gmail, Outlook, Yahoo

3. FACE RECOGNITION

SVM is used to classify and verify human faces.

Example: Unlocking smartphones, security systems

4. CUSTOMER SEGMENTATION

Groups customers based on their behavior and attributes.

Example: Targeted marketing, recommendation systems

5. STOCK MARKET PREDICTION

Predicts stock price movement (up/down).

Example: Algorithmic trading, trend prediction

6. FRAUD DETECTION

Detects fraudulent transactions in banking and credit card systems.

Example: Anomaly detection in transaction data

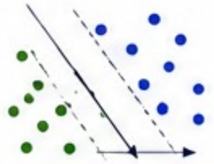
7. TEXT & SENTIMENT ANALYSIS

Classifies text into categories or sentiment (positive/negative).

Example: Movie reviews, social media sentiment analysis

KEY TAKEAWAY: SVM is a powerful and flexible algorithm used across industries—from healthcare to finance.
Its ability to handle complex data with high accuracy makes it a go-to choice for real-world machine learning problems.

SVM = Simplicity in concept,
Power in performance, Impact in reality.



SUPPORT VECTOR MACHINES (SVM)

Types, How They Work, Kernel Choices & How to Select the Best SVM

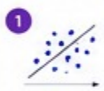
SVM Core Idea

Find the hyperplane that best separates classes by maximizing the margin between them.

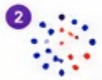
1. WHAT IS SVM?

- SVM is a supervised ML algorithm used for classification and regression.
- It finds the optimal hyperplane that maximizes the margin between classes.
- Works well in high-dimensional spaces and is effective with small to medium datasets.
- Uses only support vectors (important training points) to build the model.

2. TYPES OF SVM



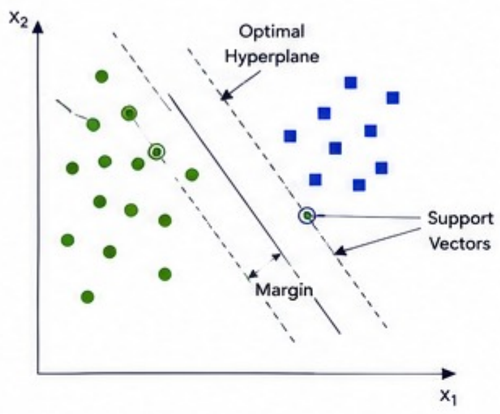
1. **Linear SVM**
   Used when data is linearly separable.



2. **Non-Linear SVM (Kernel SVM)**
   Used when data is NOT linearly separable.

3. **SVM for Regression (SVR)**
   Used for continuous value prediction.

3. LINEAR SVM (LINEARLY SEPARABLE)

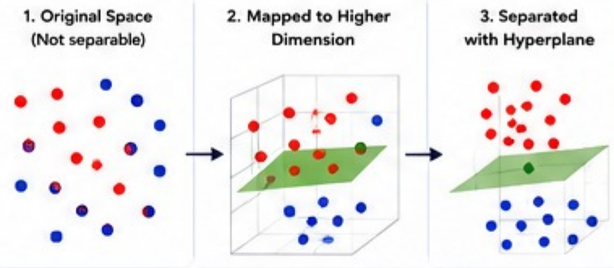


Equation of Hyperplane: $w \cdot x + b = 0$

Decision Function: $f(x) = \text{sign}(w \cdot x + b)$

4. NON-LINEAR SVM (KERNEL SVM)

Used when classes cannot be separated using a straight line.

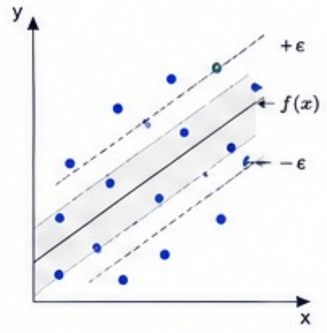


Kernel Trick: Implicitly maps data to higher dimension without actually computing the transformation.

Common Kernels

Kernel	Formula $K(x, z)$	Use When
Linear	$x^T z$	Data is linearly separable or high-dimensional
Polynomial (degree d)	$(\gamma x^T z + r)^d$	Captures polynomial relationships
RBF (Gaussian)	$e^{-\gamma \|x-z\|^2}$	Most popular. Good for complex, non-linear data
Sigmoid	$\tanh(\gamma x^T z + r)$	Similar to neural networks

5. SVM FOR REGRESSION (SVR)



- ϵ-insensitive tube: errors within ϵ are ignored.
- Only points outside the tube are used to compute the loss.
- Used for regression problems.

7. HOW TO CHOOSE THE BEST SVM?

1. **Understand the Problem**
   Classification or Regression?
2. **Check Data Linearity**
   - Plot the data (if possible).
   - If classes are linearly separable → Linear SVM
   - If not → Try Non-linear SVM (Kernel SVM)
3. **Choose a Kernel**
   - Start with RBF (default & powerful)
   - Try Linear, Polynomial if appropriate
4. **Feature Scaling**
   - Always scale features (StandardScaler)
   - SVM is sensitive to feature scales
5. **Hyperparameter Tuning**
   Key parameters: C, kernel, gamma (γ), degree (d), coef0
   - Use GridSearchCV / RandomizedSearchCV
6. **Cross-Validation**
   - Use k-fold CV (e.g., k=5 or 10)
   - Evaluate using appropriate metric
7. **Evaluate & Select**
   Choose the model with the best CV performance on validation data, then test on unseen data.

6. COMPARISON: LINEAR vs NON-LINEAR SVM

Aspect	Linear SVM	Non-Linear SVM (Kernel SVM)
Data Requirement	Works well if data is linearly separable	Works when data is not linearly separable
Complexity	Less complex	More complex
Training Time	Faster	Slower (depends on kernel & data size)
Interpretability	Easy (hyperplane in original space)	Harder (decision boundary in higher dimension)
Performance	Good on simple data	Better on complex data
Overfitting Risk	Lower	Higher (requires tuning)
Example Kernels	Linear	Polynomial, RBF, Sigmoid

8. HOW SVM MAKES DECISIONS (CLASSIFICATION)

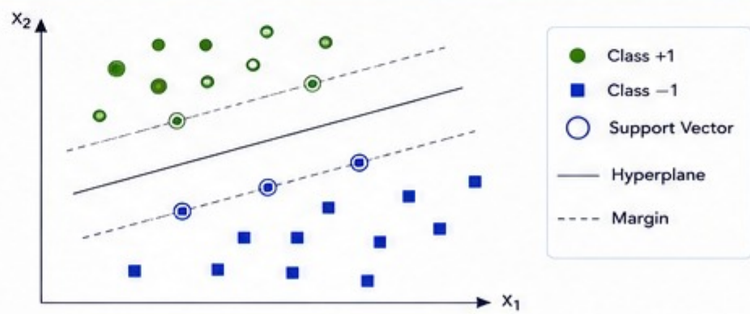
Decision Function:

$$f(x) = \text{sign}\left(\sum_{i=1}^{n} \alpha_i y_i K(x_i, x) + b\right)$$

Where,

- x_i : Support vectors
- y_i : Class label (+1 or −1)
- α_i : Lagrange multipliers
- $K(x_i, x)$: Kernel function
- b : Bias term

Only support vectors (with $\alpha_i > 0$) affect the decision boundary.



Key Hyperparameters

- **C (Regularization)**: Controls trade-off between maximizing margin and minimizing error.
  Low C → larger margin, may underfit
  High C → smaller margin, may overfit
- **gamma (γ)**: Influence of single training example.
  Low γ → smoother decision boundary
  High γ → more complex boundary
- **kernel**: Type of kernel function to use.
- **degree** (for Polynomial): Degree of the polynomial.
- **coef0** (for Poly/Sigmoid): Independent term.

9. REAL-LIFE APPLICATIONS

10. WHEN TO USE WHICH SVM?

Scenario	Recommended SVM
Small/Medium dataset, linear separable	Linear SVM
High-dimensional data (e.g., text)	Linear SVM
Complex pattern, non-linear boundary	RBF Kernel SVM
Polynomial relationships	Polynomial Kernel SVM
Regression problems	SVR (with RBF/Linear Kernel)

KEY TAKEAWAY: Start simple with Linear SVM. If it doesn't perform well, move to RBF Kernel SVM and tune hyperparameters using cross-validation.

Support Vector Machine (SVM) – Hands-on with Real Dataset

From Data to Decision Boundary

1. Dataset: Iris Flower

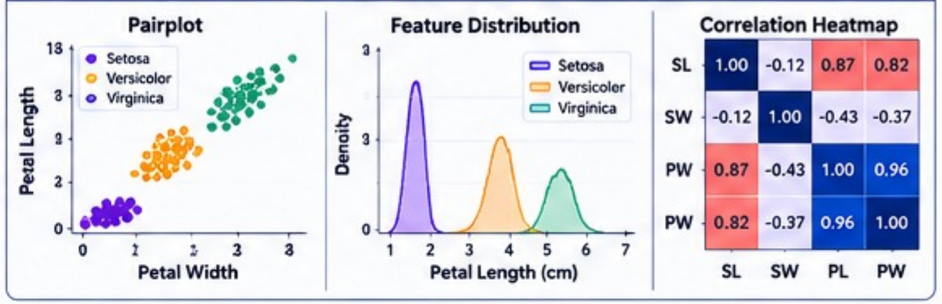
- One of the most popular ML datasets
- 150 samples, 4 numerical features
- 3 classes: Setosa, Versicolor, Virginica
- Goal: Classify flower species based on features

2. Dataset Preview

SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
6.2	3.4	5.4	2.3	virginica
5.9	3.0	5.1	1.8	virginica
6.7	3.1	4.7	1.5	versicolor

Shape: (150, 5) → 150 rows, 5 columns

3. Exploratory Data Analysis



4. SVM Model Code (Python)

```python
# 1. Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix,
  ConfusionMatrixDisplay

# 2. Load Dataset
df = pd.read_csv("https://raw.githuburraetont.com/mwaskom/
seaborn-data/master/iris.csv")

# 3. Features and Target
X = df.drop('species', axis=1)
y = df['species']

# 4. Split Data
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y)

# 5. Feature Scaling (important for SVM)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Train SVM (RBF Kernel)
model = SVC(kernel='rbf', C=1, gamma='scale', random_state=42)
model.fit(X_train_scaled, y_train)

# 7. Predictions
y_pred = model.predict(X_test_scaled)

# 8. Evaluation
print("Accuracy="; model.score(X_test_scaled, y_test))
print(classification_report(y_test, y_pred))
```

5. Decision Boundaries (2D View)

6. Model Evaluation

Test Accuracy	Precision	Recall	F1-Score
100%	1.00	1.00	1.00

Classification Report

	Precision	Recall	F1-Score	Support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	10
virginica	1.00	1.00	1.00	10
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Confusion Matrix

7. Hyperparameter Tuning (Find Best SVM)

```python
param_grid = {
  'C': [0.1, 1, 10, 100],
  'gamma': ['scale', 0.01, 0.1, 1],
  'kernel': ['linear', 'rbf', 'poly']
}

grid = GridSearchCV(SVC(), param_grid,
     cv=5, scoring='accuracy')
grid.fit(X_train_scaled, y_train)
```

Best Parameters

Kernel : rbf
C : 10
Gamma : 0.1

Best CV Accuracy
100%

8. When to Choose Which SVM?

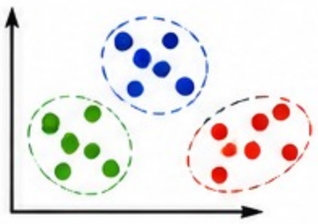
Kernel	Best For	Pros	Cons
Linear	Linearly separable data, high-dim	Fast, simple	Poor for non-linear data
Polynomial	Medium complexity non-linear data	Captures curved boundaries	May overfit
RBF	Most real-world non-linear data	Best performance, flexible	Slower, needs tuning
Sigmoid	Neural network like problems	Works in some cases	Less used

9. Real-Life Uses of SVM

- **Medical Diagnosis** (Cancer detection)
- **Spam Filtering** (Email classification)
- **Face Recognition** (Image classification)
- **Text Classification** (Sentiment analysis)
- **Bioinformatics** (Gene classification)
- **Stock Prediction** (Trend classification)

Key Takeaways

- SVM finds the optimal hyperplane to separate classes with maximum margin.
- **RBF kernel** works best in most real-world, non-linear datasets.
- **Feature scaling** is a must for SVM.
- Use **GridSearchCV** to find the best parameters.
- SVM is powerful, accurate and effective, especially for small to medium datasets.



K-MEANS CLUSTERING

Unsupervised Learning Algorithm

K-Means clustering partitions *n* observations into *k* clusters in which each observation belongs to the cluster with the nearest mean (centroid).

Key Idea

Group similar data points together based on their distance from the cluster centroid.

1. WHAT IS K-MEANS?

K-Means is an unsupervised clustering algorithm that aims to divide *n* data points into *k* clusters such that the within-cluster sum of squares (WCSS) is minimized.

2. HOW IT WORKS

It iteratively performs the following steps:

1. **Initialize Centroids**
   Choose *k* initial centroids randomly.
2. **Assign Clusters**
   Assign each data point to the nearest centroid (based on distance).
3. **Update Centroids**
   Recaclulate centroids as the mean of all points in each cluster.
4. **Repeat**
   Repeat steps 2 and 3 until centroids no longer change or max iterations reached.

5. OBJECTIVE FUNCTION

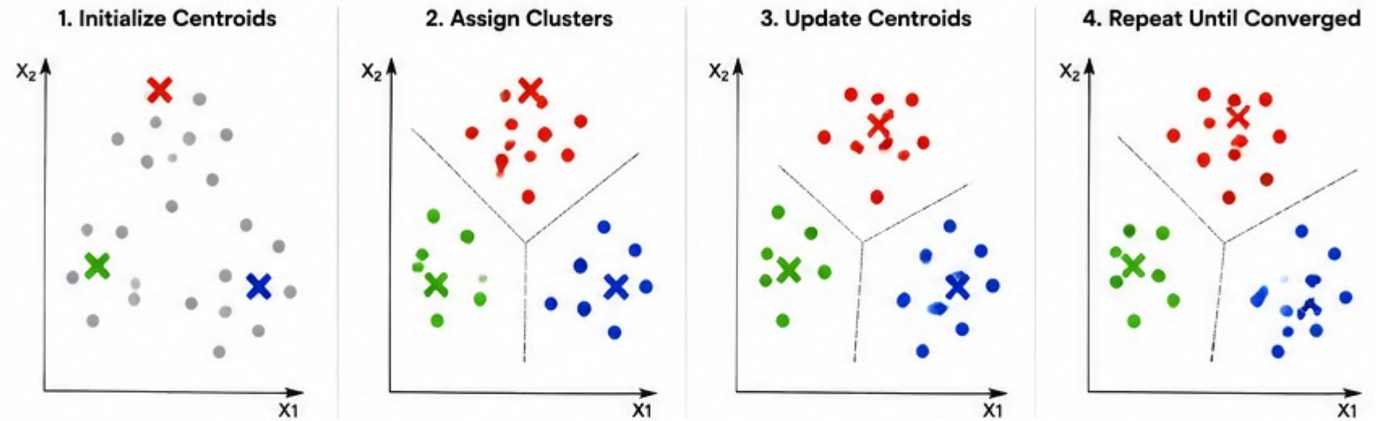
K-Means aims to minimize the Within-Cluster Sum of Squares (WCSS):

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} \| x - \mu_i \|^2$$

Where,

- k = number of clusters
- C_i = set of points in cluster i
- μ_i = centroid (mean) of cluster i
- $\| x - \mu_i \|^2$ = squared Euclidean distance

3. VISUALIZATION: HOW K-MEANS WORKS (*k* = 3)



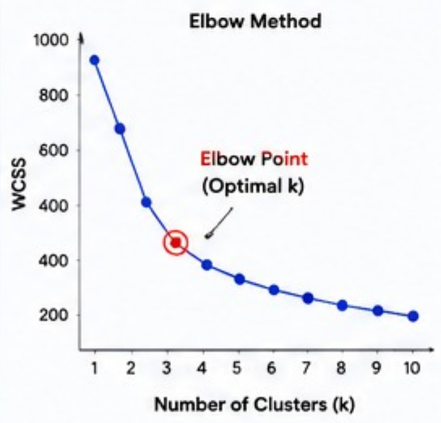
The algorithm converges when centroids do not change significantly between iterations.

4. EXAMPLE DATASET (2D)

Point	X1	X2
P1	2.0	2.0
P2	2.5	1.5
P3	2.2	2.5
P4	8.0	8.0
P5	8.5	8.5
P6	7.5	8.2
P7	5.0	2.0
P8	5.5	1.8
P9	4.5	2.2

For *k* = 3, K-Means will group these points into three clusters.

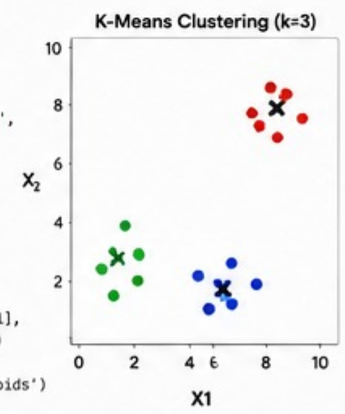
6. CHOOSING THE RIGHT *k* (ELBOW METHOD)



Choose *k* at the "elbow point" where the decrease in WCSS starts to slow down.

7. IMPLEMENTATION IN PYTHON

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Sample data
X = np.array([[2,2],[2.5,1.5],[2.2,2.5],
              [8,8],[8.5,8.5],[7.5,8.2],
              [5,2],[5.5,1.8],[4.5,2.2]])

# Apply K-Means
k = 3
kmeans = KMeans(n_clusters=k, init='k-means++',
                random_state=42, n_init=10)
kmeans.fit(X)
labels = kmeans.labels_
centroids = kmeans.cluster_centers_

# Plotting
colors = ['r', 'g', 'b']
for i in range(k):
    plt.scatter(X[labels==i, 0], X[labels==i, 1],
        c=colors[i], label=f'Cluster {i+1}')
plt.scatter(centroids[:,0], centroids[:,1],
        c='k', marker='X', s=150, label='Centroids')
plt.xlabel('X1'); plt.ylabel('X2')
plt.legend(); plt.title('K-Means Clustering (k=3)')
plt.show()
```



8. PROS & CONS

Pros

- Simple and easy to implement
- Scales well to large datasets
- Works well with spherical clusters
- Fast convergence

Cons

- Must specify *k* in advance
- Sensitive to initial centroids
- Not suitable for non-spherical clusters
- Sensitive to outliers

9. REAL-WORLD APPLICATIONS

- **Customer Segmentation**
  Group customers based on purchasing behavior.
- **Market Research**
  Identify similar products or target audiences.
- **Image Compression**
  Reduce colors by clustering pixel values.
- **Anomaly Detection (Preprocessing)**
  Identify normal patterns and detect outliers.

TIP: K-Means works best when clusters are compact and well-separated. Always scale your data before applying K-Means.

HOW TO CHOOSE THE BEST K VALUE?
ELBOW METHOD FOR K-MEANS CLUSTERING

The Elbow Method helps us choose the optimal number of clusters (K) by plotting the Within-Cluster Sum of Squares (WCSS) for different values of K and selecting the point where the rate of decrease sharply changes (the "elbow").

1. CONCEPT

- For small K, WCSS is high (clusters are large, many errors).
- As K increases, WCSS decreases (clusters become tighter).
- After a certain point, the improvement in WCSS becomes small.
- The "elbow" point is the best trade-off between compact clusters and simplicity.

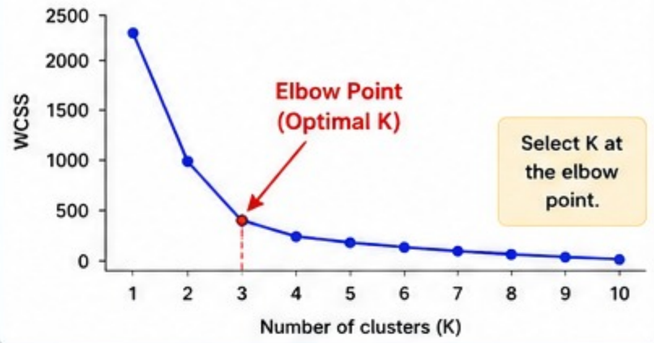
Goal: Minimize WCSS
(Within-Cluster Sum of Squares)

$$\text{WCSS} = \sum_{i=1}^{K} \sum_{x \in C_i} \|x - \mu_i\|^2$$

where μ_i is the centroid of cluster C_i

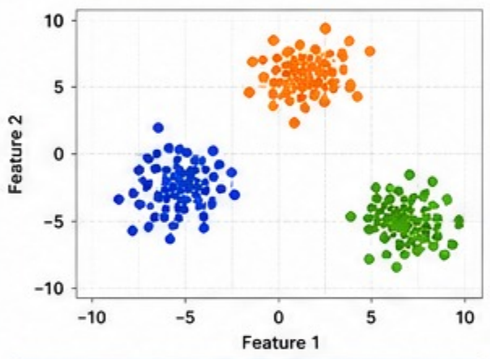
2. WHAT IS THE ELBOW?

The elbow is the point where adding more clusters does not significantly reduce WCSS.



3. SAMPLE DATASET (2 FEATURES)

We will use a sample dataset with 3 natural clusters.



Sample Data (First 10 Rows)	
X (Feature 1)	Y (Feature 2)
-8.1	-3.2
-7.3	-2.9
-8.5	-3.5
4.9	-5.1
5.2	-4.7
5.5	-5.5
1.2	5.8
1.7	6.3
2.1	6.1
0.9	5.4
Total Samples: 150	

4. PYTHON CODE TO FIND BEST K (ELBOW METHOD)

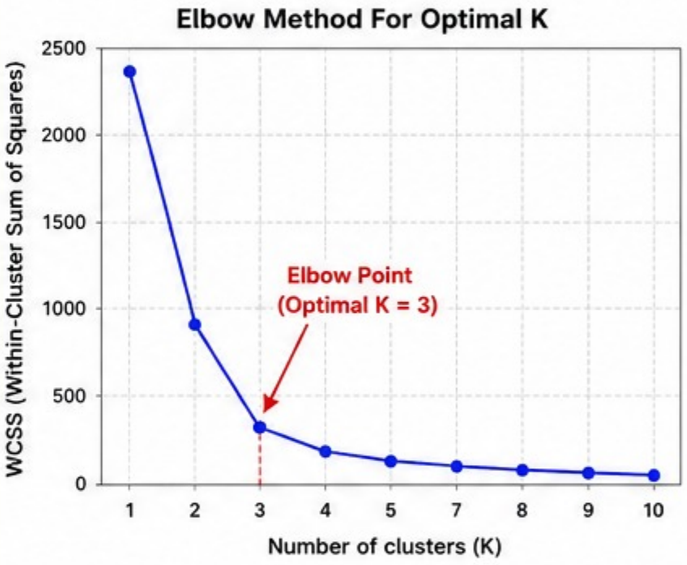
```python
# Elbow Method for K-Means
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# 1. Generate sample dataset (3 clusters)
X, y_true = make_blobs(n_samples=150, centers=3,
                       cluster_std=1.0, random_state=42)

# 2. Calculate WCSS for different K values
wcss = []
K_range = range(1, 11)  # Try K from 1 to 10

for k in K_range:
    kmeans = KMeans(n_clusters=k, init='k-means++',
                    random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)  # inertia_ = WCSS

# 3. Plot Elbow Graph
plt.figure(figsize=(8,5))
plt.plot(K_range, wcss, marker='o', linewidth=2, color='b')
plt.title('Elbow Method For Optimal K', fontsize=14, weight='bold')
plt.xlabel('Number of clusters (K)')
plt.ylabel('WCSS (Within-Cluster Sum of Squares)')
plt.grid(True, linstyyle='--', alpha=0.6)
plt.show()
```

5. WCSS RESULTS

K (Clusters)	WCSS
1	2360.45
2	935.12
3	320.41
4	245.75
5	198.30
6	165.23
7	144.02
8	129.63
9	116.48
10	105.91

Optimal K = 3
(Elbow Point)

6. ELBOW GRAPH (OUTPUT)

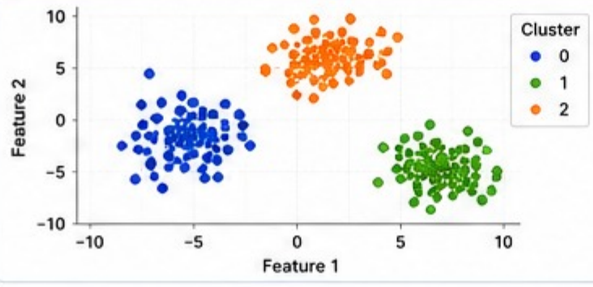


The elbow is at K = 3. After this point, the decrease in WCSS is very small, so K = 3 is the best choice.

7. HOW TO INTERPRET?

- Choose K at the elbow point.
- K < elbow: Underfitting (clusters are too large).
- K > elbow: Overfitting (clusters are too many, gain is minimal).
- Elbow gives the best balance between simplicity and accuracy.

8. FINAL CLUSTERING WITH K = 3



9. TIPS FOR CHOOSING K

- Always standardize your features before clustering.
- Use Elbow Method + Domain Knowledge together.
- Check silhouette score for better validation.
- Elbow may not always be sharp – use judgment.

10. KEY TAKEAWAY

Elbow Method is a simple and effective way to choose the best K for K-Means. Look for the point where the curve bends (the elbow) and pick that K!

In this example, Best K = 3

DBSCAN

Density-Based Spatial Clustering of Applications with Noise

DBSCAN groups together data points that are closely packed together (points with many nearby neighbors) and marks points that lie alone in low density regions as outliers (noise). No need to specify the number of clusters!

WHAT IS DBSCAN?

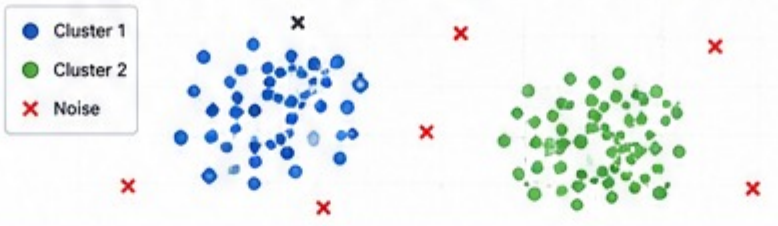
DBSCAN is an unsupervised clustering algorithm that finds clusters of arbitrary shape based on the density of data points.

KEY FEATURES

- Discovers clusters of any shape
- Automatically identifies outliers (noise)
- No need to predefine number of clusters
- Works well with real-world noisy data

CORE IDEA

Clusters are dense regions separated by areas of lower density.



REAL-LIFE USE CASES

Customer Segmentation (Retail/Banking)
Group customers with similar behavior without knowing how many segments exist.

Anomaly Detection (Fraud Detection)
Detect unusual transactions/outliers that do not belong to any dense region.

Geospatial Data Analysis
Find dense regions of locations (e.g., crime hotspots, earthquake epicenters).

Network Intrusion Detection
Identify abnormal network activities as noise points.

Image Segmentation
Group similar pixels/regions in images.

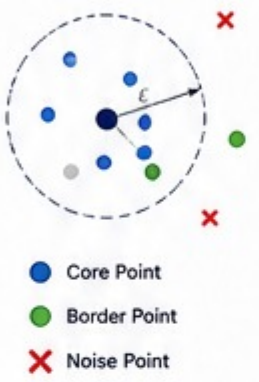
Recommendation Systems
Cluster similar users/items based on behavior without predefined groups.

KEY CONCEPTS

Core Point
A point that has at least MinPts points (including itself) within ε (epsilon) neighborhood.

Border Point
A point that has fewer than MinPts points in its ε-neighborhood but is within ε of a core point.

Noise Point (Outlier)
A point that is neither a core point nor a border point.



MATHEMATICAL CONCEPTS

Let $N_\varepsilon(p)$ be the ε-neighborhood of point p.

- Epsilon (ε): Radius of the neighborhood.
- MinPts: Minimum number of points required to form a dense region.

1. **ε-neighborhood of a point p:**

$$N_\varepsilon(p) = \{q \in D \mid dist(p, q) \leq \varepsilon\}$$

2. **Core Point:**

$$|N_\varepsilon(p)| \geq MinPts$$

3. **Border Point:**

$|N_\varepsilon(p)| < MinPts$ and $p \in N_\varepsilon(o)$ for some core point o

4. **Noise Point:**

Otherwise

Distance Metric (usually Euclidean)

$$dist(p, q) = \sqrt{\sum_{i=1}^{d} (p_i - q_i)^2}$$

PROS & CONS

Advantages

- Finds clusters of arbitrary shape
- Handles noisy data and outliers well
- No need to specify number of clusters
- Effective on large spatial databases

Limitations

- Requires good choice of ε and MinPts
- Struggles with varying density clusters
- Not suitable for high-dimensional data

PARAMETERS

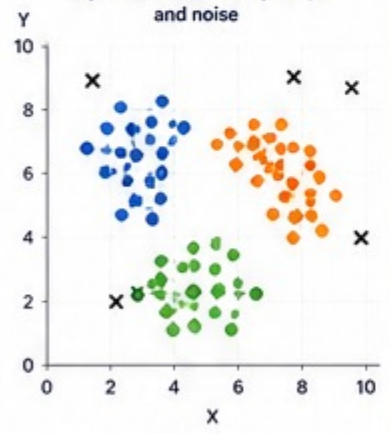
- ε (Epsilon) : Neighborhood radius
- MinPts : Minimum points to form a dense region (including itself)

HOW DBSCAN WORKS?

1. Pick a point p
2. Retrieve neighbors within ε
3. If neighbors ≥ MinPts, p is a core point
4. Expand cluster by visiting neighbors
5. Continue until no more points can be added
6. Points not reachable become noise

Unvisited points are picked and steps 1–6 are repeated.

DATASET (Sample)



Sample of Data Points

ID	X	Y
1	1.2	2.1
2	1.5	1.8
3	1.7	2.4
...	...	...
n	8.9	6.3

Dataset Info

Total Points: 150

Features: 2 (X, Y)

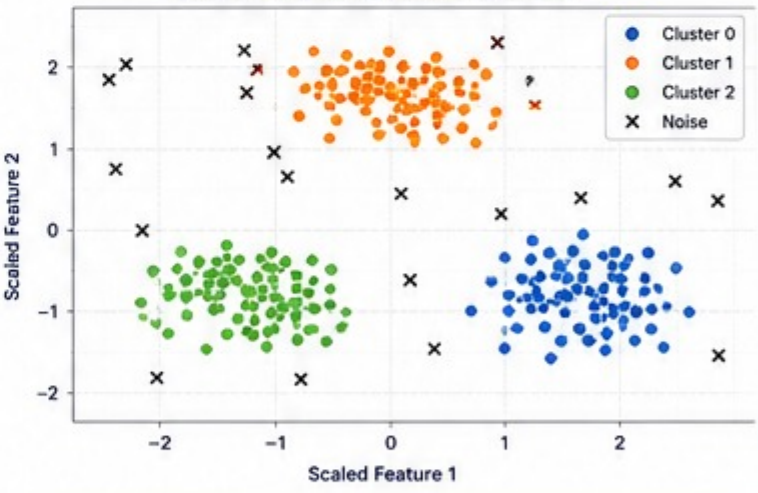
Clusters: Unknown

Noise: Present

IMPLEMENTATION (Python Code)

```python
# DBSCAN Clustering Example
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

# 1. Load dataset (sample or your own data)
data = pd.read_csv('data.csv')  # columns: X, Y
X = data[['X', 'Y']].values

# 2. (Optional) Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Apply DBSCAN
# eps = 0.6 (neighborhood radius), min_samples = 5
# metric = 'euclidean'
db = DBSCAN(eps=0.6, min_samples=5, metric='euclidean')
labels = db.fit_predict(X_scaled)

# 4. Add cluster labels to data
data['Cluster'] = labels  # -1 label means noise

# 5. Visualize
plt.figure(figsize=(8,6))
unique_labels = set(labels)
colors = plt.cm.tab10(np.linspace(0, 1, len(unique_labels)))

for k, col in zip(unique_labels, colors):
    class_member_mask = (labels == k)
    xy = X_scaled[class_member_mask]
    if k == -1:
        plt.scatter(xy[:,0], xy[:,1],  c='k', marker='x', s=60, label='Noise')
    else:
        plt.scatter(xy[:,0], xy[:,1],  c=[col],  s=60, label=f'Cluster {k}')

plt.title('DBSCAN Clustering Results')
plt.xlabel('Scaled Feature 1')
plt.ylabel('Scaled Feature 2')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)

# 6. Number of clusters (excluding noise)
num_clusters = len(set(labels)) - (1 if -1 in labels else 0)
num_noise = list(labels).count(-1)
print(f'Clusters found: {num_clusters}')
print(f'Noise points: {num_noise}')
```

RESULTS & OUTPUT



Cluster Summary

Cluster Label	Number of Points	Percentage
Cluster 0	52	34.67%
Cluster 1	47	31.33%
Cluster 2	33	22.00%
Noise (-1)	18	12.00%
Total	**150**	**100%**

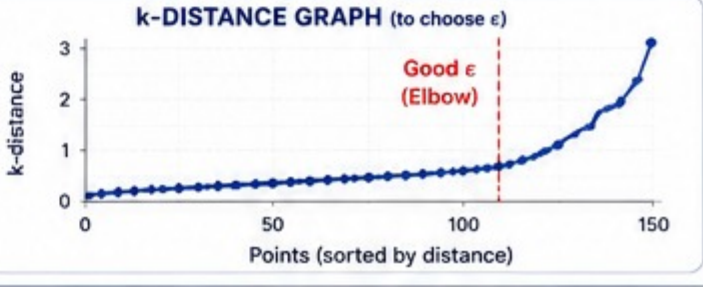
KEY TAKEAWAY

- DBSCAN groups dense regions together.
- It finds clusters of any shape and handles noise.
- Great for real-world data with outliers.

TIPS FOR BEST RESULTS

- Use k-distance graph to choose ε.
- Try MinPts between 4 to 2× number of dimensions.
- Standardize features if scales are different.

k-DISTANCE GRAPH (to choose ε)



Support Vector Machine (SVM) – Hands-on with Real Dataset

From Data to Decision Boundary

1. Dataset: Iris Flower

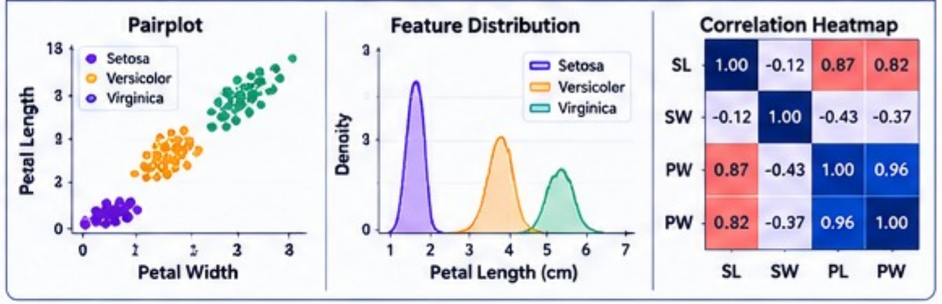
- One of the most popular ML datasets
- 150 samples, 4 numerical features
- 3 classes: Setosa, Versicolor, Virginica
- Goal: Classify flower species based on features

2. Dataset Preview

SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
6.2	3.4	5.4	2.3	virginica
5.9	3.0	5.1	1.8	virginica
6.7	3.1	4.7	1.5	versicolor

Shape: (150, 5) → 150 rows, 5 columns

3. Exploratory Data Analysis



4. SVM Model Code (Python)

```python
# 1. Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix,
  ConfusionMatrixDisplay

# 2. Load Dataset
df = pd.read_csv("https://raw.githuburraetont.com/mwaskom/
seaborn-data/master/iris.csv")

# 3. Features and Target
X = df.drop('species', axis=1)
y = df['species']

# 4. Split Data
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y)

# 5. Feature Scaling (important for SVM)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Train SVM (RBF Kernel)
model = SVC(kernel='rbf', C=1, gamma='scale', random_state=42)
model.fit(X_train_scaled, y_train)

# 7. Predictions
y_pred = model.predict(X_test_scaled)

# 8. Evaluation
print("Accuracy="; model.score(X_test_scaled, y_test))
print(classification_report(y_test, y_pred))
```

5. Decision Boundaries (2D View)

6. Model Evaluation

Test Accuracy	Precision	Recall	F1-Score
100%	1.00	1.00	1.00

Classification Report

	Precision	Recall	F1-Score	Support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	10
virginica	1.00	1.00	1.00	10
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Confusion Matrix

7. Hyperparameter Tuning (Find Best SVM)

```python
param_grid = {
  'C': [0.1, 1, 10, 100],
  'gamma': ['scale', 0.01, 0.1, 1],
  'kernel': ['linear', 'rbf', 'poly']
}

grid = GridSearchCV(SVC(), param_grid,
     cv=5, scoring='accuracy')
grid.fit(X_train_scaled, y_train)
```

Best Parameters

Kernel : rbf
C : 10
Gamma : 0.1

Best CV Accuracy
100%

8. When to Choose Which SVM?

Kernel	Best For	Pros	Cons
Linear	Linearly separable data, high-dim	Fast, simple	Poor for non-linear data
Polynomial	Medium complexity non-linear data	Captures curved boundaries	May overfit
RBF	Most real-world non-linear data	Best performance, flexible	Slower, needs tuning
Sigmoid	Neural network like problems	Works in some cases	Less used

9. Real-Life Uses of SVM

- **Medical Diagnosis** (Cancer detection)
- **Spam Filtering** (Email classification)
- **Face Recognition** (Image classification)
- **Text Classification** (Sentiment analysis)
- **Bioinformatics** (Gene classification)
- **Stock Prediction** (Trend classification)

Key Takeaways

- SVM finds the optimal hyperplane to separate classes with maximum margin.
- **RBF kernel** works best in most real-world, non-linear datasets.
- **Feature scaling** is a must for SVM.
- Use **GridSearchCV** to find the best parameters.
- SVM is powerful, accurate and effective, especially for small to medium datasets.

Support Vector Machine (SVM) – Hands-on with Real Dataset

From Data to Decision Boundary

1. Dataset: Iris Flower

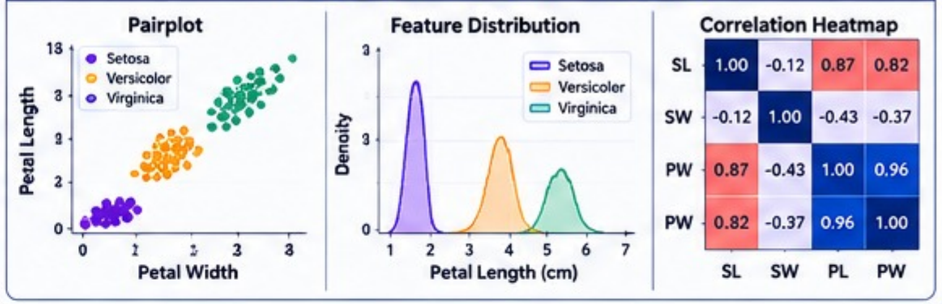
- One of the most popular ML datasets
- 150 samples, 4 numerical features
- 3 classes: Setosa, Versicolor, Virginica
- Goal: Classify flower species based on features

2. Dataset Preview

SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
6.2	3.4	5.4	2.3	virginica
5.9	3.0	5.1	1.8	virginica
6.7	3.1	4.7	1.5	versicolor

Shape: (150, 5) → 150 rows, 5 columns

3. Exploratory Data Analysis



4. SVM Model Code (Python)

```python
# 1. Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix,
  ConfusionMatrixDisplay

# 2. Load Dataset
df = pd.read_csv("https://raw.githuburraetont.com/mwaskom/
seaborn-data/master/iris.csv")

# 3. Features and Target
X = df.drop('species', axis=1)
y = df['species']

# 4. Split Data
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y)

# 5. Feature Scaling (important for SVM)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Train SVM (RBF Kernel)
model = SVC(kernel='rbf', C=1, gamma='scale', random_state=42)
model.fit(X_train_scaled, y_train)

# 7. Predictions
y_pred = model.predict(X_test_scaled)

# 8. Evaluation
print("Accuracy="; model.score(X_test_scaled, y_test))
print(classification_report(y_test, y_pred))
```

5. Decision Boundaries (2D View)

6. Model Evaluation

Test Accuracy	Precision	Recall	F1-Score
100%	1.00	1.00	1.00

Classification Report

	Precision	Recall	F1-Score	Support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	10
virginica	1.00	1.00	1.00	10
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Confusion Matrix

7. Hyperparameter Tuning (Find Best SVM)

```python
param_grid = {
  'C': [0.1, 1, 10, 100],
  'gamma': ['scale', 0.01, 0.1, 1],
  'kernel': ['linear', 'rbf', 'poly']
}

grid = GridSearchCV(SVC(), param_grid,
     cv=5, scoring='accuracy')
grid.fit(X_train_scaled, y_train)
```

Best Parameters

Kernel : rbf
C : 10
Gamma : 0.1

Best CV Accuracy
100%

8. When to Choose Which SVM?

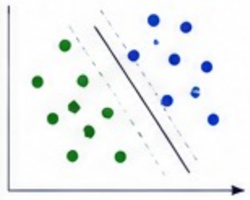
Kernel	Best For	Pros	Cons
Linear	Linearly separable data, high-dim	Fast, simple	Poor for non-linear data
Polynomial	Medium complexity non-linear data	Captures curved boundaries	May overfit
RBF	Most real-world non-linear data	Best performance, flexible	Slower, needs tuning
Sigmoid	Neural network like problems	Works in some cases	Less used

9. Real-Life Uses of SVM

- **Medical Diagnosis** (Cancer detection)
- **Spam Filtering** (Email classification)
- **Face Recognition** (Image classification)
- **Text Classification** (Sentiment analysis)
- **Bioinformatics** (Gene classification)
- **Stock Prediction** (Trend classification)

Key Takeaways

- SVM finds the optimal hyperplane to separate classes with maximum margin.
- **RBF kernel** works best in most real-world, non-linear datasets.
- **Feature scaling** is a must for SVM.
- Use **GridSearchCV** to find the best parameters.
- SVM is powerful, accurate and effective, especially for small to medium datasets.



SUPPORT VECTOR MACHINE (SVM)

POWERFUL. VERSATILE. REAL-WORLD IMPACT.

SVM is a supervised machine learning algorithm used for classification and regression.
It works by finding the optimal hyperplane that best separates data points of different classes.

1. WHAT IS SVM?

- SVM finds the decision boundary (hyperplane) that maximizes the margin between classes.
- Works well in high-dimensional spaces.
- Effective for both linear and non-linear data (using kernels).

2. HOW SVM WORKS (CLASSIFICATION)

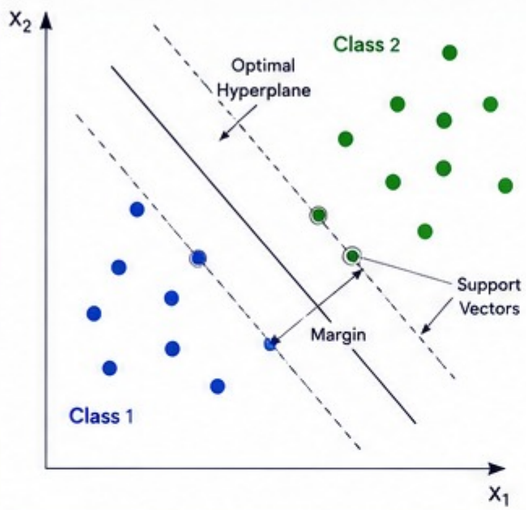
1 Find the best separating hyperplane
That maximizes the margin.

2 Support Vectors
The nearest points to the hyperplane are called support vectors. They define the boundary.

3 Maximize Margin
SVM chooses the hyperplane with the largest margin for better generalization.

4 Make Prediction
New data points are classified based on which side of the hyperplane they fall.

3. SVM: MAXIMUM MARGIN HYPERPLANE (2D EXAMPLE)



Find the hyperplane that maximizes the margin and minimizes error.

SVM Decision Function

$$f(x) = w \cdot x + b$$

Predict:
Class 1 if $f(x) < 0$
Class 2 if $f(x) > 0$

Mathematical Objective (Hard Margin)

$$\text{Minimize } \frac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1, \qquad \forall i$$

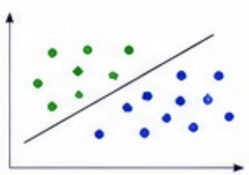
Where, w = weight vector, b = bias, $y_i \in \{-1, +1\}$, x_i = training points

For non-linearly separable data, SVM uses Kernels to map data to higher dimensions.

4. KERNEL TRICK – HANDLE NON-LINEAR DATA

Linear Kernel

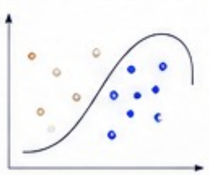
$$K(x, z) = x^T z$$



Used when data is linearly separable.

Polynomial Kernel

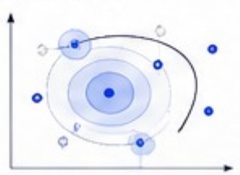
$$K(x, z) = (\gamma x^T z + r)^d$$



Captures polynomial relationships.

RBF (Gaussian) Kernel

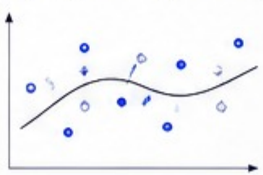
$$K(x, z) = e^{-\gamma\|x-z\|^2}$$



Most popular. Handles complex non-linear data.

Sigmoid Kernel

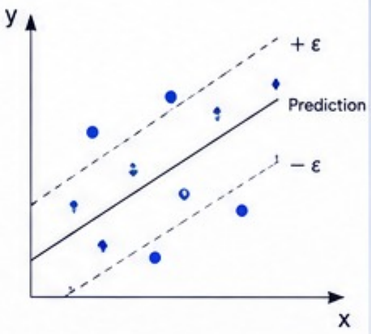
$$K(x, z) = \tanh(\gamma x^T z + r)$$



Similar to neural networks.

5. SVM FOR REGRESSION (SVR)

SVM can also be used for regression using an ϵ-insensitive loss function.



Points within the ϵ-tube are ignored.

Only points outside the tube contribute to the loss.

Why SVM?

- Works well in high-dimensional spaces
- Effective with small to medium datasets
- Robust to overfitting (with proper tuning)
- Uses only support vectors (efficient)
- Versatile: classification, regression, outlier detection, etc.

SVM IN REAL LIFE – WHERE IT MAKES AN IMPACT

1. MEDICAL DIAGNOSIS

SVM helps classify medical data to detect diseases.

Example: Cancer detection, tumor classification, heart disease prediction

2. SPAM DETECTION

Classifies emails as spam or not spam.

Example: Email filtering in Gmail, Outlook, Yahoo

3. FACE RECOGNITION

SVM is used to classify and verify human faces.

Example: Unlocking smartphones, security systems

4. CUSTOMER SEGMENTATION

Groups customers based on their behavior and attributes.

Example: Targeted marketing, recommendation systems

5. STOCK MARKET PREDICTION

Predicts stock price movement (up/down).

Example: Algorithmic trading, trend prediction

6. FRAUD DETECTION

Detects fraudulent transactions in banking and credit card systems.

Example: Anomaly detection in transaction data

7. TEXT & SENTIMENT ANALYSIS

Classifies text into categories or sentiment (positive/negative).

Example: Movie reviews, social media sentiment analysis

KEY TAKEAWAY: SVM is a powerful and flexible algorithm used across industries—from healthcare to finance.
Its ability to handle complex data with high accuracy makes it a go-to choice for real-world machine learning problems.

SVM = Simplicity in concept,
Power in performance, Impact in reality.

NAIVE BAYES

Simple Assumption. Powerful Predictions.

WHAT IS NAIVE BAYES?

Naive Bayes is a probabilistic machine learning algorithm based on Bayes' Theorem with a "naive" (independence) assumption between features.

WHY "NAIVE"?

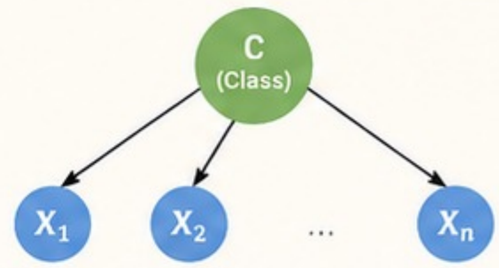
- It assumes that all features are independent of each other given the class.
- This makes computation simple and fast, even for high-dimensional data!

BAYES' THEOREM

$$P(C|X) = \frac{P(X|C)\ P(C)}{P(X)}$$

$P(C|X)$ = Posterior probability (Probability of class C given features X)
$P(X|C)$ = Likelihood (Probability of features X given class C)
$P(C)$ = Prior probability (Probability of class C)
$P(X)$ = Evidence (Probability of features X)

NAIVE BAYES ASSUMPTION



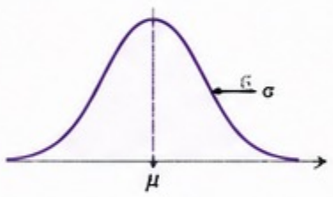
Given the class C, all features (X_1, X_2, ..., X_n) are conditionally independent.

$$P(X_1, X_2, ..., X_n \mid C) = \prod_{i=1}^{n} P(X_i \mid C)$$

TYPES OF NAIVE BAYES

1. Gaussian Naive Bayes (For Continuous Data)

Assumes features follow a Gaussian (Normal) distribution.



Used in: height, weight, income, etc.

2. Multinomial Naive Bayes (For Discrete Count Data)

Used for word counts or frequency data like text classification.

Used in: spam detection, text categorization

3. Bernoulli Naive Bayes (For Binary Data)

Used for binary/boolean features (0 or 1).

Feature	Present (1)	Absent (0)
Word in email	1	0
Has fever	1	0

Used in: spam detection, medical diagnosis

HOW NAIVE BAYES WORKS?

1. **Calculate Prior Probability $P(C)$**
   Probability of each class.
2. **Calculate Likelihood $P(X_i|C)$**
   Probability of each feature given the class.
3. **Compute Posterior $P(C|X)$**
   Use Bayes' Theorem (multiply likelihoods and prior).
4. **Predict Class**
   Choose the class with highest posterior probability.

PROS & CONS

Advantages

- Simple and fast
- Works well with small data
- Handles high-dimensional data
- Good for text classification

Limitations

- Assumes feature independence (may not hold true)
- Zero probability problem
- (solved using Laplace Smoothing)
- Less accurate when features are highly correlated

WORKED EXAMPLE (SPAM DETECTION)

Dataset

Email	Contains 'offer'	Contains 'win'	Class
E1	1	1	Spam
E2	1	0	Spam
E3	0	1	Spam
E4	0	0	Not Spam
E5	1	0	Not Spam
E6	0	1	Not Spam
E7	0	0	Not Spam
E8	1	1	Spam

We want to classify a new email:
X = {offer = 1, win = 0) → Spam or Not Spam?

Step 1: Prior Probability

P(Spam) = $\frac{4}{8}$ = 0.5
P(Not Spam) = $\frac{4}{8}$ = 0.5

Step 2: Likelihoods (Using Laplace Smoothing)

Feature	P(Feature=1 | Spam) (Count+1)/(N+2)	P(Feature=0 | Spam) (Count+1)/(N+2)
offer	(3+1)/(4+2) = 4/6 = 0.667	(1+1)/(4+2) = 2/6 = 0.333
win	(3+1)/(4+2) = 4/6 = 0.667	(1+1)/(4+2) = 2/6 = 0.333

For Not Spam:

Feature	P(Feature=1 | Not Spam) (Count+1)/(N+2)	P(Feature=0 | Not Spam) (Count+1)/(N+2)
offer	(1+1)/(4+2) = 2/6 = 0.333	(3+1)/(4+2) = 4/6 = 0.667
win	(1+1)/(4+2) = 2/6 = 0.333	(3+1)/(4+2) = 4/6 = 0.667

Step 3: Posterior Calculation

For Spam:

P(Spam|X) ∝ P(offer=1|Spam) × P(win=0|Spam) × P(Spam)
= 0.667 × 0.333 × 0.5 = 0.111

For Not Spam:

P(NotSpam|X) ∝ P(offer=1|NotSpam) × P(win=0|NotSpam) × P(NotSpam)
= 0.333 × 0.667 × 0.5 = 0.111

Step 4: Prediction

Both probabilities are equal.
(In case of tie, choose based on prior or use more data/features.)
Predicted Class: Spam / Not Spam (Tie)

USE CASES

- Spam Email Detection
- Disease Diagnosis
- Text Classification
- Sentiment Analysis
- Recommendation Systems
- Customer Churn Prediction

KEY TAKEAWAY

Naive Bayes is based on Bayes' Theorem with an independence assumption.
It is simple, efficient and works surprisingly well in many real-world problems!

Simple Mathematics Powerful Predictions!

DEEP LEARNING – REAL LIFE EXAMPLES

Deep Learning models learn patterns from large amounts of data using Artificial Neural Networks (DNN, CNN, RNN, Transformers, etc.)

REAL LIFE APPLICATIONS

1. IMAGE RECOGNITION

Identify and classify objects in images.

Examples: Self-driving cars, security systems, photo tagging.

Models: CNN, ResNet, EfficientNet

2. SPEECH RECOGNITION

Convert speech to text.

Examples: Voice assistants (Siri, Alexa), call transcription.

Models: RNN, LSTM, Transformers

3. NATURAL LANGUAGE PROCESSING (NLP)

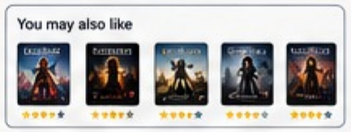
Understand, generate and analyze human language.

Examples: Chatbots, translation, sentiment analysis, summarization.

Models: LSTM, BERT, GPT, T5

4. RECOMMENDATION SYSTEMS

Recommend personalized items to users.



Examples: Netflix, Amazon, YouTube recommendations.

Models: DNN, Embeddings, Wide & Deep

5. FRAUD DETECTION

Detect unusual patterns in transactions.

Examples: Credit card fraud detection, insurance fraud, cyber fraud.

Models: DNN, Autoencoders, LSTM

6. MEDICAL IMAGING

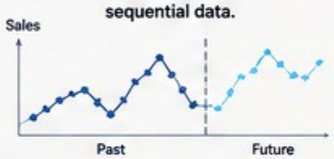
Detect diseases and abnormalities from medical images.

Examples: Cancer detection, tumor segmentation, fracture detection.

Models: CNN, U-Net, ResNet

7. TIME SERIES FORECASTING

Predict future values from sequential data.



Examples: Stock price prediction, demand forecasting, weather forecasting.

Models: LSTM, GRU, Temporal CNN

8. LANGUAGE TRANSLATION

Translate text from one language to another.

Examples: Google Translate, real-time translation apps.

Models: Seq2Seq, Transformer

9. OBJECT DETECTION

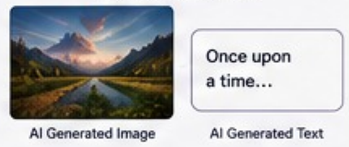
Detect and locate multiple objects in an image.

Examples: Autonomous vehicles, retail analytics, surveillance.

Models: YOLO, SSD, Faster R-CNN

10. GENERATIVE AI

Generate new content like images, text, music, videos.



Examples: DALL-E (images), ChatGPT (text), DeepMusic (music).

Models: GANs, Transformers, Diffusion

11. ANOMALY DETECTION (INDUSTRY)

Detect defects or anomalies in industrial systems.

Examples: Predictive maintenance, defect detection in manufacturing.

Models: Autoencoders, CNN, LSTM

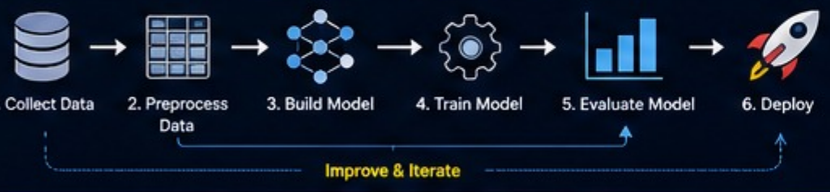
12. CUSTOMER CHURN PREDICTION

Predict customers who are likely to stop using a service.

Examples: Telecom, SaaS, Banking customer retention.

Models: DNN, XGBoost + DL, RNN

TYPICAL DEEP LEARNING WORKFLOW



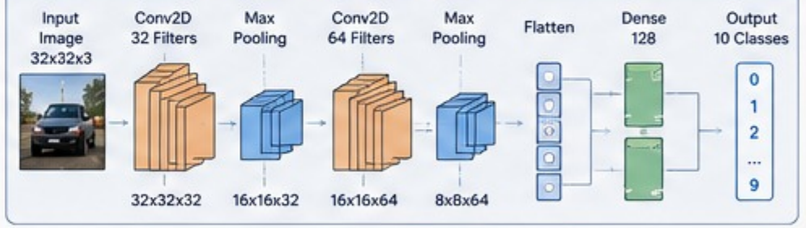
COMMON DEEP LEARNING ARCHITECTURES

CNN (Convolutional Neural Network)	RNN / LSTM / GRU (Recurrent Neural Networks)	Transformers (Attention Mechanism)	Autoencoders (Encoder–Decoder)	GANs (Generative Adversarial Networks)
Best for images	Best for sequences, time series, text	Best for NLP, translation, generation	Best for anomaly detection, dimensionality reduction	Best for gneerating new data

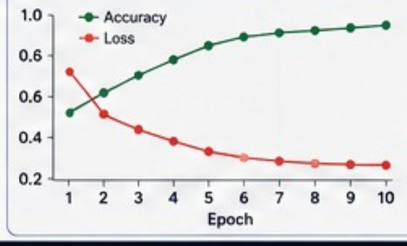
EXAMPLE: IMAGE CLASSIFICATION WITH CNN (Keras)

```python
import tensroflow as tf
from tensorflow.keras import layers, models

# 1. Load and preprocess data (Example: CIFAR-10)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
x_train, x_test = x_train/255.0, x_test/255.0

# 2. Build CNN Model
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu' ),
    layers.Dense(10, activation='softmax')
])
# 3. Compile
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
# 4. Train
history = model.fit(x_train, y_train, epochs=10,
          validation_data=(x_test, y_test))
# 5. Evaluate
loss, acc = model.evaluate(x_test, y_test, verbose=0)
print(f"Test Accuracy: {acc*100:.2f}%")
```

CNN ARCHITECTURE DIAGRAM



SAMPLE OUTPUT



Model Evaluation	
Test Accuracy	: 87.42%
Test Loss	: 0.48
Total Params	: 1,208,746

KEY TAKEAWAYS

- ✓ Deep learning learns representations automatically from data.
- ✓ It powers many real-world applications across industries.
- ✓ The choice of model depends on the type of data and problem.

TOOLS & LIBRARIES

TensorFlow · Keras · PyTorch · scikit-learn · OpenCV · Hugging Face Transformers

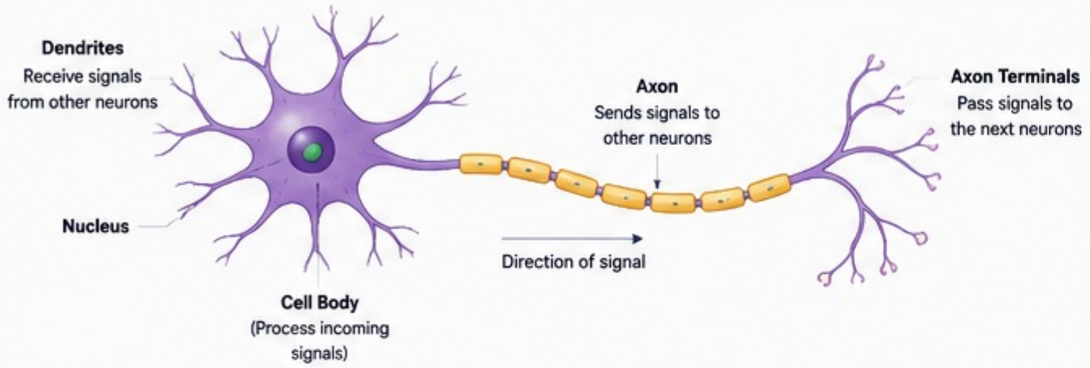
USES ACROSS INDUSTRIES

Healthcare · Finance · E-commerce · Automotive · Entertainment · Education · Manufacturing

BIOLOGICAL NEURON & DEEP LEARNING

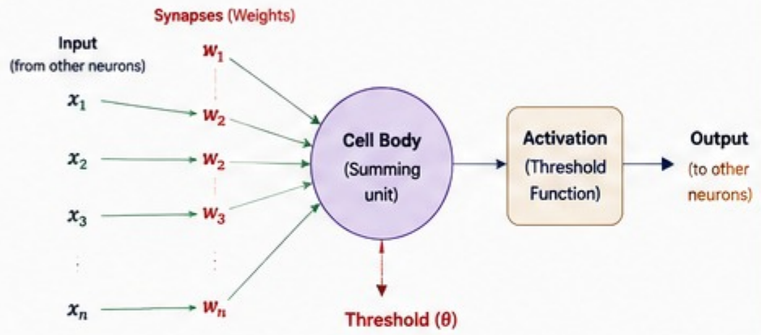
Deep Learning is inspired by how biological neurons in our brain work!

1. BIOLOGICAL NEURON (NERVE CELL)



How a Biological Neuron Works

1. Dendrites receive signals (electrical impulses).
2. Cell body processes all incoming signals.
3. If the signal is strong enough (above threshold), neuron fires.
4. Signal is sent through axon to other neurons.
5. Neurotransmitters at axon terminals pass signal to the next neuron.



Mathematical Model of Biological Neuron

Summation of inputs coming from other neurons (weighted sum)

$$z = \sum_{i=1}^{n} w_i x_i + b$$

Activation function decides whether neuron fires or not

$$y = f(z)$$

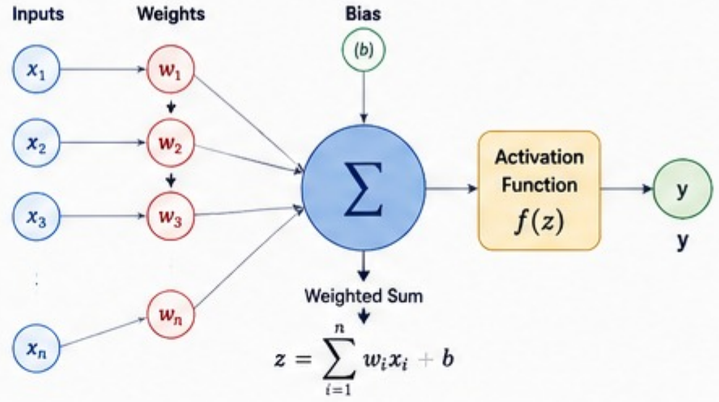
Where,

- x_i = input signals
- w_i = synaptic weights
- b = bias (threshold adjustment)
- $f(z)$ = activation function (Output)

Common Activation Functions

2. ARTIFICIAL NEURON (IN DEEP LEARNING)

An artificial neuron (perceptron) works similarly!



Biological Neuron vs Artificial Neuron

Biological Neuron	Artificial Neuron
Dendrites	Inputs ($x_1, x_2, ..., x_n$)
Synapses	Weights ($w_1, w_2, ..., w_n$)
Cell Body	Summation ($\sum$)
Threshold	Bias (b)
Activation	Activation Function (f)
Axon	Output (y)
Axon Terminals	Output to next layer

Activation Functions (Used in Deep Learning)

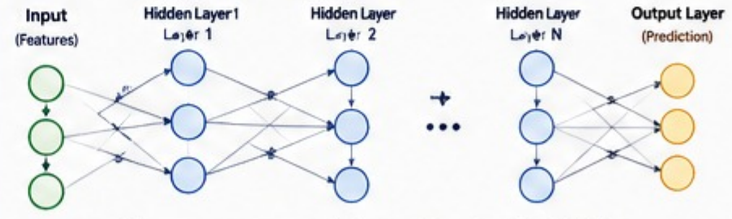
Sigmoid	Tanh	ReLU	Leaky ReLU	Softmax
$\sigma(z) = \frac{1}{1+e^{-z}}$	$\tanh(z)$	$\max(0, z)$	$\max(0.01z, z)$	$\sigma_i(z) = \frac{e^{z_i}}{\sum_j e^{z_j}}$

What Happens?

- The neuron takes many inputs.
- Multiplies each input by a weight.
- Adds them up and adds bias.
- Passes through activation function.
- Produces an output.

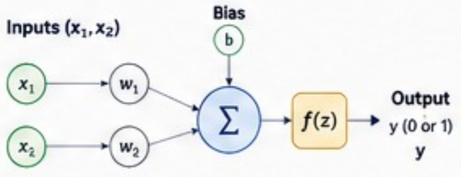
This is the basic building block of Neural Networks!

3. FROM SINGLE NEURON TO DEEP NEURAL NETWORK



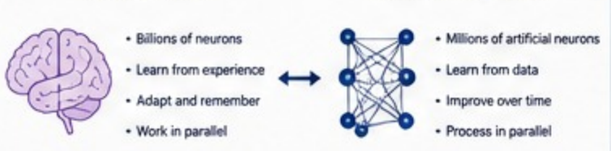
Many neurons connected in layers = **Deep Learning Model**

4. SIMPLE EXAMPLE (Binary Classification)



Predict: Is this email Spam (1) or Not Spam (0)?

5. ANALOGY



6. REAL LIFE APPLICATIONS

7. KEY TAKEAWAYS

- Biological neurons inspired artificial neurons.
- Deep Learning uses layers of artificial neurons.
- Weights are learned from data using training.
- Activation functions introduce non-linearity.
- This enables machines to learn complex patterns!

8. PRACTICAL TIP

The magic of Deep Learning lies in adjusting weights and bias automatically (using algorithms like Backpropagation and Gradient Descent) to minimize error and make better predictions.

WEIGHTS, BIAS & FORWARD PROPAGATION

PRACTICE ON HOUSING.CSV (Linear Regression with Neural Network)

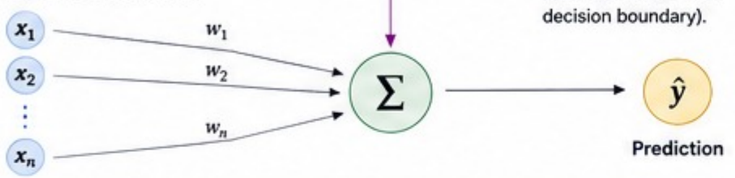
1. WEIGHTS & BIAS – WHAT & WHY?

Weights (w)

- Determine the importance of each input feature.
- Higher weight → more influence on prediction.

Bias (b)

- Acts as an additional adjustable parameter.
- Helps the model fit data better (shifts the line/ decision boundary).



$$z = w_1x_1 + w_2x_2 + ... + w_nx_n + b \quad \text{(Linear Combination)}$$

$$\hat{y} = z \quad \text{(For Linear Regression Output)}$$

2. FORWARD PROPAGATION (Single Neuron)

Step-by-step computation (No training here, just forward pass)

1. Take input features $x = [x_1, x_2, ..., x_n]$
2. Multiply each input by corresponding weight w
3. Sum all products
4. Add bias b
5. Output prediction (for linear regression): $\hat{y} = z$

Mathematical Formulation

$$z = \sum_{i=1}^{n} w_i x_i + b$$

$$\hat{y} = z$$

Example (n = 3)

Feature	Value (x)	Weight (w)	$w_i x_i$
x_1	2.0	0.5	1.00
x_2	1.5	-0.8	-1.20
x_3	3.0	1.2	3.60

Bias (b)	0.7
Sum (z)	1.00 – 1.20 + 3.60 + 0.7 = 4.10
Prediction ($\hat{y}$)	4.10

3. PRACTICE WORKFLOW ON HOUSING.CSV

1. 1 Load Housing.csv
2. Explore & Preprocess Data
3. Select Features (X) and Target (y)
4. Initialize Weights (w) and Bias (b)
5. Perform Forward Propagation
6. Make Predictions
7. Evaluate (MSE, RMSE, MAE)

Goal: Predict House Price (price) using other features in the dataset.

4. PRACTICE: IMPLEMENTATION (PYTHON)

```python
# 1. Load and Explore
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error,mean_absolute_error

df = pd.read_csv('Housing.csv')
print(df.head())
print(df.info())

# Assuming target column name is 'price'
df = df.dropna()
```

```python
# 2. Features and Target
X = df.drop('price', axis=1).values
y = df['price'].values.reshape(-1, 1)
feature_names = df.drop('price', axis=1).columns.tolist()
print('Features:'; feature_names)
print('Target: price')
```

```python
# 3. Train-Test Split & Scaling
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# 4. Initialize Weights and Bias
np.random.seed(42)
n_features = X_train_scaled.shape[1]
w = np.random.randn(n_features, 1) * 0.01  # small random weights
b = np.zeros((1,))                         # bias

print('Initial Weights:\n', w.T)
print('Initial Bias:', b[0])
```

```python
# 5. Forward Propagation
def forward_propagation(X, w, b):
    z = np.dot(X, w) + b    # linear combination
    y_pred = z              # for linear regression
    return y_pred

y_pred_train = forward_propagation(X_train_scaled, w, b)
y_pred_test  = forward_propagation(X_test_scaled, w, b)

# 6. Evaluation
mse  = mean_squared_error(y_test, y_pred_test)
rmse = np.sqrt(mse)
mae  = mean_absolute_error(y_test, y_pred_test)

print(f"MSE : {mse:.4f}")
print(f"RMSE: {rmse:.4f}")
print(f"MAE  : {mae:.4f}")
```

5. SAMPLE FORWARD PROPAGATION (1 SAMPLE)

Feature	x (Sample 1)	Weight (w)	$w_i x_i$
area	2600	0.0021	5.4600
bedrooms	3	-0.3150	-0.9450
bathrooms	2	0.4100	0.8200
stories	2	0.1200	0.2400
mainroad	1	0.3050	0.3050
guestroom	0	-0.1500	0.0000
...	...	...	...

Bias (b)	1.2500
Sum (z)	7.1300
Prediction ($\hat{y} = z$)	7.1300

$$z = \sum (w_i x_i) + b$$
$$= 5.4600 - 0.9450 + 0.8200 + 0.2400 + 0.3050 + 0 + ... + 1.2500$$
$$= 7.1300$$

$$\hat{y} = z = 7.1300$$

(Predicted House Price)

This is the output of the neuron before any training/learning.

6. DATASET (HOUSING.CSV)

area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	parking	furnishingstatus	price
10500	3	1	1	yes	no	no	no	1	furnished	39.07
2600	3	1	2	yes	no	yes	no	1	semi-furnished	120.00
3060	2	1	1	yes	no	no	no	1	semi-furnished	83.00
6650	3	2	1	yes	yes	no	no	2	furnished	39.00
9600	2	1	1	yes	no	yes	no	1	semi-furnished	70.00
...	...	...	...	...	...	...	...	...	...	...

Target Column: price (House price in lakhs)

Preprocess: Convert categorical → numeric, handle missing, scale features

7. EVALUATION METRICS

MSE (Mean Squared Error)

$$\frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

Lower is better

RMSE (Root MSE)

$$\sqrt{\frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2}$$

Lower is better

MAE (Mean Absolute Error)

$$\frac{1}{m}\sum_{i=1}^{m}|y_i - \hat{y}_i|$$

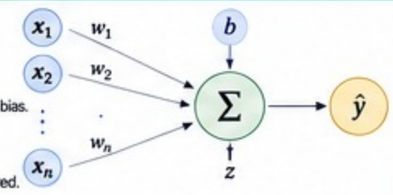
Lower is better

Note:

- Forward Propagation gives predictions using current weights and bias.
- To improve predictions, we need Training (Backpropagation + Gradient Descent).
- Here we only practiced Forward Pass (no training).

8. KEY TAKEAWAYS

- ✅ Weights control the importance of each feature.
- ✅ Bias shifts the prediction.
- ✅ Forward Propagation computes output from inputs, weights & bias.
- ✅ On Housing.csv, we used forward pass to predict prices.
- ✅ To improve accuracy, training (learning weights & bias) is required.

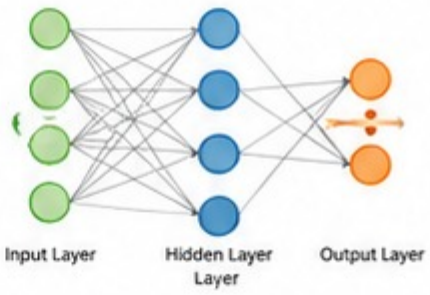


NEXT STEPS (For Full Learning)

Backpropagation → Gradient Descent → Update Weights & Bias → Train Model → Better Predictions

BACKWARD PROPAGATION
IMPLEMENTATION ON DATASETS

From Theory → Code → Datasets → Results



What is Backpropagation?

Backpropagation is an algorithm used to train neural networks. It computes the gradient of the loss function w.r.t all parameters (weights & biases) using the chain rule and updates them to minimize the error.

1. BACKPROPAGATION WORKFLOW

1. Forward Pass
Compute output & loss

2. Compute Loss
Measure error (e.g., MSE)

3. Backward Pass
Compute gradients using chain rule

4. Update Parameters
Update weights & biases (Gradient Descent)

5. Repeat
Iterate until convergence

2. MATHEMATICAL FOUNDATION (For a 1 Hidden Layer NN)

Let:

$$z^{[1]} = W^{[1]}X + b^{[1]} \quad \text{(hidden layer linear output)}$$

$$a^{[1]} = g(z^{[1]}) \quad \text{(activation)}$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]} \quad \text{(output layer linear output)}$$

$$\hat{y} = a^{[2]} = \sigma(z^{[2]}) \quad \text{(prediction)}$$

Loss (MSE): $L = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2$

Backward Pass (Gradients)

$$dZ^{[2]} = \hat{Y} - Y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]}(a^{[1]})^T$$

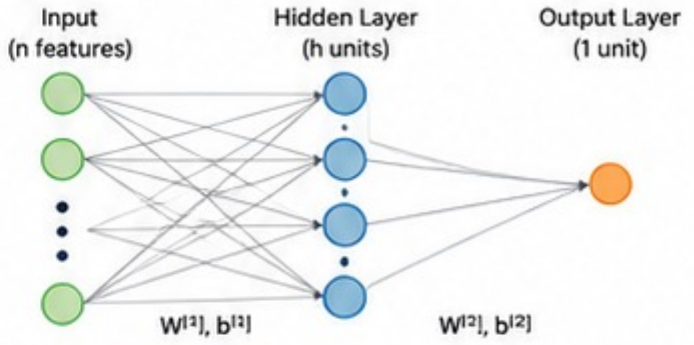
$$db^{[2]} = \frac{1}{m}\sum dZ^{[2]}$$

$$dZ^{[1]} = (W^{[2]})^T dZ^{[2]} * g'(Z^{[1]})$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]}X^T$$

$$db^{[1]} = \frac{1}{m}\sum dZ^{[1]}$$

3. NEURAL NETWORK STRUCTURE (Example)



4. IMPLEMENTATION IN PYTHON (From Scratch)

```python
import numpy as np

class NeuralNetwork:
    def __init__(self, n_x, n_h, n_y, lr=0.01):
        np.random.seed(42)
        self.W1 = np.random.randn(n_h, n_x) * 0.01
        self.b1 = np.zeros((n_h, 1))
        self.W2 = np.random.randn(ny, n_h) * 0.01
        self.b2 = np.zeros((n_y, 1))
        self.lr = lr

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def sigmoid_derivative(self, z):
        s = self.sigmoid(z)
        return s * (1 - s)

    def forward(self, X):
        self.Z1 = np.dot(self.W1, X) + self.b1
        self.A1 = self.sigmoid(self.Z1)
        self.Z2 = np.dot(self.W2, self.A1) + self.b2
        self.A2 = np.sigmoid(self.Z2)
        return self.A2

    def backward(self, X, Y):
        m = X.shape[1]
        dZ2 = self.A2 - Y
        dW2 = (1/m) * np.dot(dZ2, self.A1.T)
        db2 = (1/m) * np.sum(dZ2, axis=1, keepdims=True)
        dZ1 = np.diot(self.W2.T, dZ2) * self.sigmoid_derivative(self.Z1)
        dW1 = (1/m) * np.dot(dZ1, XT)
        db1 = (1/m) * np.sum(dZ1, axis=1, keepdims=True)
        self.W1 -= self.lr * dW1
        self.b1 -= self.lr * db1
        self.W2 -= self.lr * dW2
        self.b2 -= self.lr * db2

    def train(self, X, Y,
    for i in range(epochs=10000):
        self.forward(X)cha):
        self.backward(X, Y)
        if i % 1000 == 0:
          loss = np.mean((self.A2 - Y)**2)
        print(f"Epoch {i, Loss: {loss::60}")
```

5. DATASETS WE CAN USE

Dataset	Task	Features	Samples	Description
Iris (Classification)	3-Class Classification	4	150	Flower species classification
Breast Cancer Wisconsin	Binary Classification	30	569	Predict malignant or benign
Diabetes Regression	Regression	10	442	Predict disease progression
Housing Prices	Regression	13	506	Predict median house price
MNIST (Digits)	Multi-class Classification	784	70,000	Handwritten digit recognition

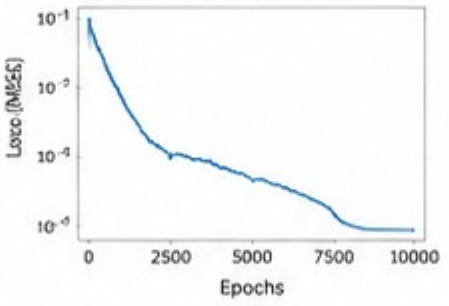
6. EXAMPLE: TRAINING ON IRIS DATASET (Classification)

Load Data (Iris Dataset) → Preprocess (One-hot Encode) → Train NN (Backpropagation) → Evaluate (Accuracy)

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.prrpecressing import OneHotEmcoder, StandardScaler
import numpy as np

# Load Iris Dataset
data = load_iris)
X = data.data.T                          # shape (4, 150)
y = data.target.reshape(1, -1)           # shape (1, 150)

# One-hot encode labels
ohe = OneHotEnceder(sparse_output=Falee)
y = ohe.fit_transform(y.T).T             # shape (3, 150)

# Standardize features
scaler = StandardScaler()
X = scaler.fit_transfarm(X.T).T          # shape (4, 150)

# Train/Test split
def split_data(X, Y, test_size=0.2):
    m = X.shape[1]
    idx = np.random.permutation(m)
    test_idx = idx[:int(m*test_size)]
    train_idx = idx[int(m*test_size):]
    retren X[:, train_ixd], X(:, test_ixd), Y(:, train_ixd), Y(:, test_idd)

X_train, X_test, Y_train, Y_test = split_data(X, y)

# Initialize & Train
nn = NeuralNetwork(n_x=4, n_h=8, n_y=3, lr=0.1)
nnn.train(X_train, Y_train, epochs=10000)

y_pred
print('Test Accuracy': np.mean(np.argmax(y_pred, ais=0) = np.art_set, ais=0)))
```

7. RESULTS (IRIS DATASET)

Training Loss Curve



Confusion Matrix (Test Set)

Actual \ Predicted	Setosa	Versicolor	Virginica
Setosa	10	0	0
Versicolor	0	9	1
Virginica	0	1	9

Test Accuracy

93.33%

8. KEY TAKEAWAYS

- Backpropagation uses chain rule to compute gradients efficiently.
- Works for any number of layers.
  Implemented from scratch to understand internals.
- Can be applied to any dataset (classification or regression).
- Foundation of deep learning frameworks (TensorFlow, PyTorch).

9. NEXT STEPS

- Try more datasets (MNIST, CIFAR-10, Housing Prices)
- Add more hidden layers (Deep Neural Networks)
- Use different activation functions (ReLU, Tanh)
- Optimize with Adam / RMSprop
- Build mini projects (Digit Recognizer, House Price Predictor)

PyTorch Real-Life Uses

PyTorch is a powerful open-source machine learning library used by researchers and industries worldwide to build AI solutions that **impact real lives.**

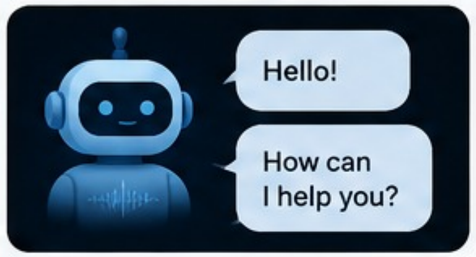
1 Computer Vision

Used in image classification, object detection, segmentation, and more.

Examples:

Meta Testa

2 Natural Language Processing (NLP)



Powering chatbots, language translation, text generation, sentiment analysis, and more.

Examples:

Microsoft Hugging Face

3 Speech Recognition & Audio Processing

Used in speech-to-text, voice assistants, speaker recognition, and audio classification.

Examples:

OpenAI Whisper

4 Recommendation Systems

Used to build personalized recommendations for movies, products, music, and content.

Examples:

Netflix amazon

5 Healthcare & Medical AI

Used in medical image analysis, disease prediction, drug discovery, and patient monitoring.

Examples:

MONAI JOHNS HOPKINS MEDICINE

6 Autonomous Vehicles & Robotics

Used for perception, object detection, path planning, and decision making in autonomous systems.

Examples:

Waymo BostonDynamics

7 Generative AI (GANs, Diffusion Models)

Used fo generate realistic images, art, videos, faces, and even 3D models.

Examples:

OpenAI stability.ai

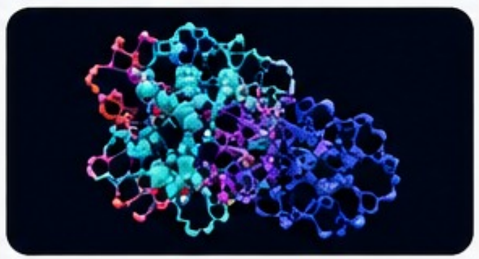
8 Fraud Detection & Anomaly Detection

Used in finance and commerce to detect fraudulent transactions and unusual patterns.

Examples:

PayPal VISA

9 Scientific Research & Simulations



Used in physics, chemistry, biology, and climate modeling for complex simulations.

Examples:

OpenFold PyG (PyTorch Geometric)

10 Time Series Forecasting & Predictive Analytics

Used to forecast stock prices, sales, demand, weather, and other time dependent data.

Examples:

Google Prophet (Meta)

PyTorch

Dynamic Computation Graph

Easy to Use Pythonic API

GPU Acceleration

Large Community & Ecosystem

Used in Research and Production Worldwide

PYTORCH – SAMPLE PROJECTS

Build AI solutions for real-world problems using PyTorch

1 House Price Prediction (Regression)

Real Life Use

Predict house prices based on area, bedrooms and age of the house.

Code

```
import torch
import torch.nn as nn
X = torch.tensor([[1000,2,5],
                  [1500,3,10],
                  [2000,4,3]], dtype=torch.float32)
y = torch.tensor([[50],[70],[90]], dtype=torch.float32)
model = nn.Linear(3,1)
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.000001)
for epoch in range(100):
    pred = model(X)
    loss = loss_fn(pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
print("Predicted Price:", model(torch.tensor([[1800,3,5]], dtype=torch.float32)))
```

Learns relationship between features and price (continuous value).

2 Student Pass/Fail Prediction (Classification)

Real Life Use

Predict whether a student will pass or fail based on study hours.

0 = Fail
1 = Pass

Code

```
import torch
import torch.nn as nn

X = torch.tensor([[2],[4],[6],[8]], dtype=torch.float32)
y = torch.tensor([[0],[0],[1],[1]], dtype=torch.float32)

model = nn.Sequential(nn.Linear(1,1), nn.Sigmoid())
loss_fn = nn.BCELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
for epoch in range(1000):
    pred = model(X)
    loss = loss_fn(pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
print("Prediction for 7 study hours:", model(torch.tensor([[7.0]])))
```

Outputs probability between 0 and 1 (>=0.5 => Pass).

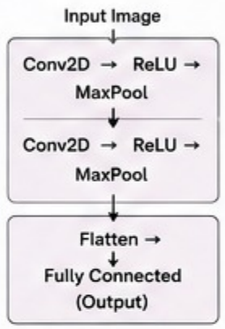
3 Image Classification – Cat vs Dog (CNN)

Real Life Use

Used in CCTV, face recognition, smart apps, medical imaging.

Code (Model Architecture)

```
import torch.nn as nn
model = nn.Sequential(
    nn.Conv2d(3,16,3,padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(16,32,3,padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(32*56*56,2)  # 2 classes: Cat, Dog
)
```

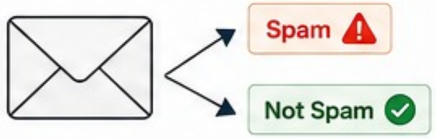


Convolution layers learn features like edges, shapes and patterns.

4 Spam Email Detection (NLP)

Real Life Use

Used in Gmail and other email services to filter spam messages.



Steps

1. Convert text to numbers (techniques like TF-IDF / Embeddings)
2. Create PyTorch Dataset and DataLoader
3. Build and train a Neural Network
4. Predict Spam (1) or Not Spam (0)

Example Data

Email	Label
Win money now!!!	1
Meeting at 5 PM	0
Congratulations! Claim prize	1
Let's catch up tomorrow	0

Text is converted to numbers and model learns patterns of spam.

5 Digit Recognition (MNIST) (Deep Learning)

Real Life Use

Used in ATM, cheque reading, postal code recognition, digit forms.

Code (Model)

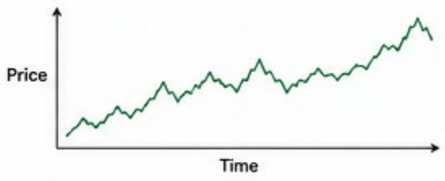
```
import torch.nn as nn
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28*28,128),
    nn.ReLU(),
    nn.Linear(128,10)  # 10 digits: 0-9
)
```

Model learns to classify images into 10 digit classes.

6 Stock Price Prediction (Time Series) (Regression)

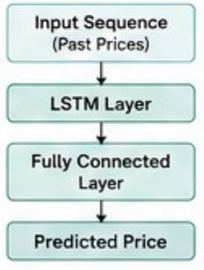
Real Life Use

Predict future stock prices to help in investment decision making.



Simple LSTM Model

```
import torch.nn as nn
class LSTMModel(nn.Module):
  def __init__(self):
      super().__init__()
      self.lstm = nn.LSTM(input_size=1, hidden_size=50,
                          num_layers=1, batch_first=True)
      self.fc = nn.Linear(50,1)
  def forward(self, x):
      out,_ = self.lstm(x)
      out = self.fc(out[:,-1,:])
      return out
```



LSTM captures time dependencies in stock price movement.

Why PyTorch?

- Easy to Use Pythonic
- Dynamic Computation Graph
- Great for Research & Production
- Strong Community Support
- GPU Acceleration

PyTorch

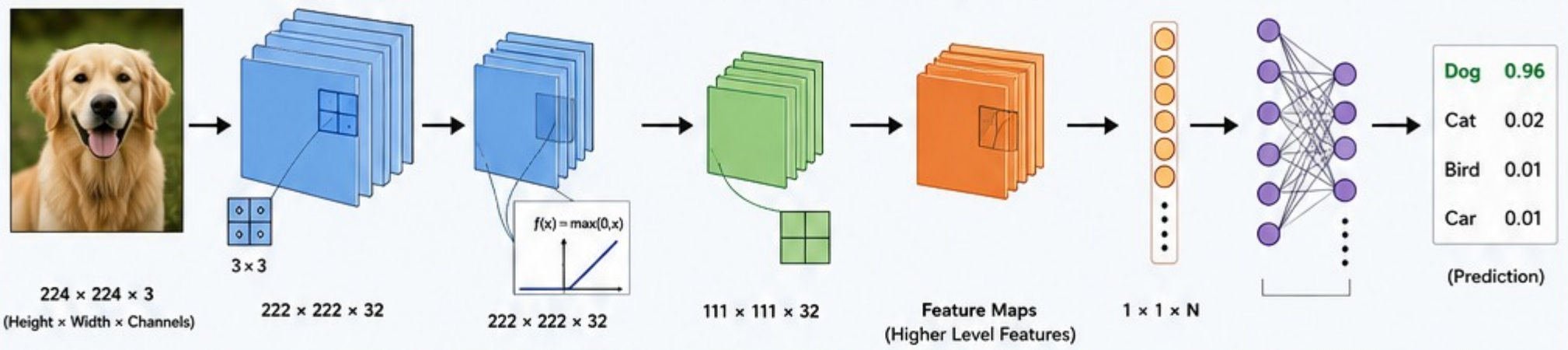
CNN (Convolutional Neural Network)

Real-Life Uses & Work at Each Layer

- Learns Features Automatically
- Requires Less Preprocessing
- Works with Images & Videos
- High Accuracy

CNN is a Deep Learning algorithm mainly used to analyze images and visual patterns.

CNN ARCHITECTURE – FROM IMAGE TO PREDICTION



Detects simple features like edges, lines, colors	Adds non-linearity Keeps positive values	Reduces spatial size Keeps important info	Learns complex features like parts of objects	Convert 2D/3D maps into 1D feature vector	Combines all features	Gives final prediction

WHAT EACH LAYER DOES?

Layer	Role / Work	What it Learns	Output / Effect
1 **Input Layer**	Takes the raw image as pixel values	No learning	Image converted to numbers (Matrix)
2 **Convolution Layer**	Applies filters (kernels) to detect patterns	Edges, lines, corners, basic textures	Feature maps (e.g., 222×222×32)
3 **ReLU Activation**	Applies ReLU function $f(x) = max(0, x)$	Adds non-linearity, keeps important signals	Negative values become 0, model learns complex patterns
4 **Pooling Layer** (Max Pooling)	Downsamples the feature maps	Retains important features, reduces computation	Smaller size feature maps (e.g., 111×111×32)
5 **Convolution Layers** (Deeper)	Applies multiple filters on previous maps	Textures → Parts of objects (eyes, nose, wheels, etc.)	High-level feature maps (more abstract)
6 **Flatten Layer**	Converts 2D/3D maps into 1D vector	No learning	Long vector containing all extracted features
7 **Fully Connected Layer**	Combines all features to make decision	Relationship between features	Scores for each class
8 **Output Layer** (Softmax)	Converts scores into probabilities	Final prediction (class with highest prob.)	Probability for each class (Sum = 1)

EXAMPLE: FACE RECOGNITION

Input: Photo of a person

CNN extracts features in layers

Learns Face Shape, Eyes, Nose, Patterns

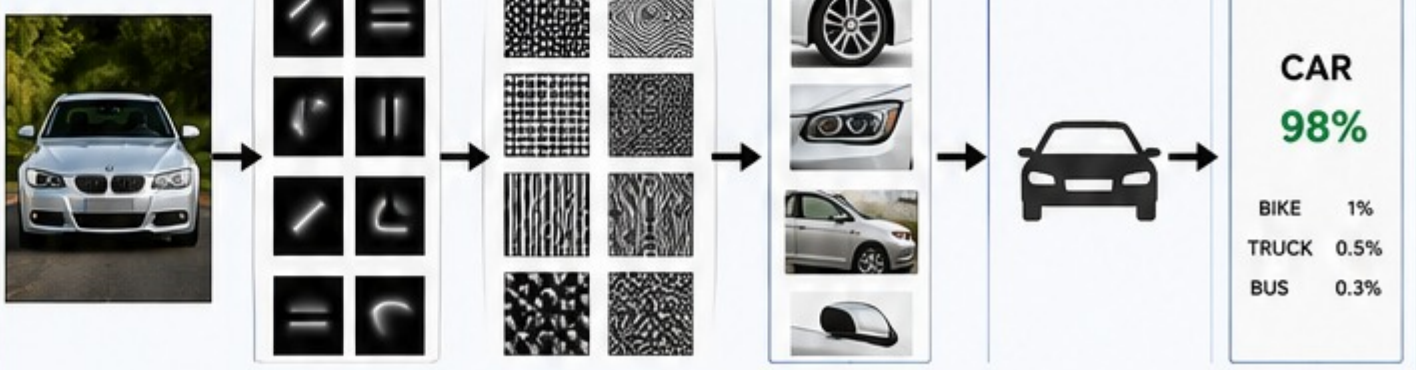
Compares with known faces

Output: Person A = 92%
Person B = 8%

REAL-LIFE USES OF CNN

- **Face Recognition** — Used in face unlock, attendance systems, security, tagging in photos
- **Self-Driving Cars** — Detects vehicles, pedestrians, traffic signs, lanes, obstacles
- **Medical Imaging** — Analyzes X-rays, CT, MRI scans to detect diseases, tumors
- **Image Classification** — Cat vs Dog, product classification, bird species, flowers, etc.
- **Object Detection** — Identifies and locates objects in images or videos
- **Satellite Imagery** — Detects buildings, roads, forests, water bodies, land use
- **Security & Surveillance** — Detects suspicious activities, weapon detection, crowd analysis
- **Retail & E-commerce** — Product recognition, visual search, recommendation systems

HOW CNN SEES AN IMAGE (HIERARCHY OF FEATURES)



What happens at each stage?

Raw pixels (RGB values)	CNN detects edges, lines, corners	CNN detects textures, patterns, combinations	CNN detects parts like wheels, lights, windows	CNN understands the complete object (Car)	CNN gives final prediction with probability

KEY TAKEAWAY

CNN automatically learns hierarchical features from images.

Pixels → Edges → Textures → Parts → Object → Prediction

More layers = More complex features
Better features = Better accuracy

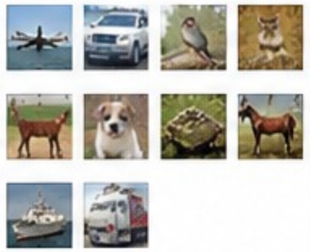
CNN IMPLEMENTATION ON CIFAR-10 DATASET

Image Classification (10 Classes)

CIFAR-10: 60,000 color images (32x32) in 10 classes, 6,000 images per class

1. CIFAR-10 DATASET OVERVIEW

Sample Images (32x32 RGB)



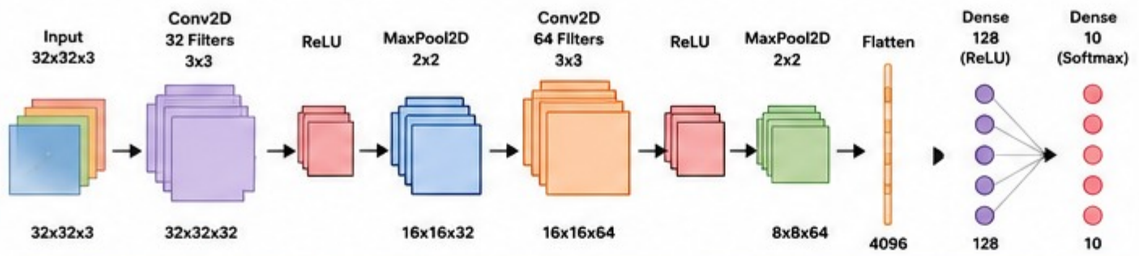
Classes (10)

0 - Airplane
1 - Automobile
2 - Bird
3 - Cat
4 - Deer
5 - Dog
6 - Frog
7 - Horse
8 - Ship
9 - Truck

Dataset Split

- Training Set : 50,000 images
- Test Set : 10,000 images
- Image Size : 32 x 32 x 3 (RGB)
- Total Classes : 10

2. CNN ARCHITECTURE



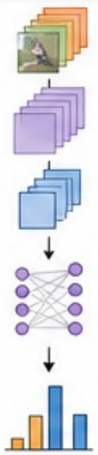
Layer (Type)	Output Shape	# Parameters	What it Does
Input Layer	32x32x3	0	Input CIFAR-10 image
Conv2D (32, 3x3)	32x32x32	896	Extracts low-level features (edges, colors)
MaxPool2D (2x2)	16x16x32	0	Reduces spatial size, keeps important features
Conv2D (64, 3x3)	16x16x64	18,496	Extracts higher-level features (textures, parts)
MaxPool2D (2x2)	8x8x64	0	Further reduces size
Flatten	4096	0	Converts 3D maps to 1D vector
Dense (128)	128	524,416	Learns combinations of features
Dense (10)	10	1,290	Outputs probability of 10 classes

Total Trainable Parameters: 545,098

3. IMPLEMENTATION (PYTHON – KERAS)

```python
import tensorflow as tf
from tensorflow.keras import layers, models

# 1. Load and preprocess CIFAR-10
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
x_train, x_test = x_train/255.0, x_test/255.0  # Normalize
y_train, y_test = y_train.flatten(), y_test.flatten()

# 2. Build CNN model
model = models.Sequential([
  layers.Conv2D(32, (3,3), padding='same', activation='relu',
        input_shape=(32,32,3)),
  layers.MaxPooling2D((2,2)),
  layers.Conv2D(64, (3,3), padding='same', activation='relu'),
  layers.MaxPooling2D((2,2)),
  layers.Flatten(),
  layers.Dense(128, activation='relu'),
  layers.Dense(10, activation='softmax')
])

# 3. Compile and train
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
history = model.fit(x_train, y_train, epochs=20,
                    validation_data=(x_test, y_test))
```

4. TRAINING PROCESS



Input Images (32x32x3)
↓
Convolutions (Feature Extraction)
↓
Pooling (Dimensionality Reduction)
↓
Fully Connected (Classification)
↓
Output (Class Probabilities)

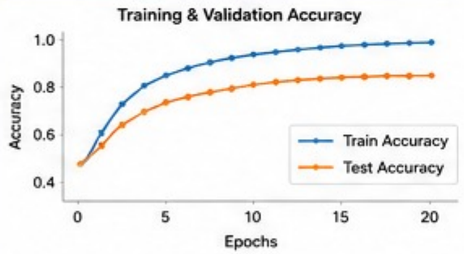
Training Configuration

- Optimizer : Adam
- Loss Function : Sparse Categorical Crossentropy
- Batch Size : 128
- Epochs : 20
- Learning Rate : 0.001

Training Steps

1. **Forward Pass** — Model predicts probabilities
2. **Loss Calculation** — Compare with true labels
3. **Backward Pass** — Compute gradients
4. **Update Weights** — Adam optimizer updates weights
5. **Repeat for all batches & epochs**

5. RESULTS



Final Performance on Test Set

Test Accuracy	Test Loss
79.21%	0.63

Class-wise Accuracy (Test Set)

Class	Accuracy (%)	Class	Accuracy (%)
Airplane	83.2	Dog	75.6
Automobile	89.1	Frog	84.3
Bird	72.4	Horse	79.0
Cat	67.8	Ship	86.7
Deer	73.6	Truck	80.2

Model achieves ~79% accuracy on CIFAR-10 test set with a simple CNN.

6. PREDICTIONS (EXAMPLES)

Image	True Label	Predicted Label	Confidence
	Airplane	Airplane	96.35%
	Automobile	Automobile	93.21%
	Bird	Bird	88.74%
	Truck	Truck	91.08%
	Ship	Ship	94.55%

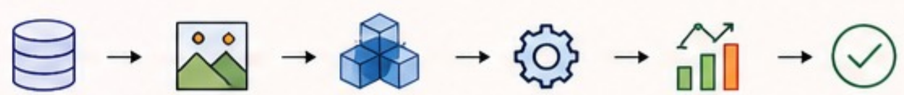
7. KEY TAKEAWAYS

- CNNs automatically learn hierarchical features from images.
- Convolution layers extract features; pooling reduces size.
- Deeper layers learn complex patterns for classification.
- CIFAR-10 is a great benchmark for image classification.
- This simple CNN can be improved using more layers, BatchNorm, Dropout, Data Augmentation, etc.

8. POSSIBLE IMPROVEMENTS

- Add more Conv layers
- Use Batch Normalization
- Add Dropout for regularization
- Data Augmentation (flip, crop, etc.)
- Use Learning Rate Scheduler
- Try advanced architectures (ResNet, DenseNet, etc.)

9. COMPLETE PIPELINE OVERVIEW



RNN (RECURRENT NEURAL NETWORK)

Understand Sequences, Remember Context, Make Predictions

1. WHAT IS RNN?

RNN (Recurrent Neural Network) is a type of neural network designed for sequential data. It maintains a "memory" (hidden state) that captures information about previous inputs in the sequence.

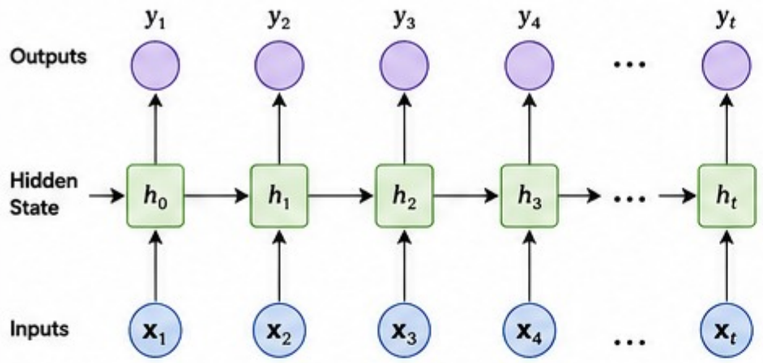
Real-Life Analogy

Like reading a sentence word by word and remembering the meaning of previous words to understand the next word.

Used For

- Text & Speech Processing
- Language Translation
- Time Series Forecasting
- Sentiment Analysis
- Music & Video Understanding
- Chatbots & More

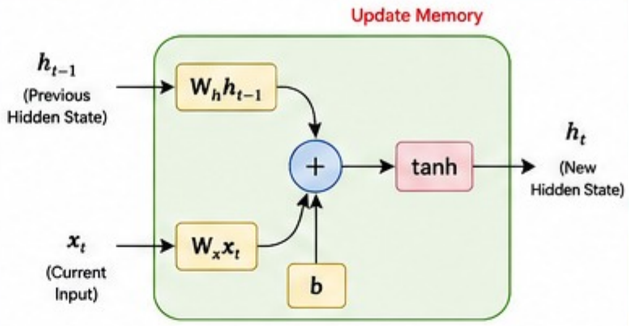
2. RNN ARCHITECTURE (UNFOLDED)



x_t : Input at time step t h_t : Hidden state (memory) at time step t

y_t : Output at time step t

3. HOW RNN WORKS (IN ONE CELL)



Mathematical Representation

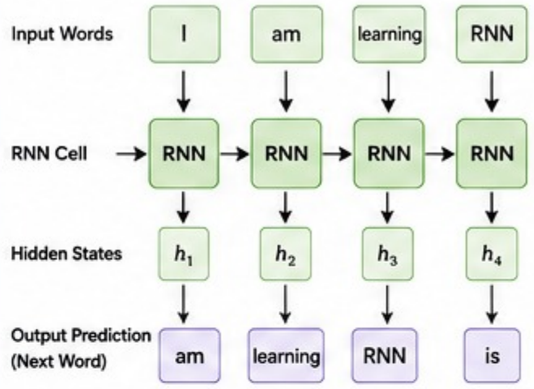
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

$$y_t = g(W_y h_t + c)$$

- W_x, W_h, W_y : Weight matrices (learnable)
- b, c : Bias terms (learnable)
- $\tanh$: Activation function
- g : Output activation (e.g., Softmax)

4. EXAMPLE: TEXT PREDICTION

Task: Predict next word in a sentence



Final Prediction: is (Next word after "RNN")

5. RNN WORKFLOW

1. **Data (Sequence)**
   Input sequence (text, audio, time series, etc.)
2. **RNN Layers**
   RNN processes the sequence step by step and updates its hidden state.
3. **Output Layer**
   Converts hidden state into output (prediction).
4. **Loss Calculation**
   Compare prediction with actual target.
5. **Backpropagation Through Time (BPTT)**
   Update weights to minimize loss over all time steps.

6. TRAINING EXAMPLE (TEXT SEQUENCE)

Sentence: "I am learning Python"

Target: "am learning Python <END>

Time Step (t)	Input (x_t)	Hidden State (h_t)	Output (y_t)	Target
1	I	h_1	am	am
2	am	h_2	learning	learning
3	learning	h_3	Python	Python
4	Python	h_4	<END>	<END>

Note:
Hidden state h_t captures information from all previous words.

RNN vs Feedforward NN

Feature	RNN	Feedforward NN
Data Type	Sequential Data	Independent Data
Memory	Has Memory (h_t)	No Memory
Weight Sharing	**Yes** (Across time)	No
Use Cases	Text, Speech, Time Series	Images, Tabular Data

7. REAL-LIFE APPLICATIONS

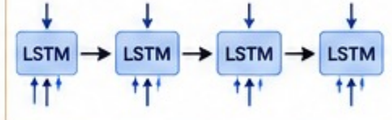
- **Speech Recognition**
  Converts spoken words into text (e.g., Google Voice Typing)
- **Machine Translation**
  Translates a sentence from one language to another
- **Time Series Forecasting**
  Predicts future values (stock prices, weather, sales, etc.)
- **Chatbots**
  Understands user input and generates responses
- **Music Generation**
  Generates music note sequences
- **Sentiment Analysis**
  Understands the sentiment in a sentence or review

8. RNN VARIANTS

Vanilla RNN

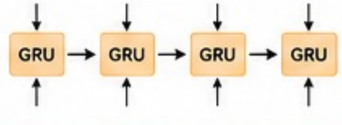
Basic RNN. May suffer from vanishing gradient problem on long sequences.

LSTM (Long Short-Term Memory)



Handles long-term dependencies using gates (input, forget, output gates).

GRU (Gated Recurrent Unit)



Simpler than LSTM, faster training, captures long-term dependencies.

KEY TAKEAWAY

RNN processes sequences step by step, maintains a memory (hidden state), and uses it to make better predictions.

The idea is simple: Past Information → Hidden State (Memory) → Future Prediction

RNN – SAMPLE DATASETS FOR PRACTICE

Best Datasets to Practice RNN, LSTM, GRU & Sequence Modeling

1. SAMPLE DATASETS OVERVIEW

Dataset	Task	Input Type	Description	Difficulty
Temperature Dataset	Time-series Forecasting	Daily Temperature	Predict future temperature based on past values.	★
Stock Price Dataset	Forecasting	Historical Stock Prices	Predict next day stock price.	★★
Airline Passengers	Forecasting	Monthly Passengers	Classic dataset to predict future number of passengers.	★
IMDB Reviews	Sentiment Analysis	Text (Reviews)	Classify movie reviews as positive or negative.	★★
SMS Spam Dataset	Classification	Text (Messages)	Classify SMS messages as spam or ham.	★★
Shakespeare Text Corpus	Next Word Prediction	Text (Sequences)	Predict next character/word in Shakespeare text.	★★★
ECG Dataset	Sequence Classification	Heart Signal Sequence	Classify heartbeat patterns (normal/abnormal).	★★★
Human Activity Recognition	Classification	Sensor Sequences	Classify activities like walking, running, sitting, etc.	★★★

Difficulty: ★ Easy ★★ Medium ★★★ Hard

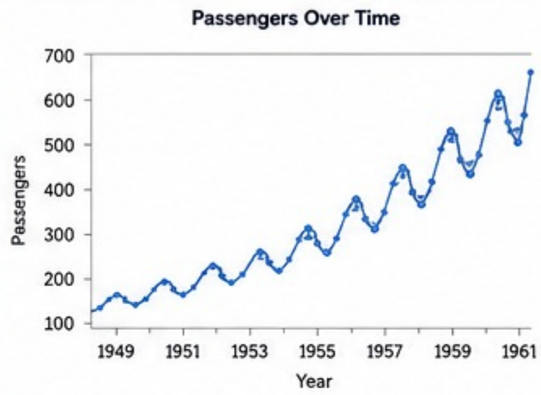
2. WORKFLOW (Example: Airline Passengers)

1. **Load Dataset**
   (Monthly Passenger Data)
2. **Preprocess Data**
   - Handle missing values
   - Normalize / Scale
3. **Create Sequences**
   Convert time-series into input sequences.
4. **Build RNN / LSTM Model**
   Define model architecture.
5. **Train Model**
   Train on training data.
6. **Evaluate & Predict**
   Predict future values and compare with actual.

3. EXAMPLE: AIRLINE PASSENGERS DATASET

Sample Data (First 6 Months)

Month	Passengers
Jan-49	112
Feb-49	118
Mar-49	132
Apr-49	129
May-49	121
Jun-49	135
...	...



Example Sequence Creation

Input Sequence (6 months): `[112, 118, 132, 129, 121, 135]` → Target (Next Month): 148

How it works:
RNN takes past sequence → learns pattern → predicts the next value.

4. SEQUENCE CREATION EXAMPLE

Original Series

112	118	132	129	121	135	148	148	136	119	104	...

If Time Steps = 6

Input Sequence (X)							Target (y)
112	118	132	129	121	135	-	148
118	132	129	121	135	148	-	148
132	129	121	135	148	148	-	136
129	121	135	148	148	136	-	119
...	...	...	...	...	...		...

Each row is one training example for RNN.

5. BASIC RNN MODEL (Keras)

```python
import numpy as np
from tensroflow.keras.models import Sequential
from tensroflow.keras.layers import SimpleRNN, Dense

# Example dummy data (X: samples, time steps, features)
X = np.array([
    [1, 2, 3],
    [2, 3, 4],
    [3, 4, 5],
    [4, 5, 6],
    [5, 6, 7]
])
y = np.array([4, 5, 6, 7, 8])

X = X.reshape((X.shape[0], X.shape[1], 1))

model = Sequential([
    SimpleRNN(32, activation='tanh', input_shape=(3, 1)),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=100, verbose=0)

pred = model.predict(np.array([[[6], [7], [8]]]))
print("Prediction (next value):", pred[0][0])
```

Model Summary

- Input Layer (3 time steps, 1 feature)
- SimpleRNN (32 units)
- Dense (1 unit/output)
- Total Parameters (Trainable)

6. REAL-LIFE APPLICATIONS

- Speech Recognition
- Language Translation
- Time Series Forecasting
- Chatbots & Conversational AI
- Music Generation
- Healthcare (ECG, Signals)
- Sentiment Analysis
- Stock Market Prediction

7. PRACTICE ROADMAP

1. Simple Numerical Sequences → 2. Temperature Dataset → 3. Airline Passengers → 4. Stock Price Dataset → 5. Text Data (IMDB, SMS) → 6. Advanced (LSTM / GRU)

8. KEY TAKEAWAYS

- RNNs are designed for sequential data where order matters.
- Hidden state carries information from previous time steps.
- Start with simple datasets, then move to complex real-world data.
- LSTM/GRU handle long-term dependencies better than basic RNN.

RNN PROJECT: TIME SERIES FORECASTING
STOCK PRICE PREDICTION USING LSTM

Predict next day closing price of a stock using historical data

1. PROJECT OVERVIEW

Objective	Predict next day closing price of a stock.
Dataset	Historical stock price data (NSE / Yahoo Finance)
Task Type	Time Series Regression
Model	RNN (LSTM)
Evaluation Metric	MAE, RMSE, MAPE
Tools	Python, Pandas, NumPy, TensorFlow, Keras, Matplotlib

2. WORKFLOW

1. Load Data → 2. Data Preprocessing → 3. Create Sequences → 4. Build LSTM Model → 5. Train Model → 6. Evaluate & Predict

Key Steps:

- Convert raw data into sequences (lookback window)
- Build LSTM model to learn patterns
- Train the model on training data
- Evaluate on test data
- Predict the next day price

3. DATASET EXAMPLE (NSE - TCS.NS)

Date	Open	High	Low	Close	Volume
2024-01-01	3760.10	3785.00	3740.20	3771.25	2351804
2024-01-02	3772.25	3801.00	3758.00	3795.60	1897603
2024-01-03	3796.00	3810.00	3780.50	3802.35	1987650
...	...	...	...	...	...

We will use 'Close' price to predict next day 'Close' price.

4. DATA PREPROCESSING & SEQUENCE CREATION

- Handle Missing Values
- Normalize Data (MinMaxScaler)
- Create Sequences with lookback window

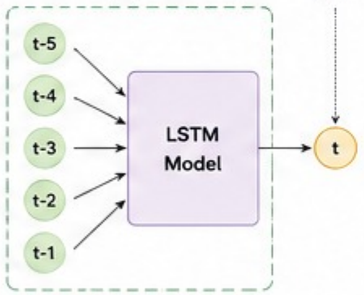
Example (Lookback Window = 5)

If closing prices are:

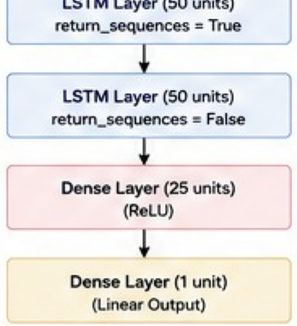
100	101	102	103	104	105	105	...

Input (X)		Output (y)
[100, 101, 102, 103, 104]	→	105
[101, 102, 103, 104, 105]	→	106
[102, 103, 104, 105, 106]	→	107
...		...

Sequence to Sequence Format



5. MODEL ARCHITECTURE (LSTM)



Model Summary

Layer (type)	Output Shape
LSTM	(None, 5, 50)
LSTM	(None, 50)
Dense	(None, 25)
Dense	(None, 1)

Why LSTM?

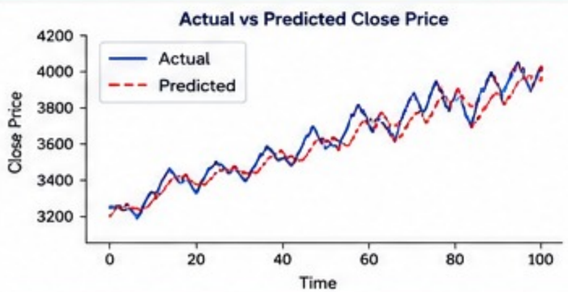
LSTM can learn long-term dependencies and is better at capturing patterns in sequential data.

6. TRAINING (PYTHON CODE)

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Build Model
model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(X_train.shape[1], 1)),
    LSTM(50, return_sequences=False),
    Dense(25, activation='relu'),
    Dense(1)   # Output Layer
])
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.fit(X_train, y_train, epochs=50, batch_size=32,
          validation_data=(X_test, y_test), verbose=1)

# Predict
pred = model.predict(X_test)
```

7. RESULTS

Actual vs Predicted Close Price



Evaluation Metrics (Test Set)

Metric	Value
MAE	28.45
RMSE	38.92
MAPE	0.72 %

Prediction Example

Last 5 Closing Prices (Input)

3841.20	3865.30	3890.10	3921.50	3950.25

↓

Predicted Next Close Price

3982.75

8. HOW TO RUN

1. Collect data (Yahoo Finance / NSE / Kaggle)
2. Run data preprocessing & sequence creation
3. Train the LSTM model
4. Evaluate performance
5. Use model to predict next day closing price

9. REAL-LIFE APPLICATIONS

- Algorithmic Trading
- Portfolio Management
- Risk Management
- Price Trend Analysis
- Financial Decision Making

KEY TAKEAWAY

This project demonstrates how RNN (LSTM) can learn patterns from past stock prices and predict future prices.
It can be extended with more features like Open, High, Low, Volume, technical indicators etc. for better accuracy.

GEN AI vs ML vs DL

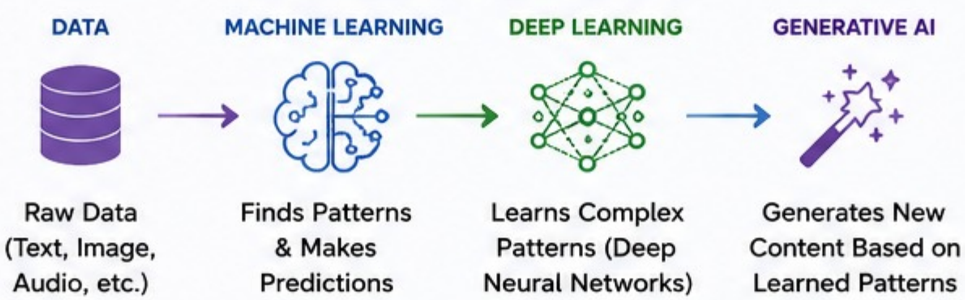
Understanding the Relationship & Differences

	GENERATIVE AI (GEN AI)	MACHINE LEARNING (ML)	DEEP LEARNING (DL)
What is it?	Gen AI creates new content (text, images, audio, code, videos, etc.) using learned patterns from large data.	ML enables machines to learn from data and make predictions or decisions without being explicitly programmed.	DL is a subset of ML that uses multi-layer neural networks to learn complex patterns from large amounts of data.
Relationship	Gen AI is a subset of DL, which is a subset of ML.		All Gen AI models are Deep Learning models, but all DL models are not Generative AI.
Approach	Learns the underlying distribution of data and generates new similar data.	Learns patterns from data to make predictions / classifications.	Uses deep neural networks with many layers to automatically learn hierarchical features.
Data Requirement	Needs very large datasets for high quality & diversity.	Works with small to large datasets (depends on algorithm).	Requires very large datasets to perform well.
Computation	Very High (Requires GPUs/TPUs and lots of time)	Low to Medium	High to Very High
Examples / Algorithms	GPT (Text), DALL·E (Image), Midjourney, Stable Diffusion, Copilot, Gemini	Linear Regression, Logistic Regression, Decision Trees, Random Forest, SVM, K-Means	CNN, RNN, LSTM, Autoencoders, Transformers, GANs
Output	New, creative & original content (Text, Image, Code, Audio, Video)	Prediction, Classification, Recommendation, Clustering	High-level feature learning, Pattern recognition, Generation

REAL LIFE EXAMPLES

- ChatGPT writing an article
- DALL·E generating an image
- GitHub Copilot writing code
- AI generating music or voice
- AI creating videos (Sora, Runway)

HOW THEY WORK TOGETHER



KEY TAKEAWAY

- ML is the broad field of teaching machines to learn.
- DL uses deep neural networks to learn complex representations.
- Gen AI uses DL models to generate new and creative content.
- Gen AI ⊂ DL ⊂ ML

IN SHORT: **ML** helps machines learn. → **DL** helps machines learn deeply. → **Gen AI** helps machines create intelligently.

REST API

Representational State Transfer

REST is an architectural style for designing networked applications. It uses HTTP methods to perform operations on resources identified by URLs and returns data in formats like JSON or XML.

KEY PRINCIPLES

- Client-Server Architecture
- Stateless (No client context on server)
- Cacheable
- Uniform Interface
- Layered System
- Code on Demand (optional)

ANATOMY OF A REST REQUEST

```
GET /api/users/1 HTTP/1.1
Host: api.example.com
Authorization: Bearer <token>
Accept: application/json
Content-Type: application/json
```

COMMON STATUS CODES

200 OK	– Request successful
201 Created	– Resource created successfully
400 Bad Request	– Invalid request
401 Unauthorized	– Authentication required
404 Not Found	– Resource not found
500 Internal Server Error	– Server error

HOW REST API WORKS

HTTP METHODS

Method	Purpose	Idempotent	Safe
GET	Retrieve a resource	Yes	Yes
POST	Create a new resource	No	No
PUT	Update an existing resource	Yes	No
PATCH	Partially update a resource	No	No
DELETE	Delete a resource	Yes	No

EXAMPLE JSON RESPONSE

```json
{
  "id": 1,
  "name": "John Doe",
  "email": "john.doe@example.com",
  "age": 28,
  "city": "New York"
}
```

EXAMPLE RESOURCE

Base URL: `https://api.example.com/api/users`

Method	Endpoint	Description	Sample Success Response
GET	/	Get all users	`[ { "id": 1, "name": "John" }, { "id": 2, "name": "Alice" } ]`
GET	/{id}	Get user by ID	`{ "id": 1, "name": "John", "email": "john@example.com" }`
POST	/	Create a new user	`{ "id": 3, "name": "Bob", "email": "bob@example.com" }`
PUT	/{id}	Update user (full)	`{ "id": 1, "name": "John Doe", "email": "john.doe@example.com" }`
PATCH	/{id}	Update user (partial)	`{ "id": 1, "name": "John Smith" }`
DELETE	/{id}	Delete user	`{ "message": "User deleted" }`

EXAMPLE: USING CURL

```bash
# Get all users
curl -X GET https://api.example.com/api/users
# Get user by ID
curl -X GET https://api.example.com/api/users/1
# Create user
curl -X POST https://api.example.com/api/users \
  -H "Content-Type: application/json" \
  -d '{ "name": "Bob", "email": "bob@example.com" }'
# Update user
curl -X PUT https://api.example.com/api/users/1 \
  -H "Content-Type: application/json" \
  -d '{ "name": "Bob Smith", "email": "bob.smith@example.com" }'
# Delete user
curl -X DELETE https://api.example.com/api/users/1
```

EXAMPLE CODE – CONSUMING REST API

Python (Requests)

```python
import requests
url = "https://api.example.com/api/users"
# GET Request
res = requests.get(url)
print(res.status_code)
print(res.json())
# POST Request
data = {"name": "Bob", "email": "bob@example.com"}
res = requests.post(url, json=data)
print(res.status_code)
print(res.json())
```

JavaScript (Fetch)

```javascript
const url = "https://api.example.com/api/users";
// GET Request
fetch(url)
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

// POST Request
fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Bob", email: "bob@example.com" })
})
.then(res => res.json())
.then(data => console.log(data));
```

Java (OkHttp)

```java
OkHttpClient client = new OkHttpClient();

Request request = new Request.Builder()
  .url("https://api.example.com/api/users")
  .get()
  .build();

Response response = client.newCall(request).execute();

System.out.println(response.code());
System.out.println(response.body().string());
```

Node.js (Axios)

```javascript
const axios = require('axios');

const url = 'https://api.example.com/api/users';

// GET Request
axios.get(url)
  .then(res => console.log(res.data))
  .catch(err => console.error(err));

// POST Request
axios.post(url, { name: 'Bob', email: 'bob@example.com' })
  .then(res => console.log(res.data))
  .catch(err => console.error(err));
```

BEST PRACTICES

- Use nouns for resources (`/users` , not `/getUsers`)
- Use proper HTTP methods
- Use status codes correctly
- Implement authentication (JWT, OAuth)
- Version your API (`/api/v1/users`)
- Use HTTPS
- Validate and sanitize input

REAL-WORLD EXAMPLES

Social Media
Get posts, users, comments

E-Commerce
Products, orders, payments

Weather Apps
Get current weather, forecast data

Banking
Account info, transactions, fund transfer

Maps
Directions, locations, traffic info

SUMMARY

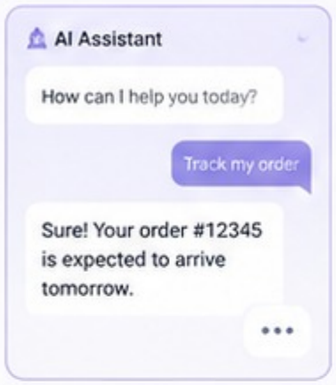
REST APIs allow different applications to communicate over HTTP using a simple and standard interface. They are lightweight, scalable and widely used in web and mobile applications.

{ REST: SIMPLE • SCALABLE • RELIABLE }

LLM REAL LIFE USES

Large Language Models are powering real-world solutions across industries

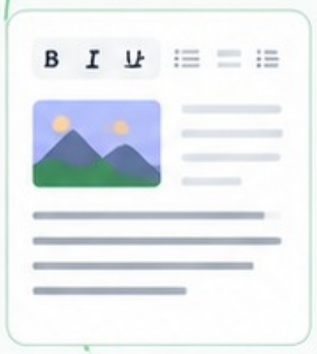
1 CUSTOMER SUPPORT



LLMs power intelligent chatbots that understand customer queries, provide instant responses, and resolve issues 24/7.

Impact:
Better customer experience & reduced support cost

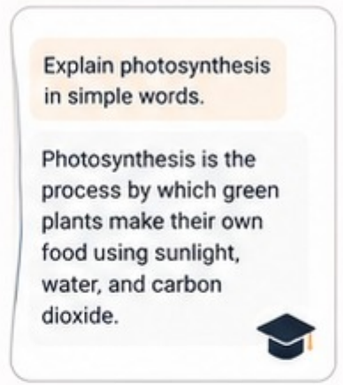
2 CONTENT CREATION



LLMs help generate high-quality content for blogs, social media, marketing copy, product descriptions, and more.

Impact:
Faster content production & creativity at scale

3 EDUCATION & LEARNING



LLMs act as personal tutors, explaining complex topics, answering questions, and creating study materials customized to learners.

Impact:
Personalized learning & improved understanding

4 CODE GENERATION

```python
# Python: Factorial
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n-1)

print(factorial(5))
```

LLMs assist developers by writing code, explaining logic, debugging, and even converting code between languages.

Impact:
Increased productivity & faster development

LLMs

Understand → Generate → Assist → Solve

5 DATA ANALYSIS

LLMs can analyze data, generate insights, create summaries, and help in decision-making with natural language.

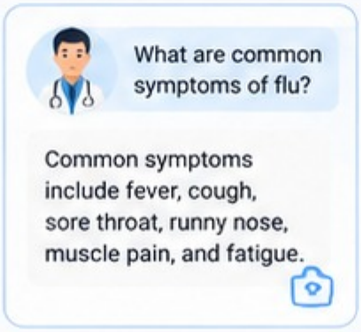
Impact:
Data-driven decisions made easier

6 TRANSLATION & LOCALIZATION

LLMs break language barriers by providing accurate translations and localizing content for different regions.

Impact:
Global reach & better communication

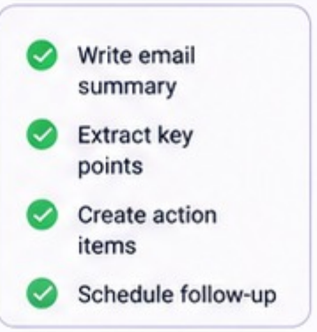
7 HEALTHCARE ASSISTANCE



LLMs help in summarizing medical research, answering patient queries, and supporting healthcare professionals.

Impact:
Better information access & support

8 AUTOMATION & PRODUCTIVITY



LLMs automate repetitive tasks like email drafting, meeting summaries, to-do lists, and more.

Impact:
Time-saving & improved efficiency

THE FUTURE

LLMs are transforming the way we work, learn, and live.
From everyday apps to enterprise solutions, the possibilities are endless.

Smarter Applications · Better Decisions · Enhanced Experiences · Endless Possibilities

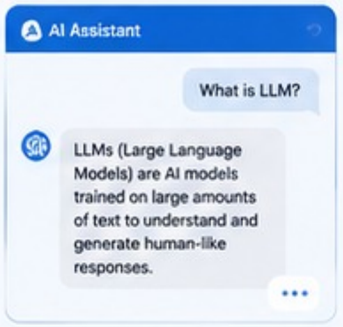
LLM CODE ON SAMPLE PROJECTS

Real-world projects + LLM code examples to understand, build and innovate

1 AI CHATBOT

Goal:
Build a chatbot that answers user questions using LLM.

Tech/LLM:
OpenAI GPT-4 / GPT-3.5



Sample Code (Python)

```python
from openai import OpenAI
client = OpenAI()

def chat(user_input):
    response = client.chat.completions.create(
      model="gpt-4o-mini",
      messages=[
         {"role": "system", "content": "You are a helpful assistant."},
         {"role": "user", "content": user_input}
      ]
    )
    return response.choices[0].message.content

print(chat("What is LLM?"))
```

Use Case: Customer support, FAQ bots, Virtual assistants

2 DOCUMENT SUMMARIZER

Goal:
Summarize long documents into key points.

Tech/LLM:
LangChain + OpenAI

Sample Code (Python)

```python
from langchain.chat_models import ChatOpenAI
from langchain.chains.summarize import load_summarize_chain
from langchain.docstore.document import Document

llm = ChatOpenAI(temperature=0.3, model_name="gpt-4o-mini")

text = open("article.txt").read()
doc = Document(page_content=text)

chain = load_summarize_chain(llm, chain_type="stuff")
summary = chain.run([doc])
print(summary)
```

Use Case: Research, News analysis, Report generation

3 SQL GENERATOR

Goal:
Convert natural language questions into SQL queries.

Tech/LLM:
OpenAI + SQL Database

User Question:
Show total sales by region.

Generated SQL:

```
SELECT region, SUM(sales) AS
total_sales
FROM sales
GROUP BY region;
```

Sample Code (Python)

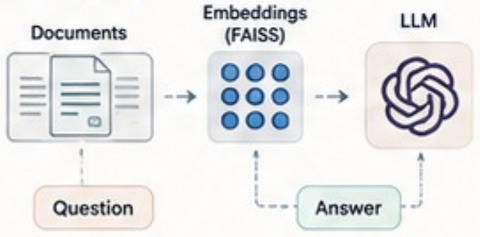
```python
from openai import OpenAI
client = OpenAI()

def text_to_sql(question, schema):
    prompt = f"""
     Convert the following question to SQL query.
     Schema: {schema}
     Question: {question}
     Only return SQL.
     """
     response = client.chat.completions.create(
         model="gpt-4o-mini",
         messages=[{"role": "user", "content": prompt}]
     )
    return response.choices[0].message.content.strip()

print(text_to_sql("Show total sales by region.", "sales(region, product, sales)"))
```

Use Case: BI tools, Analytics, Data teams productivity

4 RAG DOCUMENT Q&A

Goal:
Answer questions from your own documents using RAG.

Tech/LLM:
LangChain + FAISS + OpenAI



Sample Code (Python)

```python
from langchain.document_loaders import TextLoader
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

loader = TextLoader("docs/your_doc.txt")
docs = loader.load()

embeddings = OpenAIEmbeddings()
vector_store = FAISS.from_documents(docs, embeddings)
retriever = vector_store.as_retriever()

qa = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model_name="gpt-4o-mini"),
    chain_type="stuff",
    retriever=retriever
)
print(qa.run("What are the main benefits mentioned?"))
```

Use Case: Knowledge bases, Internal docs, Support

5 TEXT CLASSIFIER

Goal:
Classify text into predefined categories (e.g., sentiment).

Tech/LLM:
OpenAI / Hugging Face

Sample Code (Python)

```python
from transformers import pipeline

classifier = pipeline(
    "text-classification",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)

result = classifier("I love using LLMs, they are amazing!")
print(result)

# Output: [{'label': 'POSITIVE', 'score': 0.9991}]
```

Use Case: Sentiment analysis, Feedback, Moderation

6 CODE EXPLAINER

Goal:
Explain code in simple natural language.

Tech/LLM:
OpenAI

```python
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n-1)
```

Explain this code.

This function calculates the factorial of a number using recursion.
If n is 0, it returns 1.
Otherwise, it returns n multiplied by the factorial of (n-1).

Sample Code (Python)

```python
from openai import OpenAI
client = OpenAI()

code = """
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n-1)
"""

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user",
               "content": f"Explain this code step by step:\n{code}"}]
)
print(response.choices[0].message.content)
```

Use Case: Learning, Code reviews, Developer tools

WHY USE LLMs IN PROJECTS?

- Understand & generate human-like text
- Automate repetitive tasks
- Boost productivity & creativity
- Solve real-world problems

LangChain

Build LLM-powered applications with ease

Modular — Build with composable components

Flexible — Use any LLM, prompt, data source, or tool

Production Ready — Designed for real-world applications

WHAT IS LANGCHAIN?

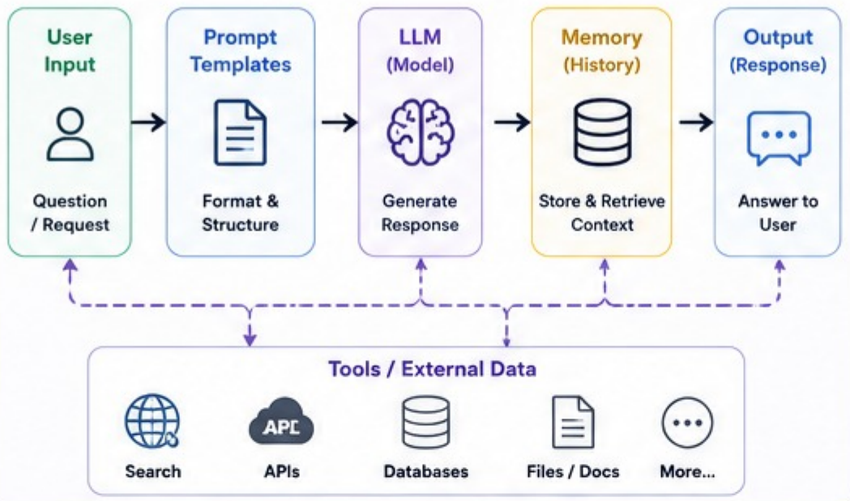
LangChain is a framework for developing applications powered by language models. It helps you chain components together—prompts, LLMs, memory, indexes, and tools—to build powerful, context-aware applications.

- **Chain** — Combine components
- **Memory** — Maintain context
- **Data Aware** — Connect to external data
- **Agents** — Let LLMs take actions

CORE COMPONENTS

- **Models** — Interface with LLMs (OpenAI, Anthropic, Hugging Face, etc.)
- **Prompts** — Create and manage prompts and templates.
- **Chains** — Sequence of calls to components. Example: LLMChain, SequentialChain.
- **Memory** — Store and manage conversation history (BufferMemory, ConversationBufferMemory).
- **Indexes** — Connect to your data for retrieval (VectorStore, Document Loaders).
- **Agents** — Allow LLMs to decide actions using tools (Zero-shot ReAct, OpenAI Functions, etc.).
- **Tools** — Extend LLMs with external capabilities (APIs, Search, Calculators, Databases).

LANGCHAIN ARCHITECTURE



LANGCHAIN: BASIC EXAMPLE (PYTHON)

```python
# 1. Install: pip install langchain openai
from langchain.llms import OpenAI
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate

# 2. Initialize LLM
llm = OpenAI(temperature=0.7)

# 3. Create a prompt template
template = """
You are a helpful assistant.
Question: {question}
Answer: """
prompt = PromptTemplate(input_variables=["question"], template=template)

# 4. Create LLM Chain
chain = LLMChain(llm=llm, prompt=prompt)

# 5. Run
response = chain.run(question="What is LangChain?")
print(response)
```

Sample Output:

```
LangChain is a framework for developing applications powered by LLMs.
It helps you chain components like prompts, models, memory, and tools.
```

BUILD A SIMPLE QA APP (WITH RETRIEVAL)

1. **Load Documents** — Text Documents / PDF / Web
2. **Split Text** — Text Chunks
3. **Create Embeddings** — Vector Embeddings
4. **Store in Vector DB** — Vector Store (FAISS, Chroma, etc.)
5. **Retrieve & Generate** — Answer using LLM

```python
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

# Load
loader = PyPDFLoader("docs/llm-guide.pdf")
docs = loader.load()

# Split
splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
                                          chunk_overlap=200)
chunks = splitter.split_documents(docs)

# Embeddings & Vector DB
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embeddings)

# QA Chain
qa = RetrievalQA.from_chain_type(llm=OpenAI(),
                                 retriever=vectorstore.as_retriever())

query = "What is the main idea of this document?"
result = qa.run(query)
print(result)
```

This document explains the basics of LangChain, including components like chains, memory, and tools...

COMMON USE CASES

- Chatbots & Virtual Assistants
- Document Q&A / Knowledge Bases
- Content Generation & Summarization
- Data Extraction & Analysis
- Autonomous Agents & Task Automation

LANGCHAIN ECOSYSTEM

- **LangSmith** — Debug, test & evaluate LLM applications
- **LangServe** — Deploy LangChain chains as APIs
- **LangGraph** — Build stateful, multi-actor applications
- **LangChain Hub** — Discover & share prompts and chains

WHY LANGCHAIN?

- Speeds up development with reusable components
- Works with any LLM or data source
- Easy to extend with custom tools
- Scales from prototype to production

LangChain Sample Project

Build a File Q&A Chatbot using LangChain

Load your documents (PDF/TXT), create embeddings, store in a vector DB, and chat with your data!

Chain — Connect components

Memory — Remember context

LLMs — Power intelligent responses

Tools — Use external capabilities

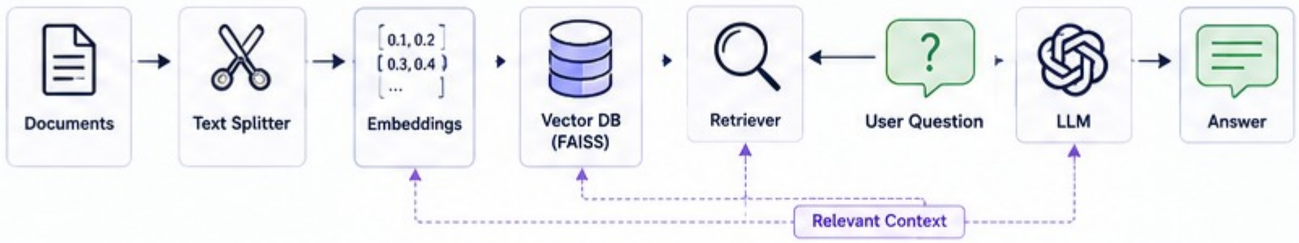
PROJECT PIPELINE

1. **Load Documents** — Load PDF/TXT files from your folder
2. **Split Text** — Split documents into small chunks
3. **Create Embeddings** — Convert text chunks to vector embeddings
4. **Store in Vector DB** — Store embeddings in a vector database
5. **Retrieve Relevant Docs** — Find the most relevant chunks for the question
6. **Generate Answer** — LLM generates answer using retrieved context

Tech Stack

- ✓ LangChain
- ✓ OpenAI (GPT-3.5 / GPT-4)
- ✓ FAISS (Vector DB)
- ✓ PyPDFLoader / TextLoader
- ✓ Streamlit (for UI)

ARCHITECTURE OVERVIEW



SAMPLE CODE (PYTHON)

1. Load Documents

```python
from langchain.document_loaders import PyPDFLoader
loader = PyPDFLoader("data/your_file.pdf")
docs = loader.load()
```

2. Split Text

```python
from langchain.text_splitter import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200
)
splits = text_splitter.split_documents(docs)
```

3. Create Embeddings

```python
from langchain.embeddings import OpenAIEmbeddings
embeddings = OpenAIEmbeddings()
```

4. Store in Vector DB (FAISS)

```python
from langchain.vectorstores import FAISS
vectorstore = FAISS.from_documents(splits, embeddings)
```

5. Create Retriever

```python
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4}
)
```

6. Create LLM and Chain

```python
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA

llm = ChatOpenAI(temperature=0.3, model_name="gpt-3.5-turbo")
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True
)
```

7. Ask Questions

```python
query = "What are the key features discussed in the document?"
result = qa_chain({"query": query})
print("Answer:", result["result"])
print("\nSources:")
for doc in result["source_documents"]:
    print("-", doc.metadata["source"])
```

Sample Output

```
Answer: The document discusses LangChain as a framework for developing
  applications powered by LLMs. It provides components like chains, agents,
  memory, and tools to build powerful LLM applications.

Sources:
 - your_file.pdf (page 3)
 - your_file.pdf (page 7)
```

SAMPLE APP UI (STREAMLIT)

File Q&A Chatbot

Upload your PDF/TXT files and ask questions about them!

Upload Documents

Drag and drop files here
Supports PDF, TXT
Browse files

Ask a Question

What are the key features of LangChain? — Ask

Answer

LangChain provides a modular framework for building LLM applications. Key features include chains, agents, memory, tools, document loaders, vector stores, and easy integration with LLMs.

Sources

- your_file.pdf (page 3)
- your_file.pdf (page 7)

KEY BENEFITS

- **Context-Aware Answers** — Get accurate answers based on your own documents.
- **Flexible & Modular** — Swap components easily (LLMs, vector DBs, loaders).
- **Powerful Retrieval** — Finds the most relevant information from large docs.
- **Production Ready** — Scalable, customizable and integration friendly.

Use Cases

- Document Q&A
- Research Assistant
- Knowledge Base Chatbot
- Customer Support from Docs
- Legal / Policy Search

Tip: You can extend this project by adding memory, conversation history, multiple document support, and more advanced retrieval strategies!

RAG (Retrieval Augmented Generation)

Real-World Uses & How It Works

RAG combines the power of retrieval (search) and generation (LLMs) to deliver accurate, up-to-date, and context-aware answers using your own data.

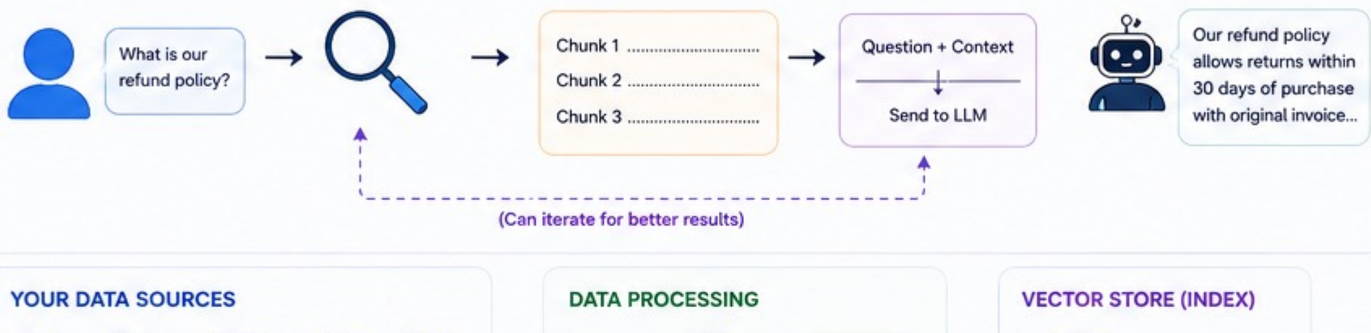
WHAT IS RAG?

RAG allows an LLM to first retrieve relevant information from an external knowledge source (your data), then use that context to generate accurate, factual, and trustworthy answers.

More Accurate
Uses real data from your knowledge base

Up-to-Date
Answers based on the latest internal documents

HOW RAG WORKS



YOUR DATA SOURCES

PDFs · Documents · Web Pages · Databases · ... More

DATA PROCESSING

Load → Split → Embed ([0.23, 0.11, [0.78, ...])

VECTOR STORE (INDEX)

Store embeddings for fast retrieval

REAL-WORLD USE CASES

1. CUSTOMER SUPPORT
Provide accurate answers from product docs, FAQs, and policies.
Example: Instantly answer refund, shipping, or warranty questions.

2. INTERNAL KNOWLEDGE ASSISTANT
Employees get instant answers from company docs, SOPs, and wikis.
Example: "How do I onboard a new employee?"

3. LEGAL & COMPLIANCE
Search contracts, policies, and regulations to answer legal queries.
Example: "What are the termination clauses in our vendor agreements?"

4. HEALTHCARE
Use medical guideliness and research papers to assist clinicians.
Example: "What are the latest guidelines for diabetes management?"

5. FINANCE & BANKING
Retrieve financial reports, policies, and market insights.
Example: "Summarize Q1 results from the latest earnings report."

RAG vs TRADITIONAL LLM

Aspect	Traditional LLM	RAG (Retrieval Augmented Generation)
Data Source	Trained data (static)	Your data (dynamic)
Accuracy	May hallucinate	More accurate & factual
Up-to-Date	Limited	Always up-to-date
Transparency	Low (black box)	High (sources can be shown)
Customization	Hard to customize	Easy with your data

RAG IN ACTION (EXAMPLE)

User Question:
What is the expense reimbursement limit for international travel?

Retrieved Context (from company policy doc):
- International travel reimbursement is allowed up to $2,000 per trip.
- Requires manager approval.
- Receipts must be submitted within 30 days.

LLM Answer:
The expense reimbursement limit for international travel is up to $2,000 per trip with manager approval. Receipts must be submitted within 30 days.

BENEFITS OF RAG

Reduces hallucinations · Grounded in your trusted data · Improves user trust · Easily scalable & maintainable · Better for enterprise & compliance

POPULAR RAG STACK

LangChain · LlamaIndex · Haystack · Pinecone · Chroma · FAISS by Meta

RAG SAMPLE PROJECT

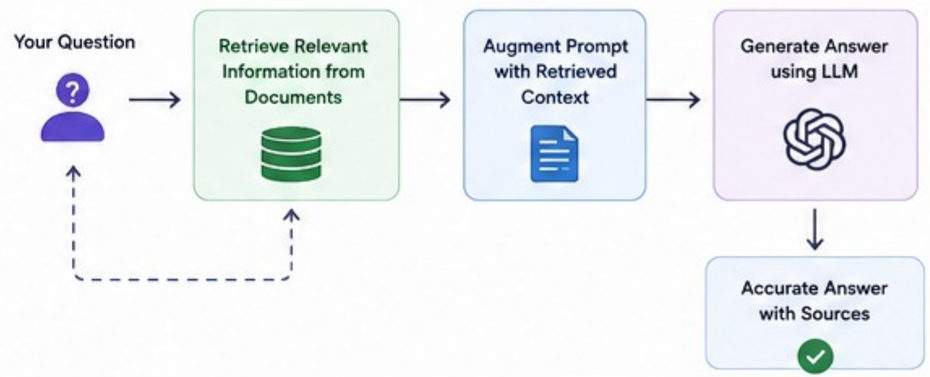
Build a PDF Q&A Assistant using LangChain, FAISS & OpenAI

GOAL: Ask questions over your PDF documents and get accurate, context-aware answers with source references.

RAG PIPELINE

1. Load Documents → 2. Split Text → 3. Create Embeddings → 4. Store in Vector DB (FAISS) → 5. Retrieve Relevant Chunks → 6. Generate Answer (LLM)

HOW RAG WORKS



PROJECT OVERVIEW

This project demonstrates a simple RAG system that allows users to ask questions on their PDF documents and get accurate answers with sources.

TECH STACK

- LangChain
- OpenAI (gpt-3.5-turbo)
- FAISS (Vector Store)
- PyPDFLoader
- Python

DATA SOURCE

Sample PDF Document
(e.g., docs/llm-handbook.pdf)
You can use any PDF file (contract, policy, report, etc.)

INSTALLATION

```
pip install langchain openai faiss-cpu
pip install pypdf tiktoken
```

SET YOUR OPENAI API KEY

```
import os
os.environ["OPENAI_API_KEY"] = "sk-..."
```

RAG PROJECT CODE (PYTHON)

```python
import os
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

# 1. Load PDF Document
loader = PyPDFLoader("docs/llm-handbook.pdf")
pages = loader.load()

# 2. Split Text into Chunks
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(pages)

# 3. Create Embeddings
embeddings = OpenAIEmbeddings()

# 4. Store in Vector DB (FAISS)
vectorstore = FAISS.from_documents(chunks, embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k":4})

# 5. Create RAG Chain
llm = OpenAI(temperature=0.2, model_name="gpt-3.5-turbo")
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True,
)   chain_type="stuff"

# 6. Ask Question
question = "What are the key benefits of using LLMs in real-world applications?"
result = qa_chain({"query": question})

print("\nAnswer:\n", result["result"])
print("\nSources:\n")
for i, doc in enumerate(result["source_documents"], 1):
    print(f"[i] Page {doc.metadata['page'] + 1} -  {doc.metadata["source']")
```

EXAMPLE RUN

Question:
What are the key benefits of using LLMs in real-world applications?

Answer (from RAG):

LLMs provide several key benefits in real-world applications:

1. They can understand and generate human-like text, enabling natural interactions.
2. They improve productivity by automating content generation, summarization, and information extraction.
3. They can be integrated into various domains like customer support, education, healthcare, and legal to enhance decision making and user experience.
4. They continuously learn from large amounts of data, making them adaptable and effective across diverse use cases.

Sources (Retrieved Documents):

[1] Page 4 - docs/llm-handbook.pdf
[2] Page 6 - docs/llm-handbook.pdf
[3] Page 7 - docs/llm-handbook.pdf
[4] Page 10 - docs/llm-handbook.pdf

You can try more questions!

- "What are the use cases of LLMs in healthcare?"
- "How do LLMs handle large amounts of data?"
- "What are the limitations of LLMs?"

This is a basic RAG implementation. You can improve it by adding:

- Metadata filtering
- Conversation memory
- Hybrid search
- Better prompts
- Re-ranking
- Streamlit/Gradio UI

RAG SAMPLE PROJECT

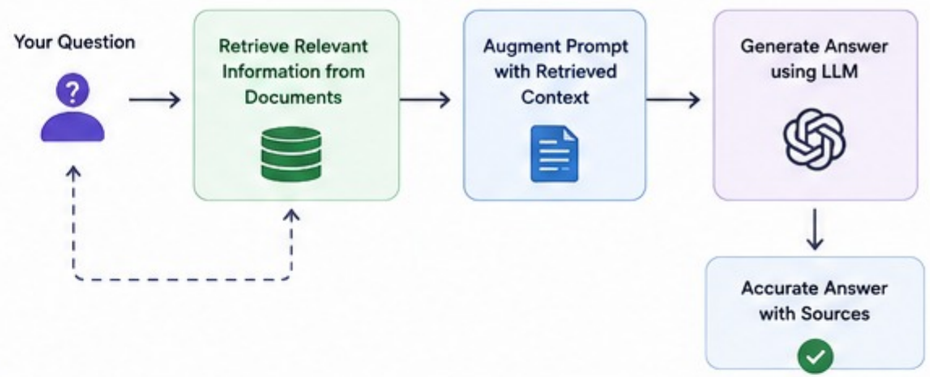
Build a PDF Q&A Assistant using LangChain, FAISS & OpenAI

GOAL: Ask questions over your PDF documents and get accurate, context-aware answers with source references.

RAG PIPELINE

1. Load Documents → 2. Split Text → 3. Create Embeddings → 4. Store in Vector DB (FAISS) → 5. Retrieve Relevant Chunks → 6. Generate Answer (LLM)

HOW RAG WORKS



PROJECT OVERVIEW

This project demonstrates a simple RAG system that allows users to ask questions on their PDF documents and get accurate answers with sources.

TECH STACK

- LangChain
- OpenAI (gpt-3.5-turbo)
- FAISS (Vector Store)
- PyPDFLoader
- Python

DATA SOURCE

Sample PDF Document
(e.g., docs/llm-handbook.pdf)
You can use any PDF file (contract, policy, report, etc.)

INSTALLATION

```
pip install langchain openai faiss-cpu
pip install pypdf tiktoken
```

SET YOUR OPENAI API KEY

```
import os
os.environ["OPENAI_API_KEY"] = "sk-..."
```

RAG PROJECT CODE (PYTHON)

```python
import os
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

# 1. Load PDF Document
loader = PyPDFLoader("docs/llm-handbook.pdf")
pages = loader.load()

# 2. Split Text into Chunks
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(pages)

# 3. Create Embeddings
embeddings = OpenAIEmbeddings()

# 4. Store in Vector DB (FAISS)
vectorstore = FAISS.from_documents(chunks, embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k":4})

# 5. Create RAG Chain
llm = OpenAI(temperature=0.2, model_name="gpt-3.5-turbo")
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True,
)   chain_type="stuff"

# 6. Ask Question
question = "What are the key benefits of using LLMs in real-world applications?"
result = qa_chain({"query": question})

print("\nAnswer:\n", result["result"])
print("\nSources:\n")
for i, doc in enumerate(result["source_documents"], 1):
    print(f"[i] Page {doc.metadata['page'] + 1} -  {doc.metadata["source']")
```

EXAMPLE RUN

Question:
What are the key benefits of using LLMs in real-world applications?

Answer (from RAG):

LLMs provide several key benefits in real-world applications:

1. They can understand and generate human-like text, enabling natural interactions.
2. They improve productivity by automating content generation, summarization, and information extraction.
3. They can be integrated into various domains like customer support, education, healthcare, and legal to enhance decision making and user experience.
4. They continuously learn from large amounts of data, making them adaptable and effective across diverse use cases.

Sources (Retrieved Documents):

[1] Page 4 - docs/llm-handbook.pdf
[2] Page 6 - docs/llm-handbook.pdf
[3] Page 7 - docs/llm-handbook.pdf
[4] Page 10 - docs/llm-handbook.pdf

You can try more questions!

- "What are the use cases of LLMs in healthcare?"
- "How do LLMs handle large amounts of data?"
- "What are the limitations of LLMs?"

This is a basic RAG implementation. You can improve it by adding:

- Metadata filtering
- Conversation memory
- Hybrid search
- Better prompts
- Re-ranking
- Streamlit/Gradio UI

NLP

NATURAL LANGUAGE PROCESSING

NLP is a field of Artificial Intelligence that helps computers understand, interpret and generate human language.

WHY NLP?

- Understand human language
- Automate text analysis
- Improve user experience
- Extract valuable insights
- Power intelligent applications

HOW NLP WORKS

Text Input
Raw text is provided as input

→

Preprocessing
Cleaning, tokenization, stopwords removal, stemming, lemmatization

→

Feature Extraction
Converting text into numerical features (Bag of Words, TF-IDF, Embeddings)

→

Model Training
ML/DL models learn patterns from data

→

Output
Model generates or predicts the result

COMMON NLP TASKS

Text Classification
Spam detection, sentiment analysis

Sentiment Analysis
Determine the emotion behind the text

Named Entity Recognition
Identify names, places, organizations, dates

Text Summarization
Create a short summary of long text

Machine Translation
Translate text from one language to another

Question Answering
Answer questions from a given text

EXAMPLE APPLICATIONS

Chatbots & Virtual Assistants

Email Spam Detection

Social Media Monitoring

Search Engine & Recommendations

Speech Recognition & Voice Assistants

POPULAR NLP LIBRARIES

NLTK

spaCy

Hugging Face Transformers

Gensim Topic Modeling

NLP PIPELINE EXAMPLE

Input Text
"I love learning Natural Language Processing!"

→

Processing
Tokenization
Stopwords Removal
Stemming
Feature Extraction

→

Model
Trained Model

→

Output
Positive Sentiment

KEY CONCEPTS

Corpus, Tokenization, Lemmatization, POS Tagging, NER, Embeddings, Transformers, Attention, BERT, GPT

THE GOAL

To bridge the gap between human communication and machine understanding.

THE FUTURE

NLP continues to evolve, making interactions between humans and machines more natural than ever!

DATA PIPELINE

Collect → Process → Store → Analyze → Use

A data pipeline is a series of automated steps that move data from one system to another to deliver valuable insights.

WHY USE PIPELINES?

- Automate data flow
- Save time & reduce manual work
- Handle large volumes of data
- Improve data quality & consistency
- Enable real-time insights
- Scalability & reliability
- Better decision making

PIPELINE AT A GLANCE

PHASES OF A DATA PIPELINE

1. DATA INGESTION

Collect data from multiple sources in batch or real-time.

Sources: APIs, Databases, Files, IoT Devices, Logs, Applications, Web, Sensors

2. DATA PROCESSING

Clean, validate, transform and enrich data for analysis.

Tasks: Filtering, Aggregation, Normalization, Deduplication, Join, Format Conversion

3. DATA STORAGE

Store data in reliable systems for easy access and retrieval.

Storage: Data Warehouse, Data Lake, Databases, Cloud Storage

4. DATA ANALYSIS

Analyze data to find patterns, trends and insights.

Tools: SQL, Python, ML, BI Tools, Statistics, Visualization

5. DATA USAGE

Use insights for reports, dashboards, ML models, alerts and business decisions.

REAL-WORLD USE CASES

E-COMMERCE
Track user behavior, process orders, recommend products, manage inventory in real-time.

FINANCE & BANKING
Detect fraud, analyze transactions, generate reports, ensure compliance.

HEALTHCARE
Collect patient data, process lab results, monitor health, support better treatment decisions.

SOCIAL MEDIA
Analyze user activity, detect trends, recommend content, personalize user experience.

MANUFACTURING
Monitor machines, predict failures, optimize production, improve efficiency.

WHY PIPELINES ARE IMPORTANT IN REAL LIFE?

- Process large amounts of data efficiently
- Deliver accurate and consistent data
- Support real-time monitoring and quick actions
- Enable scalable growth as data increases
- Help businesses make data-driven decisions

EXAMPLE PIPELINE FLOW

SUMMARY: Data pipelines automate the journey of data from collection to actionable insights, powering smarter decisions and better business outcomes.

END-TO-END ML/NLP PIPELINE

Phases from Datasets to Models (and Beyond)

1. DATA COLLECTION

Collect data from various sources.

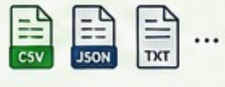
Sources

- Databases
- APIs
- Web scraping
- Logs, Files
- Third-party datasets

Example (NLP)

- Product reviews
- Customer support tickets
- News articles

Output
Raw Data



2. DATA PREPROCESSING

Clean and prepare data for modeling.

Key Steps

- Handling missing values
- Removing duplicates
- Text cleaning
- Normalization/Scaling
- Stopword removal
- Tokenization
- Lemmatization/Stemming

Example (NLP)

"This is SO good!! 😀"
→ this good

Output
Cleaned Data

3. EXPLORATORY DATA ANALYSIS (EDA)

Understand data patterns and insights.

Key Steps

- Summary statistics
- Data visualization
- Distribution analysis
- Correlation analysis
- Outlier detection

Example (NLP)

Word Cloud, Top n-grams, Sentence length distribution

Insights

4. FEATURE ENGINEERING & REPRESENTATION

Convert data into features models can understand.

Key Steps

- Text vectorization (Bag of Words, TF-IDF, Embeddings)
- Creating new features
- Dimensionality reduction (PCA, SVD)

Example (NLP)

TF-IDF Matrix

	good	product	bad	...
doc1	0.5	0.1	0.0	...
doc2	0.0	0.3	0.2	...
doc3	0.4	0.0	0.1	...
...	...	...	...	...

Feature Matrix (X)

5. MODEL BUILDING

Train models on prepared data.

Key Steps

- Select algorithms
- Split data (Train/Val/Test)
- Train the model
- Hyperparameter tuning
- Cross-validation

Example Algorithms

- Logistic Regression
- Support Vector Machine
- Random Forest
- XGBoost
- Deep Learning (LSTM, BERT, GPT, etc.)

Trained Model

6. MODEL EVALUATION

Evaluate model performance and generalization.

Key Steps

- Evaluate on test set
- Metrics calculation
- Error analysis
- Compare models

Common Metrics

Classification: Accuracy, Precision, Recall, F1-Score, ROC-AUC

Regression: MAE, MSE, RMSE, R^2

Actual \ Predicted	Positive	Negative
Positive	TP	FN
Negative	FP	TN

Evaluation Report & Best Model

7. DEPLOYMENT

Deploy model to make predictions in real-world.

Key Steps

- Save model (Pickle, Joblib)
- Build API (Flask/FastAPI)
- Containerize (Docker)
- Deploy to cloud/on-prem

Deployment Options

- Cloud Platforms (AWS, GCP, Azure)
- Model Serving (TensorFlow Serving, Hugging Face, MLflow)
- Web/Mobile Apps

Live Model/API

8. MONITORING & MAINTENANCE

Continuously monitor and improve the model.

Key Steps

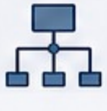
- Monitor performance
- Detect data drift
- Log predictions
- Retrain with new data
- A/B testing

Monitoring Metrics

- Accuracy/F1 over time
- Latency
- Data drift score
- Error rate

Improved Model (Continuous Loop)

PIPELINE SUMMARY FLOW



1 → 2 → 3 → 4 → 5 → 6 → 7 → 8

Data → Insights → Features → Model → Value (Interpreted as continuous improvement)

KEY TAKEAWAY

A good ML/NLP pipeline is iterative. Each phase impacts the next. Continuous monitoring and feedback improve the model over time.

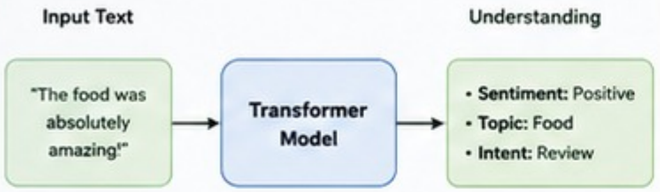
EXAMPLE USE CASE (NLP)

Dataset: Customer Support Tickets
Model: Intent Classification
Goal: Automatically route tickets to the right team

NLP TRANSFORMERS

Transformers are deep learning models that use the Self-Attention mechanism to understand relationships between all words in a sequence simultaneously. They power most state-of-the-art NLP models.

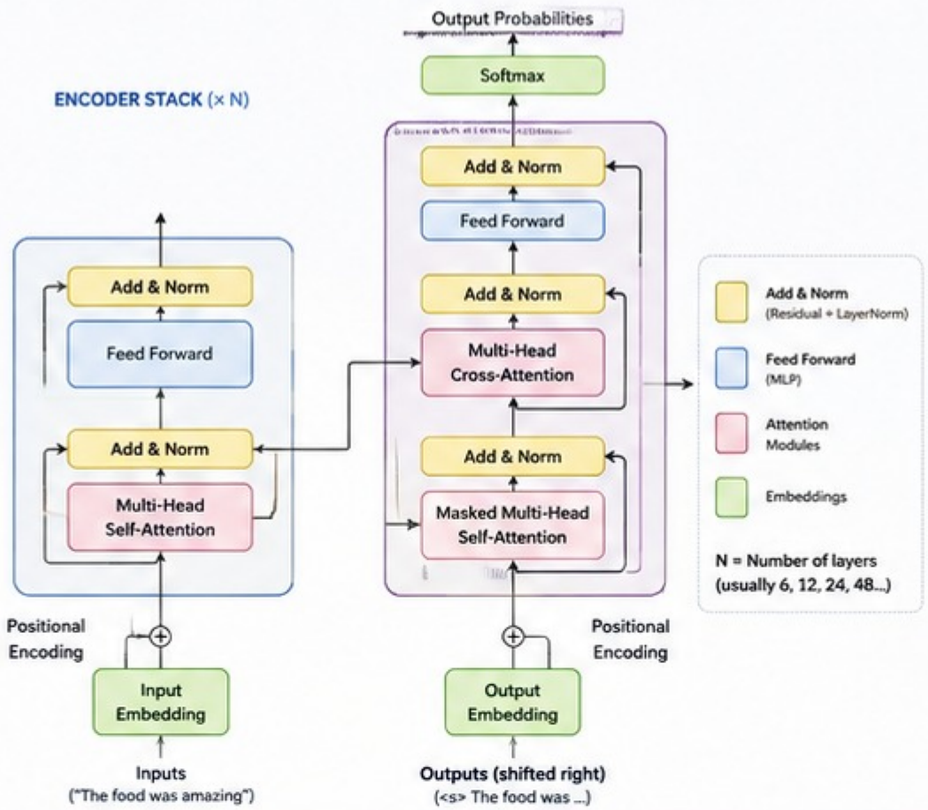
1. THE BIG PICTURE



Why Transformers?

- Capture long-range dependencies
- Process all words in parallel (faster training)
- Highly scalable
- Better performance on many NLP tasks

2. TRANSFORMER ARCHITECTURE



3. SELF-ATTENTION (The Heart)

Self-Attention lets each word look at every other word and decide how much attention to pay.

Example Sentence:

The | food | was | absolutely | amazing | !

Self-Attention Visualization

How it works (Math):

1. Create 3 vectors for each word
   Query (Q), Key (K), Value (V)
2. Attention Score = $softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)$
3. Output = Attention Score × V

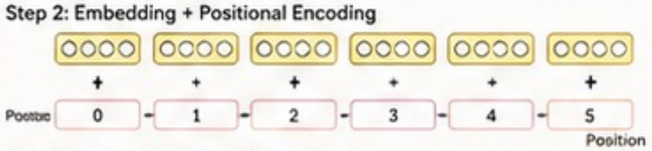
4. HOW IT WORKS (Example)

Input Sentence: I | love | NLP | transformers

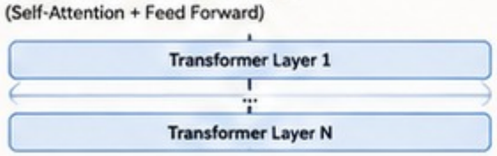
Step 1: Tokenization

ids	[101]	I	love	NLP	transformers	[SEP]
Token IDs	101	1045	2293	17953	42421	102

Step 2: Embedding + Positional Encoding



Step 3: Pass through Transformer Layers
(Self-Attention + Feed Forward)



Step 4: Output for Downstream Tasks

Classification (Sentiment)	NER (Person, Org)	Translation (Generate Text)	QA (Answer)

5. POPULAR TRANSFORMER MODELS

Model	Type	Key Features	Used For
BERT (2018)	Encoder Only	Bidirectional understanding (uses context from both left & right)	Text classification, NER, QA, Sentence similarity
GPT (2018)	Decoder Only	Autoregressive generation (left to right)	Text generation, Chatbots, Completion
T5 (2019)	Encoder-Decoder	Text-to-Text framework ("Everything is a text")	Translation, Summarization, QA, Generation
RoBERTa (2019)	Encoder Only	Improved BERT training (more data, no NSP)	Better performance on BERT tasks
BART (2019)	Encoder-Decoder	Denoising autoencoder + generation	Summarization, Generation, Translation
ViT (for Vision)	Transformer (Encoder)	Applies transformer to images (vision tasks)	Image classification, Object detection

Many newer models (DeBERTa, ELECTRA, Longformer, LLaMA, PaLM, etc.) build on these ideas with improvements in training and efficiency.

6. APPLICATIONS

Text Classification
Sentiment analysis, spam detection, intent classification

Named Entity Recognition (NER)
Extract people, organizations, locations, dates, etc.

Machine Translation
Translate text from one language to another

Text Summarization
Generate short summaries of long documents

Question Answering
Extract answers from a passage

Chatbots & Virtual Assistants
Power conversational AI (e.g., ChatGPT)

Content Generation
Blog writing, story generation, code generation

Key Takeaway: Transformers changed NLP by using attention to understand context better, train faster (in parallel), and scale to very large models that achieve state-of-the-art results.

Fun Fact: The original paper "Attention Is All You Need" (2017) introduced transformers and removed RNNs/CNNs entirely!

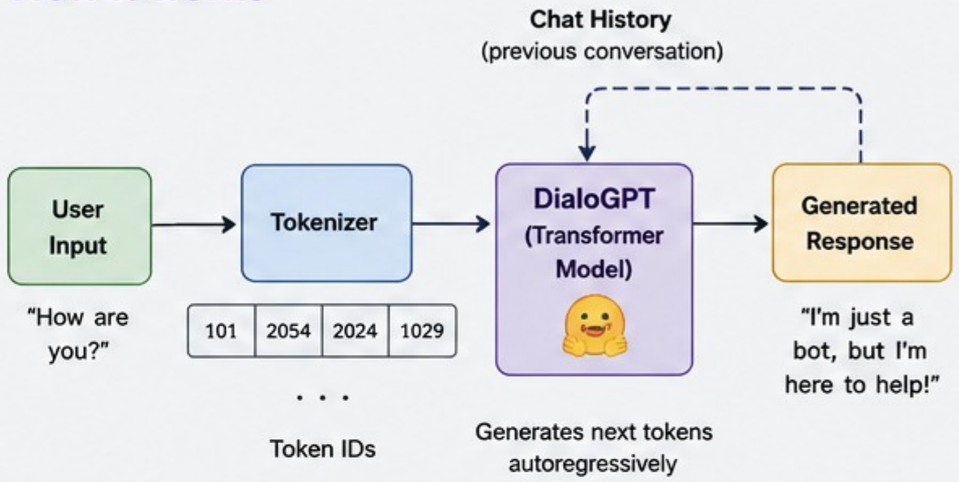
Build a Chatbot using Transformers (Hugging Face)

A simple conversational chatbot using a pre-trained Transformer model (DialoGPT)

chatbot_transformers.py

```python
# Run: pip install transformers torch

from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

# 1. Load pre-trained model and tokenizer
model_name = "microsoft/DialoGPT-medium"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

# 2. Set pad token (DialoGPT doesn't have one by default)
tokenizer.pad_token = tokenizer.eos_token

# 3. Chat function
def get_response(user_input, chat_history_ids):
    # Encode the new user input
    new_user_input_ids = tokenizer.encode(user_input + tokenizer.eos_token,
                                          return_tensors='pt')
    # Append the new user input tokens to the chat history
    bot_input_ids = torch.cat([chat_history_ids, new_user_input_ids], dim=-1)
    # Generate a response
    output_ids = model.generate(
        bot_input_ids,
        max_length=1000,
        pad_token_id=tokenizer.eos_token_id,
        do_sample=True,
        top_k=50,
        top_p=0.95,
        temperature=0.7
    )
    # Decode and return the response (only the new part)
    response = tokenizer.decode(output_ids[:, bot_input_ids.shape[-1]:][0], skip_special_tokens=True)
    return response, output_ids

# 4. Start conversation loop
print("🤖 Chatbot: Hello! I am your assistant. Type 'exit' to end the chat.")
chat_history_ids = torch.empty((1, 0), dtype=torch.long)

while True:
    user_input = input("You: ")
    if user_input.lower() in ["exit", "quit"]:
        print("Chatbot: Goodbye! Have a great day! 👋")
        break
    response, chat_history_ids = get_response(user_input, chat_history_ids)
    print("Chatbot:", response.strip())
```

How it works



Example Conversation

You: Hello!

Chatbot: Hello! How can I assist you today?

You: What is Python?

Chatbot: Python is a high-level programming language known for its simplicity and readability. It is widely used in web development, data science, automation, and more.

You: Who created Python?

Chatbot: Python was created by Guido van Rossum and first released in 1991.

You: exit

Chatbot: Goodbye! Have a great day! 👋

Requirements

```
pip install transformers torch
```

- **transformers** → Hugging Face library
- **torch** → PyTorch (backend)

NLTK (Natural Language Toolkit)

NLTK is a powerful Python library for working with human language data.

1. What is NLTK?

- Platform to build Python programs to work with human language data.
- Provides easy-to-use interfaces to 50+ corpora and lexical resources.
- Well-suited for text processing, classification, tokenization, stemming, tagging, parsing and more.

2. Features

- Tokenization
- Stopwords
- Stemming & Lemmatization
- Part-of-Speech Tagging
- Named Entity Recognition
- Parsing
- Text Classification
- Sentiment Analysis
- Access to Corpora (brown, gutenberg, wordnet)

3. Installation

```
pip install nltk

import nltk
nltk.download() # Download packages
```

Common downloads:

punkt, stopwords, wordnet, averaged_perceptron_tagger, brown, maxent_ne_chunker, words

4. Basic Operations with Examples

1. Tokenization

```
import nltk
from nltk.tokenize import word_tokenize,
   sent_tokenize

text = "NLTK is amazing! It makes NLP easy."

# Word Tokenization
words = word_tokenize(text)
print(words)

# Sentence Tokenization
sents = sent_tokenize(text)
print(sents)
```

Output (words):
`['NLTK', 'is', 'amazing', '!', 'It', 'makes', 'NLP', 'easy','.']`
Output (sentences):
`['NLTK is amazing!', 'It makes NLP easy.']`

2. Stopwords

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))

text = "This is a sample sentence showing
stopwords."
words = word_tokenize(text)
filtered = [w for w in words if w.lower()
            not in stop_words]
print(filtered)
```

Output:
`['sample', 'sentence', 'showing', 'stopwords','.']`

3. Stemming

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()

words = ["running", "runner",
       "easily", "fairly", "studies"]
stemmed = [ps.stem(w) for w in words]
print(stemmed)
```

Output:
`['run', 'runner', 'easili', 'fairli', 'studi']`

4. Lemmatization

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()

words = ["running", "runner", "better", "cats"]
lemmatized = [lemmatizer.lemmatize(w)
              for w in words]
print(lemmatized)
```

Output:
`['running', 'runner', 'better', 'cat']`

5. Part-of-Speech Tagging

```
from nltk import pos_tag, word_tokenize
text = "NLTK is a powerful library."
words = word_tokenize(text)
tags = pos_tag(words)
print(tags)
```

Output:
`[('NLTK', 'NNP'), ('is', 'VBZ'), ('a', 'DT'),
('powerful', 'JJ'), ('library', 'NN'), ('.','.')]`

6. Named Entity Recognition

```
from nltk import ne_chunk, pos_tag, word_tokenize
text = "Elon Musk is the CEO of Tesla."
words = word_tokenize(text)
tags = pos_tag(words)
entities = ne_chunk(tags)
print(entities)
```

Output (tree):
```
(S
  (PERSON Elon/NNP Musk/NNP)
  is/VBZ the/DT CEO/NN of/IN
  (ORGANIZATION Tesla/NNP) ./.)
```

7. WordNet (Lexical Database)

```
from nltk.corpus import wordnet as wn
syns = wn.synsets('good')
print(syns[0].definition())
print(syns[0].examples())
```

Output:
`that which is pleasing or valuable or useful`
`['a good reputation", 'good manners']`

8. Text Classification (Naive Bayes Example)

```
from nltk.classify import NaiveBayesClassifier
from nltk.corpus import movie_reviews

# (Load movie reviews)
# classifier = NaiveBayesClassifier.train(train_set)
# print(classifier.classify(test_set[0][0]))
# print(nltk.classify.util.accuracy(classifier, test_set))
```

Output:
pos
Accuracy: 0.80+ (varies)

5. Popular Corpora in NLTK

- **brown** — Collection of texts from the Brown Corpus (literature, news, etc.)
- **gutenberg** — E-books from Project Gutenberg
- **reuters** — Collection of Reuters news articles
- **movie_reviews** — Movie reviews for sentiment analysis
- **stopwords** — List of stopwords for many languages
- **wordnet** — Lexical database of English

6. Commonly Used Modules

`nltk.tokenize`	Tokenization tools
`nltk.corpus`	Access to corpora
`nltk.stem`	Stemmers and Lemmatizers
`nltk.tag`	POS tagging
`nltk.chunk`	Chunking / NER
`nltk.classify`	Text classification
`nltk.parse`	Parsing (CFG, dependency)
`nltk.sentiment`	Sentiment analysis
`nltk.metrics`	Evaluation metrics

7. Applications

- **Text preprocessing** (Tokenization, stopword removal, stemming)
- **Information Retrieval** (Search, indexing)
- **Text Classification** (Spam detection, sentiment analysis)
- **Chatbots & Conversational AI**
- **Machine Translation**
- **Language Processing Research**

Key Takeaway: NLTK provides all the building blocks for NLP tasks. It is beginner-friendly, well-documented, and widely used in academia and industry.

Resources:
Official Docs: https://www.nltk.org/
Book: Natural Language Processing with Python NLTK

Sentiment Analysis in Python using NLTK (VADER)

nltk
Natural Language Toolkit

VADER (Valence Aware Dictionary and sEntiment Reasoner) is a lexicon and rule-based sentiment analysis tool, specifically tuned for social media text.

```python
# 1. Import VADER SentimentIntensityAnalyzer
from nltk.sentiment import SentimentIntensityAnalyzer
import nltk

# 2. Download the VADER lexicon (only first time)
nltk.download('vader_lexicon')

# 3. Initialize the analyzer
analyzer = SentimentIntensityAnalyzer()

# 4. Function to get sentiment
def get_sentiment(text):
    scores = analyzer.polarity_scores(text)
    compound = scores['compound']

    # Determine sentiment label based on compound score
    if compound >= 0.05:
        sentiment = "Positive 😀"
    elif compound <= -0.05:
        sentiment = "Negative ☹️"
    else:
        sentiment = "Neutral 😐"
    return scores, sentiment

# 5. Analyze example sentences
texts = [
    "I love this product! It works great! 😀",
    "This is the worst experience ever. Totally disappointed!🤬",
    "The product is okay, not good but not bad either.",
    "Absolutely fantastic service!!! Highly recommended!! 🎉",
    "I will never buy this again. Waste of money. 👎"
]
# 6. Run analysis and display results
for text in texts:
    scores, sentiment = get_sentiment(text)
    print(f"Text: {text}")
    print(f"Scores: {scores}")
    print(f"Sentiment: {sentiment}")
print('-' * 80)
```

What does VADER return?

Score	Range	Description
positive	0 to 1	Proportion of positive sentiment
negative	0 to 1	Proportion of negative sentiment
neutral	0 to 1	Proportion of neutral sentiment
compound	-1 to +1	Normalized, weighted composite score

Compound Score Interpretation

- compound >= 0.05 → Positive
- -0.05 < compound < 0.05 → Neutral
- compound <= -0.05 → Negative

How VADER Works

Uses a sentiment lexicon where each word has a valence score.

Applies rules for:

- Punctuation (e.g., !, ?, !!!)
- Capitalization (e.g., GREAT vs great)
- Degree modifiers (e.g., very good)
- Negations (e.g., not good)
- Contrast (e.g., good but bad)

Example Output

```
Text: I love this product! It works great!😀
Scores: {'neg': 0.0, 'neu': 0.182, 'pos': 0.818, 'compound': 0.8720}
Sentiment: Positive 😀
--------------------------------------------------------------------------------
Text: This is the worst experience ever. Totally disappointed! 😡
Scores: {'neg': 0.715, 'neu': 0.285, 'pos': 0.0, 'compound': -0.8176}
Sentiment: Negative ☹️
--------------------------------------------------------------------------------
Text: The product is okay, not good but not bad either.
Scores: {'neg': 0.0, 'neu': 0.738, 'pos': 0.262, 'compound': 0.3612}
Sentiment: Neutral 😐
--------------------------------------------------------------------------------
Text: Absolutely fantastic service!!! Highly recommended!! 🎉
Scores: {'neg': 0.0, 'neu': 0.155, 'pos': 0.845, 'compound': 0.9177}
Sentiment: Positive 😀
--------------------------------------------------------------------------------
Text: I will never buy this again. Waste of money. 👎
Scores: {'neg': 0.776, 'neu': 0.224, 'pos': 0.0, 'compound': -0.8934}
Sentiment: Negative ☹️
```

Complete Code (All-in-One)

```
from nltk.sentiment import SentimentIntensityAnalyzer; import nltk;
nltk.download('vader_lexicon'); analyzer = SentimentIntensityAnalyzer();
texts = ["I love this!😀", "This is bad 😡", "It is okay.", "Amazing!!! 🎉"];
for text in texts: scores = analyzer.polarity_scores(text);
compound = scores['compound']; sentiment = "Positive"  if compound >= 0.05
else "Negative"  if compound <= -0.05
else "Neutral"; print(text, "->", sentiment, scores)
```

Tips

- Best for short texts like reviews, tweets, comments.
- Not recommended for long, complex documents.
- For advanced needs, try ML/DL models (BERT, RoBERTa, etc.).

Applications

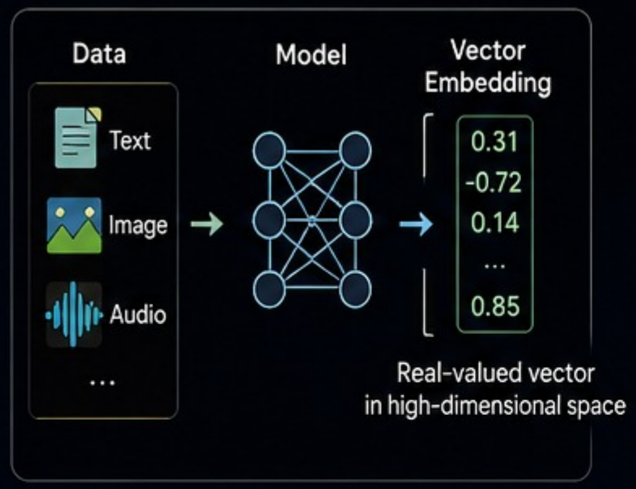
- Social media monitoring
- Customer review analysis
- Brand sentiment tracking
- Chatbots & conversational AI

Key Takeaway

VADER in NLTK is simple, fast and effective for sentiment analysis of social media and short text data.

VECTOR EMBEDDINGS

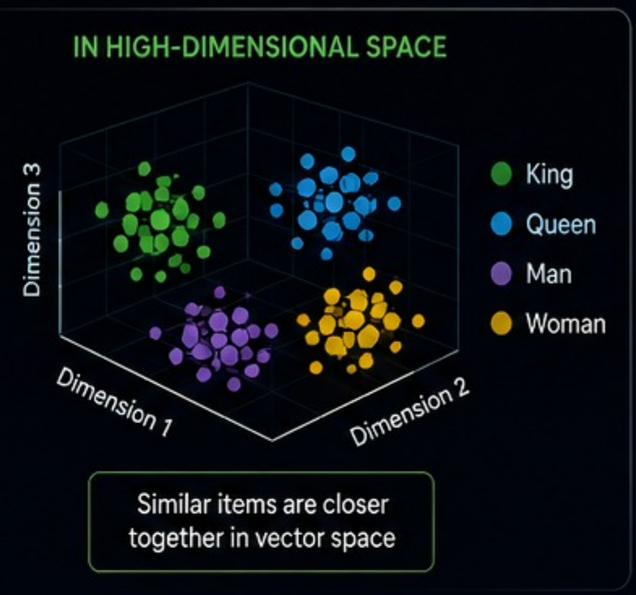
REAL LIFE USES IN DATA SCIENCE



Vector embeddings convert data (text, images, audio, etc.) into dense numerical vectors that capture meaning and context.

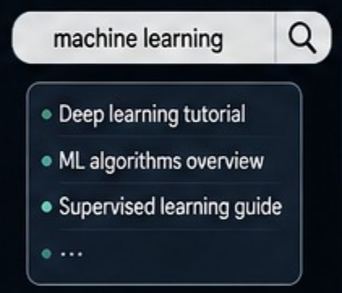
WHY VECTOR EMBEDDINGS?

- Capture semantic meaning
- Measure similarity effectively
- Work with many data types
- Power many AI/ML applications



REAL LIFE USE CASES

1 SEMANTIC SEARCH



How it helps: Understands meaning, not just keywords.

Example: Search engines, enterprise document search, e-commerce search.

2 RECOMMENDER SYSTEMS

How it helps: Finds items similar to what users like.

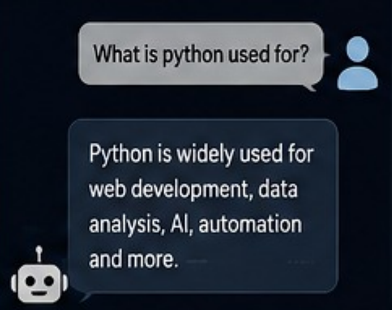
Example: Netflix/YouTube recommendations, product recommendations.

3 DOCUMENT CLUSTERING

How it helps: Groups similar documents automatically.

Example: Organizing reports, news articles, support tickets.

4 CHATBOTS & QUESTION ANSWERING



How it helps: Retrieves relevant information to answer queries.

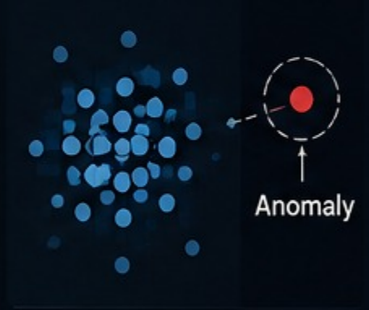
Example: Chatbots, virtual assistants, helpdesk systems.

5 IMAGE SEARCH & RECOGNITION

How it helps: Finds visually similar images.

Example: Reverse image search, product matching, fashion recommendations.

6 FRAUD DETECTION & ANOMALY DETECTION



How it helps: Detects unusual patterns in data.

Example: Credit card fraud detection, network intrusion detection, anomaly monitoring.

DATA TYPES THAT CAN BE EMBEDDED

Text · Images · Audio · Video · Graphs · Structured Data · and more

KEY TAKEAWAY

Vector embeddings help machines understand similarity and context, enabling smarter search, recommendations, automation and decisions.

WORD EMBEDDINGS: REAL LIFE USES & HOW TO DO (With Practice)

Word embeddings convert words into dense vectors of numbers that capture meaning, context and relationships between words.

REAL LIFE USES OF WORD EMBEDDINGS

1. Text Classification

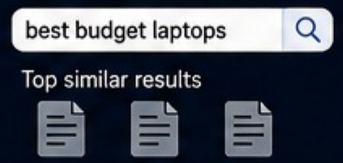
Capture semantic meaning of words to classify emails, tweets, news articles, etc.

2. Sentiment Analysis

Understand the meaning of words in a sentence to predict positive, negative or neutral sentiment.

3. Search & Information Retrieval

Find documents similar to a query even if exact words do not match.



4. Recommender Systems

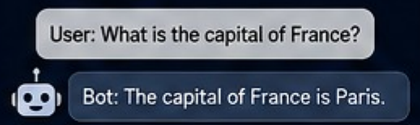
Understand user preferences and recommend movies, books, products, etc.

5. Machine Translation

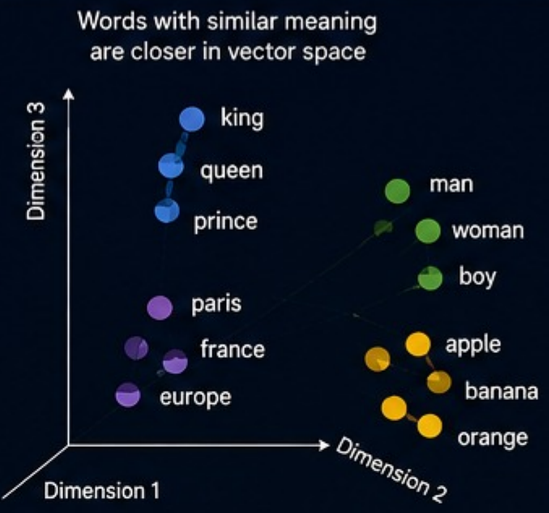
Represent words in a way that helps models translate between languages better.

6. Chatbots & Q&A Systems

Understand user questions and generate accurate responses.



HOW WORD EMBEDDINGS WORK (VISUALIZATION)



Example: king - man + woman ≈ queen

Word	Vector (sample values)				
	1	2	3	...	n
king	0.12	-0.35	0.22	...	0.15
queen	0.10	-0.30	0.21	...	0.17
man	-0.20	0.10	-0.05	...	0.08
woman	-0.18	0.12	-0.07	...	0.09
...					

Key Idea: Words are represented as vectors of real numbers. Machine learning models can understand relationships, similarities and context using these vectors.

HOW TO DO WORD EMBEDDINGS IN PYTHON (PRACTICAL)

STEP 1: Install Library

```
!pip install gensim
```

STEP 2: Import Libraries

```
import gensim
from gensim.models import Word2Vec

sentences = [
  ['king','is','a','good','man'],
  ['queen','is','a','wise','woman'],
  ['paris','is','in','france'],
  ['apple','is','a','fruit'],
  ['boy','is','playing'],
  ['girl','is','reading']
]
```

STEP 3: Train Word2Vec Model

```
model = Word2Vec(
    sentences,
    vector_size=100,    # dimensions of vectors
    window=5,           # context window size
    min_count=1,        # ignore rare words
    workers=4
)
print("Model training complete!")
```

STEP 4: Get Word Vector

```
vector = model.wv['king']
print(vector[:10])     # first 10 values
```

STEP 5: Find Similar Words

```
similar_words = model.wv.most_similar('king', topn=5)
print(similar_words)
```

STEP 6: Analogy (King - Man + Woman ≈ Queen)

```
result = model.wv.most_similar(
    positive=['king','woman'],
    negative=['man'],
    topn=1
)
print(result)
```

SAMPLE OUTPUT (You will get similar results)

```
Model training complete!
[ 0.0213 -0.1456  0.0321  0.0876 -0.0112 -0.0921  0.0210
  0.0132  0.0678 -0.0234]

[('queen', 0.8721), ('prince', 0.8453), ('monarch', 0.7722),
 ('throne', 0.7015), ('ruler', 0.6890)]

[('queen', 0.81)]
```

PRACTICE IDEAS

1. Train on Larger Data

Use your own sentences, books or Wikipedia data.

2. Try with Different Models

Word2Vec (CBOW/Skip-gram), GloVe, FastText.

3. Build Applications

- Semantic Search Engine
- Recommendation System
- Chatbot
- Document Similarity Checker

TIP: The quality of embeddings depends on the quantity and quality of text data and proper hyperparameter tuning.

POPULAR EMBEDDING TECHNIQUES

Word2Vec (2013)

GloVe (2014)

FastText (2016)

BERT (Contextual)

Sentence Transformers

BENEFITS

- ✓ Capture semantic meaning
- ✓ Improve performance in NLP tasks
- ✓ Work with ML/DL models
- ✓ Reduce dimensionality & sparsity

WORD EMBEDDINGS – PRACTICE EXAMPLE ON DATASET

Learn how to build word embeddings and use them for Sentiment Analysis

1. PROBLEM STATEMENT

Build a sentiment classification model on movie reviews using word embeddings (Word2Vec).

Goal:

Classify a review as Positive or Negative.

Positive → Negative

2. DATASET

IMDB Dataset of 50K Movie Reviews
(25K positive, 25K negative)

	review	sentiment
1	A great movie with wonderful acting	positive
2	Awesome film! I really enjoyed it	positive
3	Terrible movie. Waste of time	negative
4	Worst film ever. Very boring	negative
...	...	...

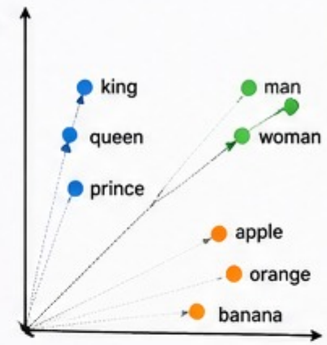
Features: Text (review) **Target:** sentiment (0/1)

3. WORKFLOW

Text Preprocessing
↓
Train Word Embedding (Word2Vec)
↓
Convert Reviews to Average Vectors
↓
Train ML Model (Logistic Regression)
↓
Evaluate Model

4. WORD EMBEDDINGS (Word2Vec)

Words are converted into dense vectors of real numbers that capture meaning.



Example:

king - man + woman ≈ queen

Word Vector (Sample – 5 dims)

Word	1	2	3	4	5
king	0.21	-0.13	0.45	0.12	-0.31
queen	0.18	-0.11	0.40	0.10	-0.28
man	-0.22	0.23	-0.31	0.41	0.05
woman	-0.19	0.21	-0.28	0.38	0.07

5. IMPLEMENTATION IN PYTHON (PRACTICAL)

Step 1: Import Libraries

```
import pandas as pd
import re
from gensim.models import Word2Vec
from gensim.utils import simple_preprocess
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import
     accuracy_score, classification_report
```

Step 2: Load Dataset

```
df = pd.read_csv('IMDB Dataset.csv')
print(df.shape)
df.head()
```

	review	sentiment
0	One of the other reviewers has mentioned...	positive
1	A wonderful little production. ...	positive
2	I thought this was a wonderful way to spend...	positive
3	Basically there's a family where a little boy...	negative
4	Petter Mattei's "Love in the Time of Money"...	negative
...	...	...

Step 3: Text Preprocessing

```
def preprocess(text):
    text = re.sub('<[^<]+?>', ' ', text)  # remove html tags
    text = re.sub(['^a-zA-Z]]', ' ', text)  # remove special chars
    text = text.lower()
    return simple_preprocess(text, deacc=True)

df['tokens'] = df['review'].apply(preprocess)
df['sentiment'] = df['sentiment'].map({'positive':1, 'negative':0})
print(df[['tokens', 'sentiment']].head(2))
```

Step 4: Train Word2Vec Model

```
sentences = df['tokens'].tolist()

model = Word2Vec(sentences=sentences,
                 vector_size=100, window=5,
                 min_count=2, workers=4, sg=1)

model.save('word2vec_imdb.model')
print('Vocabulary size:', len(model.wv))
```

Vector size = 100 means each word is represented as 100-dimensional dense vector.

Step 5: Convert Review to Average Vector

```
import numpy as np

def document_vector(tokens, model, size):
    vec = np.zeros(size).reshape((1, size))
    count = 0
    for word in tokens:
        if word in model.wv:
            vec += model.wv[word].reshape((1, size))
            count += 1
    if count != 0:
        vec /= count
    return vec


X = np.vstak(df['tokens'].apply(lambda x:
  document_vecter(x, model, 100)))
y = df['sentiment'].values
```

Step 6: Train & Evaluate Model

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print('Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred,
        target_names=['Negative', 'Positive']))
```

SAMPLE OUTPUT

```
Accuracy: 0.8745
              precision    recall  f1-score   support

    Negative       0.86      0.89      0.87      4968
    Positive       0.89      0.86      0.87      5032

    accuracy                           0.87     10000
   macro avg       0.87      0.87      0.87     10000
weighted avg       0.87      0.87      0.87     10000
```

TOP 5 MOST SIMILAR WORDS (Example)

```
model.wv.most_similar('king')
[('queen', 0.872),
 ('prince', 0.845),
 ('monarch', 0.801),
 ('ruler', 0.782),
 ('throne', 0.769)]
```

```
model.wv.most_similar('good')
[('great', 0.862),
 ('fantastic', 0.811),
 ('excellent', 0.797),
 ('amazing', 0.780),
 ('wonderful', 0.779)]
```

WHAT YOU PRACTICED

- ☑ Loaded and explored a real-world text dataset
- ☑ Preprocessed text data
- ☑ Trained Word2Vec word embeddings
- ☑ Converted text to vector representations
- ☑ Built a Machine Learning model
- ☑ Evaluated performance

KEY TAKEAWAYS

- Word embeddings capture semantic meaning of words.
- Similar words are closer in vector space.
- They help ML models understand text better.
- Useful for many NLP tasks:
  Sentiment Analysis, Search, Recommendations, Chatbots, etc.

NEXT STEPS

- Try different models: CBOW vs Skip-gram
- Use other embeddings: GloVe, FastText
- Build deep learning models (LSTM/CNN) on embeddings
- Hyperparameter tuning for better performance

WORD EMBEDDINGS PRACTICE: TF-IDF vs WORD2VEC

On Movie Reviews Dataset (IMDB 50K)

Goal: Compare TF-IDF (sparse) vs Word2Vec (dense) representations for Sentiment Analysis

1. DATASET

IMDB Dataset
50,000 reviews
(25K positive, 25K negative)

- Text (review)
- Target (sentiment: 0/1)

2. WORKFLOW

1. Load & Explore Data
2. Text Preprocessing
3. Create Features (TF-IDF / Word2Vec)
4. Train ML Model (Logistic Regression)
5. Evaluate & Compare Results

3. TEXT PREPROCESSING

- Lowercase
- Remove HTML tags
- Remove special characters & digits
- Remove stopwords
- Lemmatize

Example:

Raw:
This MOVIE was AWESOME!!! 10/10 😮

Clean: this movie awesome

4. SAMPLE DATA

Review (after preprocessing)	Sentiment
a great movie with wonderful acting	1
absolutely loved it super entertaining	1
terrible movie waste of time	0
boring and disapponiting plot	0
...	...

5A. TF-IDF REPRESENTATION

How it works: Represents text as sparse vectors based on Term Frequency–Inverse Document Frequency.

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=5000)
X_tfidf = vectorizer.fit_transform(df['review'])

print(X_tfidf.shape)     # (50000, 5000)
```

TF-IDF Vector Sample (5 terms)

	good	movie	bad	acting	boring
a great movie ...	0.21	0.45	0.00	0.19	0.00
absolutely loved ...	0.18	0.38	0.00	0.00	0.00
terrible movie ...	0.00	0.31	0.34	0.00	0.22
boring and ...	0.00	0.22	0.00	0.00	0.43

Model with TF-IDF Features

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
 X_tfidf, df['sentiment'], test_size=0.2, random_state=42)

lr_tfidf = LogisticRegression(max_iter=1000)
lr_tfidf.fit(X_train, y_train)
y_pred_tfidf = lr_tfidf.predict(X_test)

acc_tfidf = accuracy_score(y_test, y_pred_tfidf)
print(f"TF-IDF Accuracy: {acc_tfidf:.4f}")
print(classification_report(y_test, y_pred_tfidf))
```

5B. WORD2VEC REPRESENTATION

How it works: Learns dense vector representations of words in a continuous vector space.

```
from gensim.models import Word2Vec
import numpy as np

sentences = [review.split() for review in df['review']]

w2v_model = Word2Vec(sentences=sentences,
            vector_size=100, window=5, min_count=2,
            workers=4, sg=1, seed=42)

print(f"Vocabulary Size: {len(w2v_model.wv):,}")
```

Word2Vec Vector Sample (5 dims)

Word	Dim 1	Dim 2	Dim 3	Dim 4	Dim 5
good	0.21	-0.43	0.12	0.31	-0.27
movie	0.19	0.07	-0.13	0.04	0.22
bad	-0.29	0.38	0.11	-0.20	0.05
acting	0.10	-0.22	0.24	0.15	-0.34
boring	-0.18	0.29	-0.07	-0.11	0.26

Convert Review to Average Word2Vec Vector

```
def get_avg_w2v(tokens, model, vector_size=100):
    vec = np.zeros(vector_size)
    count = 0
    for word in tokens:
        if word in model.wv:
            vec += model.wv[word]
            count += 1
    if count != 0: vec /= count
    return vec
```

Model with Word2Vec Features

```
X_w2v = np.array([get_avg_w2v(tokens, w2v_model)
                  for tokens in sentences])
X_train, X_test, y_train, y_test = train_test_split(
  X_w2v, df['sentiment'], test_size=0.2, random_state=42)
lr_w2v = LogisticRegression(max_iter=1000)
lr_w2v.fit(X_train, y_train)
y_pred_w2v = lr_w2v.predict(X_test)

acc_w2v = accuracy_score(y_test, y_pred_w2v)
print(f"Word2Vec Accuracy: {acc_w2v:.4f}")
print(classification_report(y_test, y_pred_w2v))
```

6. RESULTS COMPARISON

Performance Comparison

Model / Representation	Accuracy	Precision (Pos)	Recall (Pos)	F1-Score (Pos)
TF-IDF (LogReg)	0.8861	0.8840	0.8891	0.8865
Word2Vec (Avg Vector) (LogReg)	0.8492	0.8427	0.8554	0.8489

Interpretation

TF-IDF performs slightly better for this dataset with Logistic Regression. Word2Vec captures semantic meaning in dense vectors and is useful for deep learning models and transfer learning.

7. WORD2VEC SIMILARITY EXAMPLES

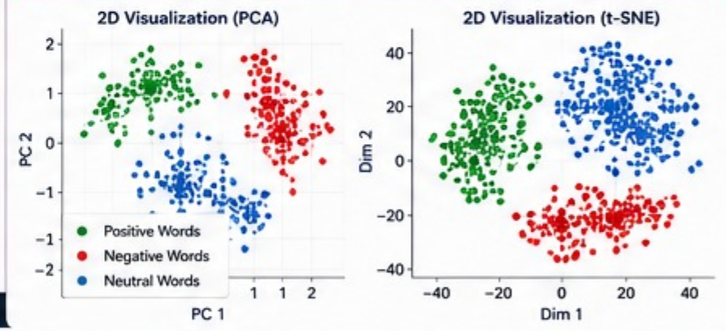
Top 5 similar words to "king"

queen	0.872
prince	0.846
monarch	0.804
throne	0.769
ruler	0.742

Top 5 similar words to "movie"

film	0.861
movies	0.849
cinema	0.781
theater	0.744
flick	0.701

8. VISUALIZATION (WORD2VEC)



9. SAMPLE OUTPUT (Classification Report)

TF-IDF + Logistic Regression

	precision	recall	f1-score	support
0	0.88	0.88	0.88	4968
1	0.89	0.89	0.89	5032
accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

Accuracy: 0.8861

Word2Vec (Avg) + Logistic Regression

	precision	recall	f1-score	support
0	0.85	0.84	0.85	4968
1	0.85	0.86	0.85	5032
accuracy			0.85	10000
macro avg	0.85	0.85	0.85	10000
weighted avg	0.85	0.85	0.85	10000

Accuracy: 0.8492

ENVIRONMENT & LIBRARIES

- Python 3.x
- pandas, numpy
- scikit-learn
- nltk, gensim
- matplotlib, seaborn

10. KEY TAKEAWAYS

- ✅ TF-IDF is simple, sparse, and works well with linear models.
- ✅ Word2Vec provides dense semantic representations.
- ✅ Word embeddings are more powerful with deep learning models (LSTM, CNN, Transformer).
- ✅ Average Word2Vec is a good baseline; advanced methods use context.
- ✅ Always preprocess text carefully for better results.

11. NEXT STEPS

- ☑ Try other embeddings: GloVe, FastText
- ☑ Use Word2Vec with LSTM / Transformers
- ☑ Hyperparameter tuning
- ☑ Try different ML models (SVM, XGBoost)
- ☑ Deploy as a web app (Streamlit / Flask)

Useful Commands

```
pip install gensim scikit-learn pandas numpy nltk matplotlib seaborn
python -m nltk.downloader stopwords wordnet
```

Dataset Source

IMDB Large Movie Review Dataset
(https://ai.stanford.edu/~amaas/data/sentiment/)

About Word2Vec

Developed by Google (Mikolov et al., 2013)
Two architectures: CBOW & Skip-gram (sg=0 or 1)

WORD EMBEDDINGS PRACTICE: TF-IDF vs WORD2VEC

On Movie Reviews Dataset (IMDB 50K)

Goal: Compare TF-IDF (sparse) vs Word2Vec (dense) representations for Sentiment Analysis

1. DATASET

IMDB Dataset
50,000 reviews
(25K positive, 25K negative)

- Text (review)
- Target (sentiment: 0/1)

2. WORKFLOW

1. Load & Explore Data
2. Text Preprocessing
3. Create Features (TF-IDF / Word2Vec)
4. Train ML Model (Logistic Regression)
5. Evaluate & Compare Results

3. TEXT PREPROCESSING

- Lowercase
- Remove HTML tags
- Remove special characters & digits
- Remove stopwords
- Lemmatize

Example:

Raw:
This MOVIE was AWESOME!!! 10/10 😮

Clean: this movie awesome

4. SAMPLE DATA

Review (after preprocessing)	Sentiment
a great movie with wonderful acting	1
absolutely loved it super entertaining	1
terrible movie waste of time	0
boring and disapponiting plot	0
...	...

5A. TF-IDF REPRESENTATION

How it works: Represents text as sparse vectors based on Term Frequency–Inverse Document Frequency.

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=5000)
X_tfidf = vectorizer.fit_transform(df['review'])

print(X_tfidf.shape)     # (50000, 5000)
```

TF-IDF Vector Sample (5 terms)

	good	movie	bad	acting	boring
a great movie ...	0.21	0.45	0.00	0.19	0.00
absolutely loved ...	0.18	0.38	0.00	0.00	0.00
terrible movie ...	0.00	0.31	0.34	0.00	0.22
boring and ...	0.00	0.22	0.00	0.00	0.43

Model with TF-IDF Features

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
 X_tfidf, df['sentiment'], test_size=0.2, random_state=42)

lr_tfidf = LogisticRegression(max_iter=1000)
lr_tfidf.fit(X_train, y_train)
y_pred_tfidf = lr_tfidf.predict(X_test)

acc_tfidf = accuracy_score(y_test, y_pred_tfidf)
print(f"TF-IDF Accuracy: {acc_tfidf:.4f}")
print(classification_report(y_test, y_pred_tfidf))
```

5B. WORD2VEC REPRESENTATION

How it works: Learns dense vector representations of words in a continuous vector space.

```
from gensim.models import Word2Vec
import numpy as np

sentences = [review.split() for review in df['review']]

w2v_model = Word2Vec(sentences=sentences,
            vector_size=100, window=5, min_count=2,
            workers=4, sg=1, seed=42)

print(f"Vocabulary Size: {len(w2v_model.wv):,}")
```

Word2Vec Vector Sample (5 dims)

Word	Dim 1	Dim 2	Dim 3	Dim 4	Dim 5
good	0.21	-0.43	0.12	0.31	-0.27
movie	0.19	0.07	-0.13	0.04	0.22
bad	-0.29	0.38	0.11	-0.20	0.05
acting	0.10	-0.22	0.24	0.15	-0.34
boring	-0.18	0.29	-0.07	-0.11	0.26

Convert Review to Average Word2Vec Vector

```
def get_avg_w2v(tokens, model, vector_size=100):
    vec = np.zeros(vector_size)
    count = 0
    for word in tokens:
        if word in model.wv:
            vec += model.wv[word]
            count += 1
    if count != 0: vec /= count
    return vec
```

Model with Word2Vec Features

```
X_w2v = np.array([get_avg_w2v(tokens, w2v_model)
                  for tokens in sentences])
X_train, X_test, y_train, y_test = train_test_split(
  X_w2v, df['sentiment'], test_size=0.2, random_state=42)
lr_w2v = LogisticRegression(max_iter=1000)
lr_w2v.fit(X_train, y_train)
y_pred_w2v = lr_w2v.predict(X_test)

acc_w2v = accuracy_score(y_test, y_pred_w2v)
print(f"Word2Vec Accuracy: {acc_w2v:.4f}")
print(classification_report(y_test, y_pred_w2v))
```

6. RESULTS COMPARISON

Performance Comparison

Model / Representation	Accuracy	Precision (Pos)	Recall (Pos)	F1-Score (Pos)
TF-IDF (LogReg)	0.8861	0.8840	0.8891	0.8865
Word2Vec (Avg Vector) (LogReg)	0.8492	0.8427	0.8554	0.8489

Interpretation

TF-IDF performs slightly better for this dataset with Logistic Regression. Word2Vec captures semantic meaning in dense vectors and is useful for deep learning models and transfer learning.

7. WORD2VEC SIMILARITY EXAMPLES

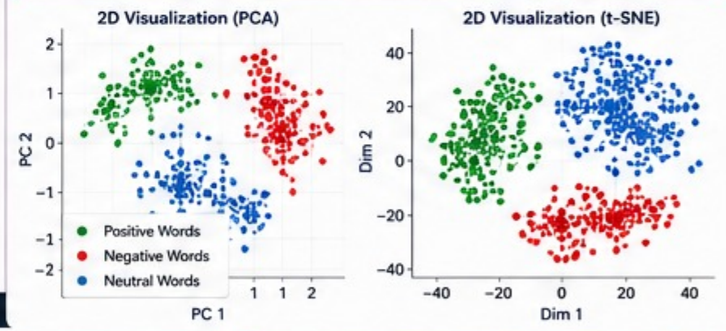
Top 5 similar words to "king"

queen	0.872
prince	0.846
monarch	0.804
throne	0.769
ruler	0.742

Top 5 similar words to "movie"

film	0.861
movies	0.849
cinema	0.781
theater	0.744
flick	0.701

8. VISUALIZATION (WORD2VEC)



9. SAMPLE OUTPUT (Classification Report)

TF-IDF + Logistic Regression

	precision	recall	f1-score	support
0	0.88	0.88	0.88	4968
1	0.89	0.89	0.89	5032
accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

Accuracy: 0.8861

Word2Vec (Avg) + Logistic Regression

	precision	recall	f1-score	support
0	0.85	0.84	0.85	4968
1	0.85	0.86	0.85	5032
accuracy			0.85	10000
macro avg	0.85	0.85	0.85	10000
weighted avg	0.85	0.85	0.85	10000

Accuracy: 0.8492

ENVIRONMENT & LIBRARIES

- Python 3.x
- pandas, numpy
- scikit-learn
- nltk, gensim
- matplotlib, seaborn

10. KEY TAKEAWAYS

- ✅ TF-IDF is simple, sparse, and works well with linear models.
- ✅ Word2Vec provides dense semantic representations.
- ✅ Word embeddings are more powerful with deep learning models (LSTM, CNN, Transformer).
- ✅ Average Word2Vec is a good baseline; advanced methods use context.
- ✅ Always preprocess text carefully for better results.

11. NEXT STEPS

- ☑ Try other embeddings: GloVe, FastText
- ☑ Use Word2Vec with LSTM / Transformers
- ☑ Hyperparameter tuning
- ☑ Try different ML models (SVM, XGBoost)
- ☑ Deploy as a web app (Streamlit / Flask)

Useful Commands

```
pip install gensim scikit-learn pandas numpy nltk matplotlib seaborn
python -m nltk.downloader stopwords wordnet
```

Dataset Source

IMDB Large Movie Review Dataset
(https://ai.stanford.edu/~amaas/data/sentiment/)

About Word2Vec

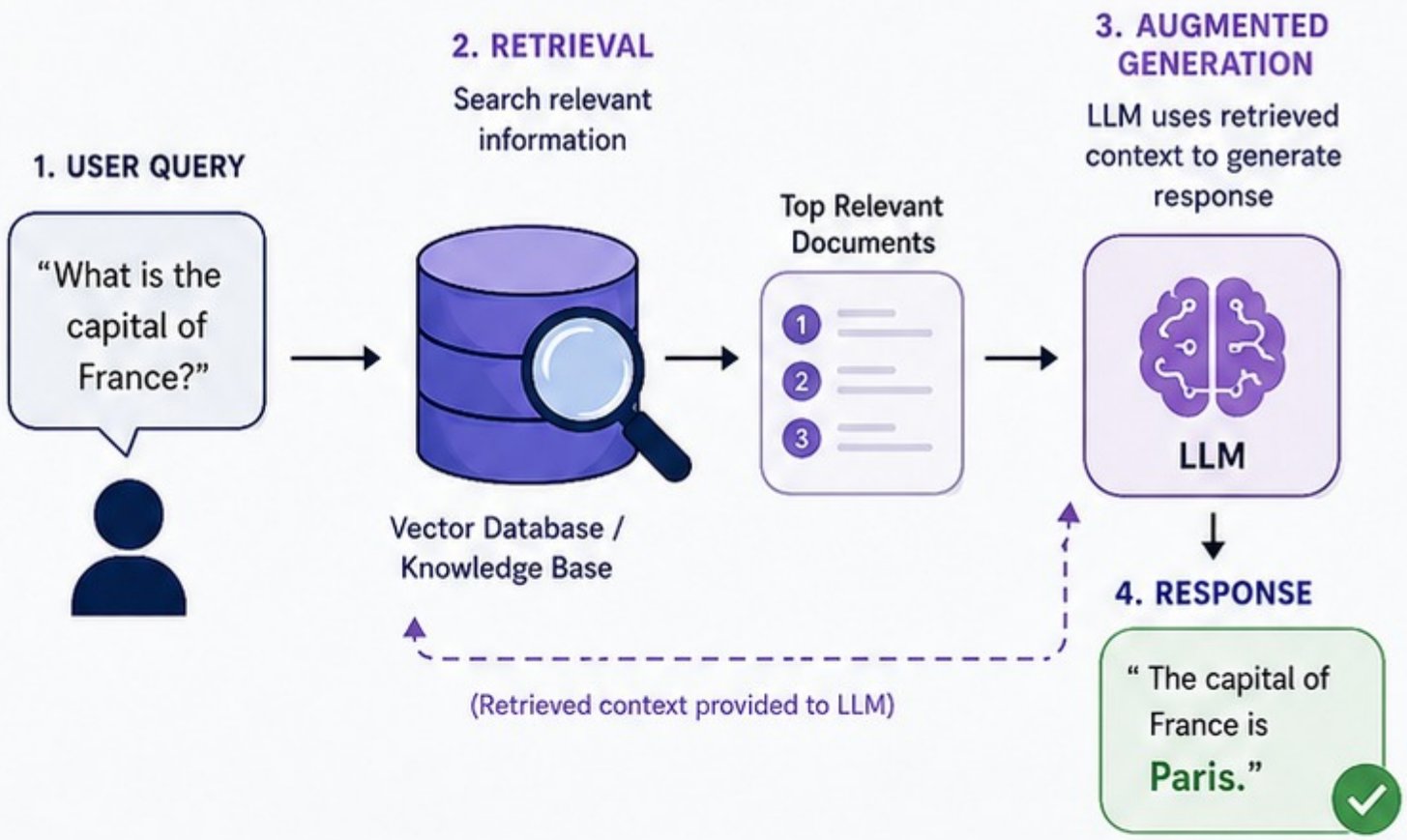
Developed by Google (Mikolov et al., 2013)
Two architectures: CBOW & Skip-gram (sg=0 or 1)

RAG

(Retrieval-Augmented Generation)

Combine External Knowledge with Generative AI

RAG enhances LLMs by retrieving relevant information from external sources and providing it as context to generate accurate, up-to-date responses.



HOW RAG WORKS

1. DATA INGESTION
Load data from various sources (PDFs, Docs, Websites, etc.)

2. INDEXING
Split, clean and convert data to embeddings. Store in a vector database.

3. RETRIEVAL
Convert user query to embedding and retrieve the most relevant information.

4. GENERATION
Pass retrieved context + query to LLM and generate accurate answer.

RAG vs. LLM (Without RAG)

Aspect	LLM (Without RAG)	RAG (With RAG)
Knowledge Source	Limited to training data	Uses external, up-to-date data
Accuracy	May generate incorrect info	More accurate & reliable
Hallucination	Higher chance	Significantly reduced
Update Frequency	Requires model retraining	Just update the knowledge base
Transparency	Hard to trace source	Sources can be referenced
Use Cases	General text generation	Enterprise, QA, Research, Customer support, etc.

REAL LIFE USES

Enterprise Search
Find answers from company docs, policies, and manuals.

Customer Support
Get accurate answers from FAQs, support docs, tickets.

Research & Education
Summarize papers, books and academic content.

Content Creation
Generate content using brand guidelines and internal data.

TECH STACK (EXAMPLE)

Data Sources	PDF, Word, Web, Notion, ...
Embedding Model	OpenAI, Hugging Face, cohere
Vector Database	Pinecone, Weaviate, Chroma
LLM	OpenAI, Meta, Gemini, ...
Frameworks	LangChain, LlamaIndex, Haystack

BENEFITS OF RAG

- More Accurate Responses
- Up-to-Date Information
- Reduces Hallucinations
- Cost Effective (No need to retrain LLMs)
- Easy to Integrate with Existing Systems

RAG: Chat with Your PDF

Ask questions and get accurate answers
from your documents using Retrieval-Augmented Generation

RAG combines the power of LLMs with your documents to provide accurate, context-aware answers with references.

Your PDF

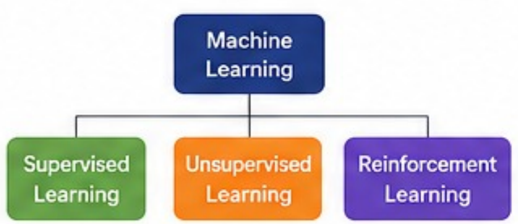
Machine_Learning_Basics.pdf
24.6 MB • 58 pages

12 / 58 — 100% +

3.2 Types of Machine Learning

Machine Learning can be broadly classified into three types:

- **Supervised Learning** – Learn from labeled data.
- **Unsupervised Learning** – Find patterns in unlabeled data.
- **Reinforcement Learning** – Learn by interacting with an environment.



Search in document...

Chat with PDF

+ New Chat

What are the types of machine learning? 10:30 AM

The three main types of machine learning are:

1. Supervised Learning – The model is trained on labeled data, where the input and output are known.
2. Unsupervised Learning – The model finds patterns and structures in data without labeled responses.
3. Reinforcement Learning – The model learns by interacting with an environment and receiving rewards or penalties.

These are explained in detail in Section 3.2 of the document.

Sources: Page 12 | Page 13 | Page 14

Can you explain supervised learning with an example? 10:32 AM

In supervised learning, the model learns from labeled examples. For example, in a spam email detection system, the model is trained on emails labeled as “spam” or “not spam” and learns to predict the label for new emails.

Source: Page 15

Ask a question about your PDF...

Sources

Page 12 Score: 0.92
Machine Learning can be broadly classified into three types: Supervised Learning, Unsupervised Learning and Reinforcement Learning.

Page 13 Score: 0.89
In Supervised Learning, the model is trained on labeled data. The algorithm learns a mapping from input to output based on example input-output pairs.

Page 14 Score: 0.85
Unsupervised Learning deals with unlabeled data. The goal is to find hidden patterns or intrinsic structures in the data.

Page 15 Score: 0.83
Example of Supervised Learning: Spam email detection, image classification, house price prediction, medical diagnosis, etc.

WORKER AGENT

Autonomous Doers That Get the Job Done

WHAT IS A WORKER AGENT?

A Worker Agent is an autonomous agent that specializes in executing specific tasks or sub-tasks assigned by another agent (e.g., Orchestrator or Manager Agent). It focuses on "doing" rather than planning.

- Receives task
- Processes information
- Uses tools (if needed)
- Returns result

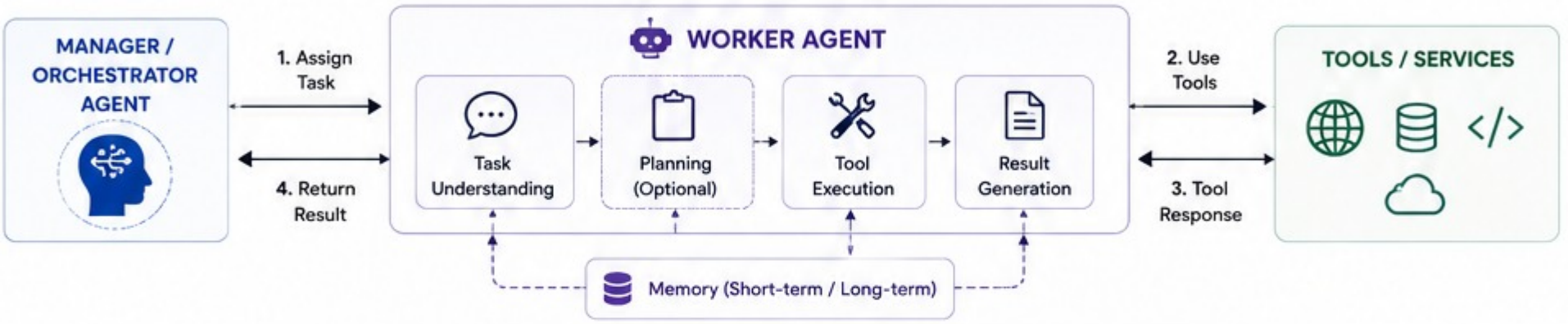
ROLE OF A WORKER AGENT

- Executes assigned tasks efficiently
- Uses tools, APIs, or functions
- Handles errors and edge cases
- Provides results in expected format
- Reports status back to the manager
- Can work in parallel with other workers

KEY CHARACTERISTICS

- **Specialized** – Focused on one type of task
- **Autonomous** – Works independently
- **Tool-Enabled** – Uses external tools/APIs
- **Reliable** – Consistent and error-aware
- **Scalable** – Many workers can run in parallel
- **Observable** – Logs and metrics for tracking

WORKER AGENT ARCHITECTURE



TYPES OF WORKER AGENTS

DATA WORKER AGENT

Works with data: fetch, clean, transform, aggregate, validate.

Example:
Extract sales data from DB and summarize.

RETRIEVAL WORKER AGENT

Searches and retrieves information from knowledge sources.

Example:
Find relevant docs from vector store.

CODE WORKER AGENT

Generates, runs, and tests code to solve problems.

Example:
Write Python code to analyze a file.

ANALYSIS WORKER AGENT

Performs analysis, calculations, and derives insights.

Example:
Compute KPIs and create insights.

INTEGRATION WORKER AGENT

Interacts with external APIs, services, and applications.

Example:
Send email via SMTP or post to Slack.

VALIDATION WORKER AGENT

Validates data, checks rules, and ensures quality.

Example:
Validate user input or data schema.

WORKER AGENT WORKFLOW (EXAMPLE)

1. **Receive Task** – Worker receives a task from the manager agent.
2. **Understand Task** – Understand goal, inputs, constraints and requirements.
3. **Plan (Optional)** – Break task into steps (if complex) and decide tools.
4. **Execute** – Use tools/APIs or logic to execute the task.
5. **Generate Result** – Process outputs and generate final structured result.
6. **Return & Report** – Send result back to manager with status, logs, or metadata.

SIMPLE WORKER AGENT EXAMPLE (PYTHON)

```python
from langchain.tools import tool
from langchain.agents import AgentExecutor, create_react_agent
from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate

@tool
def get_weather(city: str) -> str:
    """Get current weather for a city."""
    # Call to real API (example: OpenWeather)
    return f"Weather in {city} is sunny, 28°C"

tools = [get_weather]

prompt = PromptTemplate.from_template("""
    Answer the task using the tools.
    Tools: {tools}
    Question: {input}
    Thought  :{agent_scratchpad}
""")

llm = ChatOpenAI(temperature=0)
agent = create_react_agent(llm, tools, prompt)
worker = AgentExecutor(agent=agent, tools=tools, verbose=True)

result = worker.invoke({"input": "What is the weather in Delhi?"})
print(result['output'])  # Worker returns the result
```

COMMON USE CASES

- **Data Processing** – Extract, clean, and transform large datasets.
- **API Integration** – Fetch or send data using external APIs.
- **Document Processing** – Read, parse, summarize, and extract information.
- **Code Execution** – Run code, debug, and return results.
- **Web Research** – Search the web and collect relevant information.
- **Quality Check** – Validate outputs and ensure compliance with rules.
- **Report Generation** – Create summaries, reports, and presentations.
- **Notification / Automation** – Send alerts, emails, or trigger workflows.

BEST PRACTICES

- Give clear task description and expected output format.
- Provide necessary tools and handle errors gracefully.
- Keep worker focused on one responsibility (single purpose).
- Log inputs, tools used, and outputs for observability.
- Use retries and fallbacks for reliability.

BENEFITS

- Modular & reusable
- Easy to maintain
- Scalable & parallel
- Reliable & observable
- Specialized & efficient
- Works with any LLM

EXAMPLE TOOLS

Web Search · Database · File Reader · Python REPL · More

KEY TAKEAWAY: Worker Agents are the "doers" in an agent system. They take tasks, use tools, execute reliably, and return accurate results—helping the overall system achieve its goals efficiently.

MULTIMODAL GENERATIVE AI

Generating. Understanding. Creating. Across Every Modality

WHAT IS MULTIMODAL GEN AI?

Multimodal Generative AI models can understand, reason over, and generate content across multiple types of data (modalities) such as text, image, audio, video, and more. These models bridge different modalities to create richer, more natural, and context-aware outputs.

Text + Image + Audio + Video + 3D/Other

KEY CAPABILITIES

- Understand multiple input modalities
- Cross-modal reasoning & alignment
- Generate across one or more modalities
- Content transformation (e.g., text → image)
- Context-aware & highly interactive
- Support creative & productivity use cases

BENEFITS

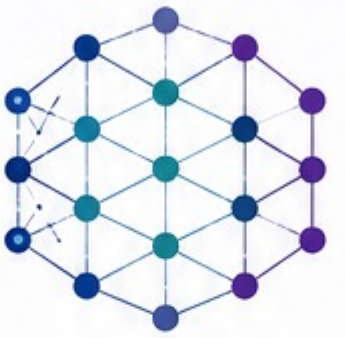
- More natural human-AI interaction
- Richer and more accurate outputs
- One model for many tasks
- Improved accessibility & inclusivity
- Higher productivity & automation
- Enables new creative possibilities

MULTIMODAL GENERATIVE AI ARCHITECTURE

MULTIMODAL INPUT
- Text
- Image
- Audio
- Video
- Sensors / 3D
- ...

INPUT ENCODERS
- Text Tokenizer
- Image Encoder
- Audio Encoder
- Video Encoder
- Other Encoders

MULTIMODAL FUSION & REASONING



Transformer / Diffusion / Hybrid
Alignment + Attention + Reasoning

GENERATIVE DECODER(S)
- Text Decoder
- Image Decoder
- Audio Decoder
- Video Decoder
- 3D / Other Decoder

MULTIMODAL OUTPUT
- Text
- Image
- Audio
- Video
- 3D / Other
- ...

FEEDBACK LOOP (Fine-tuning / RLHF / Human Feedback)

TYPES OF MULTIMODAL GEN AI MODELS

Text + Image	Image captioning, text-to-image, VQA, visual reasoning	Examples: BLIP, Flamingo, LLaVA, GPT-4V, Gemini
Text + Audio	Speech-to-text, audio captioning, TTS, music generation	Examples: Whisper, MusicLM, AudioLM, SpeechGPT
Text + Video	Video captioning, video QA, text-to-video generation	Examples: Video-LLaVA, Gen-2, Pika, Runway
Image + Audio	Image-to-audio, sound synthesis, audio visualizer	Examples: ImageBind, CLIP-Audio, Make-An-Audio
Text + Image + Audio + Video	Fully multimodal understanding and generation	Examples: GPT-4o, Gemini 1.5, Emu, NExT-GPT

TYPICAL WORKFLOW

1. Provide multimodal input
2. Model encodes each modality and aligns representations
3. Model reasons and generates across modalities
4. Decoders generate the required outputs
5. User reviews and interacts; feedback improves the model

COMMON USE CASES

Content Creation
Generate images, videos, music, and text together

AI Assistants
Understand voice, image, and text to provide smart responses

Education
Explain concepts using text, images, and voice

Healthcare
Analyze medical images, reports, and generate summaries

Marketing & Advertising
Create multi-format campaigns with consistent context

Accessibility
Convert between text, speech, images, and video for inclusion

MULTIMODAL INPUT → OUTPUT EXAMPLES

Text → Image
"A futuristic city at sunset"

Image → Text

"A serene lake surrounded by mountains and trees."

Audio → Text

"Meeting starts at 10 AM and budget is approved."

Text + Image → Answer
What is shown in this image?

"Golden Gate Bridge in San Francisco."

Text → Video
"A cat playing with a ball"

Text + Image → New Image
"Turn this sketch into a colorful painting"

EMERGING TRENDS

- Real-time multimodal understanding
- Long-context multimodal models
- 3D & spatial understanding
- Multimodal agents & tool use
- Personalized multimodal creation
- Multimodal memory & world models

POPULAR TOOLS & LIBRARIES

OpenAI · Google Gemini · Hugging Face Transformers · LangChain · LlamaIndex · Whisper · DALL·E · Stable Diffusion · PyTorch

CHALLENGES

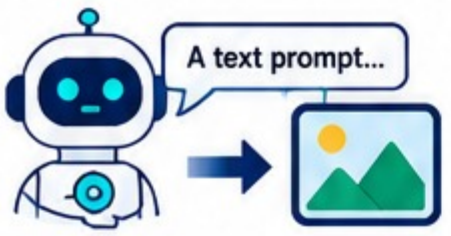
- High computational and memory requirements
- Data alignment across modalities is complex
- Bias and fairness across different data types
- Hallucinations and inconsistent outputs
- Evaluation and benchmarking are difficult
- Privacy and security concerns

BEST PRACTICES

- Start simple and iterate
- Use high-quality, diverse multimodal data
- Align modalities effectively
- Evaluate with human feedback
- Leverage transfer learning & pre-trained models
- Monitor outputs and handle edge cases

KEY TAKEAWAY

Multimodal Generative AI unlocks the power of all types of data—enabling AI to understand and create like never before.



TEXT TO IMAGE

Turn Your Words Into Visuals

Describe. Imagine. Generate.

Same Prompt
Infinite Possibilities

WHAT IS TEXT TO IMAGE?

Text to Image is a type of Generative AI that creates realistic or artistic images from natural language descriptions (text prompts) using deep learning models like diffusion models.

HOW IT WORKS?

1. You give a text prompt.
2. The text is converted into embeddings (using a text encoder like CLIP).
3. A diffusion model generates an image step-by-step from noise.
4. The final image is produced based on the prompt (and optional settings).

KEY FEATURES

- High-quality, realistic and artistic outputs
- Supports various styles (photo, anime, 3D, sketch, etc.)
- Can follow detailed instructions
- Supports negative prompts (to avoid undesired elements)
- Widely used in art, design, marketing, education, etc.
- Models: DALL·E, Stable Diffusion, Midjourney, Flux, etc.

TEXT TO IMAGE PROCESS

Text Prompt
"A peaceful mountain lake at sunset"

→ **Text Encoder** (CLIP, T5, etc.)

→ **Text Embedding**
0.12 -0.43 0.98 ...

→ **Diffusion Model** (Generative Model)

→ **Denoising Process** (From noise to image)

→ **Generated Image**

EXAMPLE OUTPUTS

Prompt
"A golden retriever wearing sunglasses at the beach"

Prompt
"A futuristic city with flying cars, cinematic style"

Prompt
"A cozy reading corner with books, warm lights, realistic photo"

Prompt
"Anime girl with headphones, cyberpunk style"

Prompt
"A cute panda in a bamboo forest, 4k, realistic"

Prompt
"A pencil sketch of a castle on a hill"

PYTHON EXAMPLE (Stable Diffusion)

```python
# Install libraries (if not installed)
!pip install diffusers transformers accelerate

from diffusers import StableDiffusionPipeline
import torch

# Load model
model_id = "runwayml/stable-diffusion-v1-5"
pipe = StableDiffusionPipeline.from_pretrained(
    model_id, torch_dtype=torch.float16
)
pipe = pipe.to("cuda")  # use "cpu" if no GPU

# Generate image
prompt = "A peaceful mountain lake at sunset"
image = pipe(prompt, num_inference_steps=30).images[0]
image.save("text_to_image_output.png")
image.show()
```

PROMPT TIPS

- **Be specific** — Detailed prompts give better results.
- **Use style keywords** (e.g., realistic, cinematic, anime, watercolor)
- **Specify details** — Include colors, lighting, camera angle, etc.
- **Use negative prompts** (e.g., low quality, text, blurry)
- **Experiment** — Try different prompts and settings.
- **Iterate** — Refine your prompt to get the desired result.

POPULAR MODELS

- **DALL·E 3** — High-quality, easy to use
- **Midjourney** — Artistic and creative
- **Stable Diffusion** — Open-source, customizable
- **Flux** — High realism and detail
- **Llama 3 + SD** — Emerging open models

COMMON USE CASES

- **Art & Design** — Concept art, illustrations
- **Marketing** — Ads, product visuals
- **Education** — Diagrams, visual explanations
- **Entertainment** — Storyboarding, character design
- **Prototyping** — UI/UX mockups, ideas to visuals

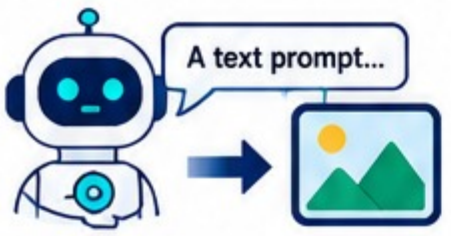
CHALLENGES

- May generate incorrect details
- Struggles with complex text (e.g., text in images)
- Biases in training data
- Requires good prompt engineering
- High compute resources for training

KEY TAKEAWAY

A good prompt can turn your imagination into reality.

Right Prompt. Big Possibilities.



TEXT TO IMAGE

Turn Your Words Into Visuals

Describe. Imagine. Generate.

Same Prompt
Infinite Possibilities

WHAT IS TEXT TO IMAGE?

Text to Image is a type of Generative AI that creates realistic or artistic images from natural language descriptions (text prompts) using deep learning models like diffusion models.

HOW IT WORKS?

1. You give a text prompt.
2. The text is converted into embeddings (using a text encoder like CLIP).
3. A diffusion model generates an image step-by-step from noise.
4. The final image is produced based on the prompt (and optional settings).

KEY FEATURES

- High-quality, realistic and artistic outputs
- Supports various styles (photo, anime, 3D, sketch, etc.)
- Can follow detailed instructions
- Supports negative prompts (to avoid undesired elements)
- Widely used in art, design, marketing, education, etc.
- Models: DALL·E, Stable Diffusion, Midjourney, Flux, etc.

TEXT TO IMAGE PROCESS

Text Prompt
"A peaceful mountain lake at sunset"

→ **Text Encoder** (CLIP, T5, etc.)

→ **Text Embedding**
0.12 -0.43 0.98 ...

→ **Diffusion Model** (Generative Model)

→ **Denoising Process** (From noise to image)

→ **Generated Image**

EXAMPLE OUTPUTS

Prompt
"A golden retriever wearing sunglasses at the beach"

Prompt
"A futuristic city with flying cars, cinematic style"

Prompt
"A cozy reading corner with books, warm lights, realistic photo"

Prompt
"Anime girl with headphones, cyberpunk style"

Prompt
"A cute panda in a bamboo forest, 4k, realistic"

Prompt
"A pencil sketch of a castle on a hill"

PYTHON EXAMPLE (Stable Diffusion)

```python
# Install libraries (if not installed)
!pip install diffusers transformers accelerate

from diffusers import StableDiffusionPipeline
import torch

# Load model
model_id = "runwayml/stable-diffusion-v1-5"
pipe = StableDiffusionPipeline.from_pretrained(
    model_id, torch_dtype=torch.float16
)
pipe = pipe.to("cuda")  # use "cpu" if no GPU

# Generate image
prompt = "A peaceful mountain lake at sunset"
image = pipe(prompt, num_inference_steps=30).images[0]
image.save("text_to_image_output.png")
image.show()
```

PROMPT TIPS

- **Be specific** — Detailed prompts give better results.
- **Use style keywords** (e.g., realistic, cinematic, anime, watercolor)
- **Specify details** — Include colors, lighting, camera angle, etc.
- **Use negative prompts** (e.g., low quality, text, blurry)
- **Experiment** — Try different prompts and settings.
- **Iterate** — Refine your prompt to get the desired result.

POPULAR MODELS

- **DALL·E 3** — High-quality, easy to use
- **Midjourney** — Artistic and creative
- **Stable Diffusion** — Open-source, customizable
- **Flux** — High realism and detail
- **Llama 3 + SD** — Emerging open models

COMMON USE CASES

- **Art & Design** — Concept art, illustrations
- **Marketing** — Ads, product visuals
- **Education** — Diagrams, visual explanations
- **Entertainment** — Storyboarding, character design
- **Prototyping** — UI/UX mockups, ideas to visuals

CHALLENGES

- May generate incorrect details
- Struggles with complex text (e.g., text in images)
- Biases in training data
- Requires good prompt engineering
- High compute resources for training

KEY TAKEAWAY

A good prompt can turn your imagination into reality.

Right Prompt. Big Possibilities.

LOCAL LLMs

Powerful AI. 100% Private. Runs on Your Machine.

Local LLMs are large language models that run entirely on your local device (CPU, GPU or NPU) without relying on cloud APIs. You get privacy, control, offline access and zero recurring costs.

KEY BENEFITS

- Privacy & Data Security
- Works Offline
- No API Costs
- Full Control & Customization
- Low Latency
- Run on Your Hardware

USE CASES

- Private Chatbots
- Document Analysis & QA
- Code Generation
- Writing & Summarization
- Research Assistant
- Automation & Productivity

HOW LOCAL LLMs WORK

1 Input (Prompt)
You type your question or give an instruction.

Explain quantum computing in simple terms.

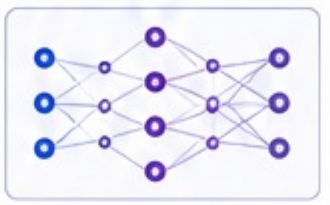
→

2 Tokenization
The text is converted into tokens (numbers) that the model understands.

1015 | 205 | 2025 | 102 | ...

→

3 Model Processing
The local LLM processes tokens using its neural network layers.



→

4 Decoding
The model generates tokens one by one (auto-regressively).

5021 | 309 | 2001 | 5021 | ...

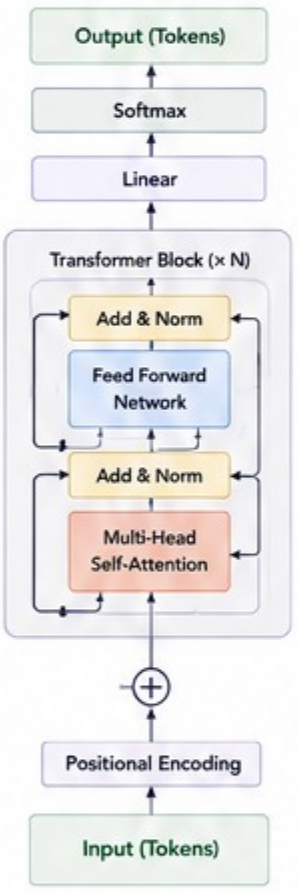
→

5 Output (Response)
The tokens are converted back to human-readable text.

Quantum computing uses qubits... to solve problems...

All of this happens on your local machine — your data never leaves your device.

LOCAL LLM ARCHITECTURE



Input Embedding
Converts tokens into dense vectors.

Multi-Head Self-Attention
Allows the model to focus on different parts of the input.

Feed Forward Network
Processes features and adds non-linearity.

Add & Norm
Residual connection and normalization for stable training.

Linear + Softmax
Maps to vocabulary and predicts next token.

POPULAR LOCAL LLMs

Llama 3 (8B / 7B / 3B / 1B)
High quality, strong reasoning, long context, open weights.

Mistral (7B)
Efficient, fast and great for general tasks.

Phi-3 (3.8B / 7B / 14B)
Small, fast, good performance on many tasks.

Gemma 2 (2B / 9B / 27B)
Google's open models, great quality.

Qwen 2 (0.5B – 72B)
Wide range of sizes, multilingual & strong.

DeepSeek (6.7B / 33B)
Great coding & reasoning capabilities.

... and many more (TinyLlama, StableLM, Falcon, etc.)

RUN LOCAL LLMs ANYWHERE

Hardware Options

High-End GPU
(NVIDIA RTX 3060 / 4070 / 4090)

Apple Silicon (M1 / M2 / M3)
Unified memory advantage

CPU Only
Works with smaller models (slower but possible)

Supported Platforms

Windows | macOS | Linux

Popular Runtimes / Tools

Ollama
Run & manage models easily

LM Studio
GUI for local LLMs

GPT4All
Privacy-first desktop app

text-generation-webui
Feature-rich Web UI

llama.cpp
Run models efficiently on CPU/GPU

vLLM
High-throughput inference engine

SYSTEM REQUIREMENTS (GUIDELINE)

Model Size (Parameters)	VRAM / RAM (Minimum)	Use Case
1B – 3B	4 – 8 GB	Basic chat, summarization
7B	8 – 16 GB	General purpose tasks
13B – 14B	16 – 24 GB	Better quality, long context
30B – 70B	24 – 80+ GB	Advanced reasoning, coding

More VRAM/RAM = Better performance, longer context, higher quality.

EXAMPLE: RUN LLAMA 3 WITH OLLAMA

```
# 1. Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# 2. Pull a model
ollama pull llama3:8b

# 3. Run the model
ollama run llama3:8b

# 4. Chat with the model
>>> Explain photosynthesis in simple terms.

Photosynthesis is the process used by plants...
>>>
```

That's it! You're chatting with a local LLM.

LOCAL LLMs vs CLOUD LLMs

LOCAL LLMs	CLOUD LLMs
Your data stays local	Data leaves your device
Works offline	Requires internet
No usage limits	Usage limits / quotas
One-time setup cost	Recurring API costs
Full customization	Limited customization
Lower latency	Higher latency

Choose Local LLMs when privacy, control and cost-efficiency matter most.

BEST PRACTICES

- Use a GPU with more VRAM for better performance.
- Choose the right model size for your hardware.
- Quantization (Q4, Q5, Q8) helps reduce memory usage.
- Keep your drivers, runtimes and tools updated.
- Use system prompts to guide model behavior.
- Backup models and configs for reproducibility.

CHALLENGES

- Requires capable hardware
- Large models need a lot of memory
- Slower on CPU compared to cloud
- Model quality varies by size
- Setup & configuration complexity

KEY TAKEAWAY

Local LLMs put the power of AI in your hands — private, offline, and under your control.

Pick the right model, the right tool, and unleash the power locally!

Hugging Face

The AI community platform

Build, train, and deploy Machine Learning models with open-source tools, datasets, and models.

WHY HUGGING FACE?

- ✔ Largest hub of open-source models & datasets
- ✔ Easy to use libraries and tools
- ✔ Train, fine-tune and deploy models
- ✔ Collaborate with the community
- ✔ Free and open-source

WHAT IS HUGGING FACE?

Hugging Face is an AI community platform with tools and resources to build, train and deploy ML models.

Models · Datasets · Spaces · Hub

KEY FEATURES

- Access to 500K+ models and datasets
- State-of-the-art NLP, CV, Audio & more
- Powerful libraries: Transformers, Datasets, Diffusers, PEFT, Accelerate, etc.
- Collaborative model hosting (Hub)
- Deploy demos with Spaces
- Easily integrate with your workflow

POPULAR USE CASES

- Natural Language Processing
- Computer Vision
- Speech & Audio Processing
- Generative AI (Text, Image, Video)
- Chatbots & Assistants
- Research & Prototyping

HUGGING FACE ECOSYSTEM: HOW IT WORKS

1 Explore

Search for models, datasets, and other resources on the Hub.

bert-base-uncased
by google
125M downloads

2 Load

Use 🤗 Transformers to load a model with a few lines of code.

```
from transformers
import pipeline

nlp = pipeline(
 "sentiment-analysis")
```

3 Fine-tune

Fine-tune models on your custom datasets.

Your Dataset
CSV · JSON · TXT

4 Deploy

Deploy your model as an interactive demo using Spaces.

Spaces Demo

5 Share

Share your model, dataset or Space with the community.

Model Card
Like 127
Downloads 2.3M

GET STARTED: QUICK EXAMPLE (NLP)

```
# 1. Install
pip install transformers

# 2. Load a pre-trained model
from transformers import pipeline
nlp = pipeline("sentiment-analysis")

# 3. Run inference
result = nlp("Hugging Face is amazing!")
print(result)

# Output
[{'label': 'POSITIVE', 'score': 0.999}]
```

One line is all you need to run a powerful pre-trained model!

POPULAR LIBRARIES

Transformers	State-of-the-art models for NLP, CV, Audio & more
Datasets	Load and process datasets efficiently
Diffusers	Create images with Stable Diffusion and other models
PEFT	Parameter-Efficient Fine-Tuning (LoRA, QLoRA, etc.)
Accelerate	Train models faster on any hardware
Evaluate	Evaluate model performance with ease

HUGGING FACE HUB

The Hub is a central place to discover, host and collaborate on ML assets.

- **Models** – Pre-trained & fine-tuned models for every task
- **Datasets** – Public and private datasets for any domain
- **Spaces** – Host ML demos and build web apps
- **Model Cards** – Document your model with all details
- **Discussions** – Connect and collaborate with the community

WHAT CAN YOU DO?

Text Generation

Once upon a time in a land far away...

Use models like GPT-2, Llama 3, Mistral

Image Generation

Generate stunning images with Stable Diffusion, SDXL, etc.

Audio / Speech

Transcribe speech, text-to-speech and audio generation.

Question Answering

Extract answers from context using powerful models.

Zero-shot Classification

POSITIVE
NEUTRAL
NEGATIVE

Classify text without training using zero-shot learning.

HUGGING FACE SPACES

Turn your ML models into interactive apps in minutes!

Sentiment Analysis Demo

Enter text below

Hugging Face makes ML easy!

Analyze

POSITIVE — Score: 0.998

- No need to manage servers
- Share with a link
- Supports Gradio, Streamlit & more

POPULAR MODELS ON THE HUB

mistralai/Mistral-7B-Instruct-v0.2	7B	⬇ 2.1M
meta-llama/Meta-Llama-3-8B-Instruct	8B	⬇ 3.4M
stabilityai/stable-diffusion-xl-base-1.0	2.6B	⬇ 4.2M
google/bert-base-uncased	110M	⬇ 5.6M
openai/whisper-large-v3	1.6B	⬇ 1.8M

... and thousands more!

USE HUGGING FACE IN PYTHON

```
from transformers import AutoTokenizer, AutoModel

model_name = "mistralai/Mistral-7B-Instruct-v0.2"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

inputs = tokenizer("Hello Hugging Face!", return_tensors="pt")
outputs = model.generate(**inputs, max_new_tokens=50)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

Perfect for research, prototyping and production!

JOIN THE COMMUNITY

- **Forums** discuss.huggingface.co
- **Discord** hf.co/join/discord
- **GitHub** github.com/huggingface
- **X (Twitter)** @huggingface

Open-source. Community-driven. Built for everyone.

huggingface.co

PROMPT VS QUERY

Understanding the Key Difference

PROMPT (for LLMs / Generative AI)

A prompt is an **instruction** or input given to an AI model to **generate** a response, content, or **output**.

Purpose	Guide the AI to generate, create, explain, summarize, or perform a task.
Nature	Open-ended, flexible, and natural language.
Outcome	Generates new content or responses based on understanding.
Used In	LLMs / Generative AI tools (ChatGPT, Claude, Gemini, etc.)
Example	*"Explain the theory of relativity" in simple terms."*

Purpose	
Nature	
Outcome	
Used In	
Example	

QUERY (for Search / Databases)

A query is a request for **specific information** from a search engine or database.

Purpose	Find specific data, records, or information from a source.
Nature	Specific, structured, and often uses keywords or syntax.
Outcome	Retrieves existing data or results from the source.
Used In	Search engines / Databases (Google, SQL, Elasticsearch, etc.)
Example	*"What is the population of Japan in 2024?"*

PROMPT EXAMPLES (LLM)

- Write a short story about a time traveler.
- Summarize the benefits of exercise.
- Translate this paragraph into French.
- Explain quantum computing in simple terms.
- Generate ideas for a YouTube channel about AI.

KEY TAKEAWAY

Use a **Prompt** when you want the AI to think, create or explain.

Use a **Query** when you want to find specific facts or data.

QUERY EXAMPLES (SEARCH / DB)

- weather in Delhi today
- best laptops under 60000
- SELECT * FROM employees WHERE dept = 'HR';
- documents about climate change PDF
- population of Canada 2024

Aspect	PROMPT (LLM)	QUERY (Search / DB)
Flexibility	High – Can be conversational and creative	Low – Needs precise terms/syntax
Creativity	High – Generates new ideas and content	None – Returns existing data only
Context Understanding	Understands context and nuance	Limited to keywords and filters
Examples of Tools	ChatGPT, Claude, Gemini, Llama, Mistral	Google Search, SQL, MongoDB, Elasticsearch

WHEN TO USE WHAT?

Use **PROMPT** for brainstorming, writing, learning, coding help, summarization, and creative tasks.

Use **QUERY** for finding facts, data analysis, records, documents, products, or answers that already exist.

GOOD PROMPT VS BAD PROMPT

The difference is in the details.

GOOD PROMPT Clear, specific, and well-structured		BAD PROMPT Vague, unclear, and unstructured
Explain the concept of **blockchain technology** in simple terms for a beginner.	**CLARITY** Be clear and specific	Tell me about blockchain.
You are a **marketing expert**. I run an **online store** that sells **eco-friendly products**. Write a social media post to promote our new **bamboo toothbrush**.	**CONTEXT** Provide relevant background information	Write a post for my product.
List **5 benefits** of meditation for **mental health** and explain each in **one or two sentences**.	**SPECIFICITY** Be detailed about the output you want	Tell me the benefits of meditation.
Write a **300-word** blog introduction about the importance of sleep. Use a **friendly tone** and **avoid medical jargon**.	**CONSTRAINTS** Set limits or preferences	Write about sleep.
Create a **step-by-step** guide on how to change a car tire. Format it as a **numbered list** with **clear, short steps**.	**STRUCTURE** Specify the format or structure	How do I change a tire?
Give **3 examples** of **renewable energy sources** and explain how each one works briefly.	**EXAMPLES** Use examples to clarify your request	Give examples of renewable energy.

KEY TAKEAWAY

- Good prompts get better, more accurate, and more useful responses.
- Provide detail, context, and clear instructions for the best results.

Golden Formula for a Good Prompt:

Role/Context + Task + Details + Format + Tone (Optional)